ARM PrimeCell

Synchronous Static Memory Controller (PL093)

Revision: r0p3-00rel0

Design Manual

Confidential

ARM PrimeCell Synchronous Static Memory Controller (PL093)

Revision: r0p3-00rel0

Design Manual

Proprietary notice

Document confidentiality status

This document is Confidential. THis document can only be distributed according to the terms set out in your license agreement.

Product status

The information in this document is Final (information on a developed product).

Web address

http://www.arm.com/

Feedback

ARM Limited welcomes feedback on both the product, and the documentation.

Feedback on this document

If you have any comments about this document, please send email to errata@arm.com giving:

- the document title
- the document number
- the page number(s) to which your comments refer
- a concise explanation of your comments

General suggestion for additions and improvements are also welcome.

Feedback on the ARM PrimeCell

If you have any comments or suggestions about this product, please contact your supplier giving:

- the Product name
- a concise explanation of your comments.

Contents

1 ABOUT THIS DOCUMENT

1.1 Change history

1.1.1 Current status and anticipated changes

This document has been reviewed and the review comments have been incorporated.

1.1.2 Change history

Issue	Date	Change
A01	10th June, 2001	First Draft
Rev.0	4 July 2002	Final Version
B	11 March 2003	Minor change to section 8.1.34 for release r0p1-00ltd0
C	4 March 2004	Update for r0p2-00rel0 release
D	8 Oct 2004	Update for r0p3-00rel0 release

1.2 References

This document refers to the following documents.

Ref.	Doc. No.	Title
1	ARM IHI 0011	AMBA Specification Rev2.0
2	ARM DDI 0236	ARM PrimeCell Synchronous Static Memory Controller (PL093) Technical Reference Manual
3	PL093 INTM 0000	ARM PrimeCell Synchronous Static Memory Controller (PL093) Integration Manual

1.3 Terms and abbreviations

This document uses the following terms and abbreviations.

Term	Meaning
AHB	Advanced High performance Bus, an AMBA bus designed for high-performance peripherals.
AHB master	A device that can send out requests over an AHB bus.
AHB slave	A device that fulfils requests provided by an AHB master.
AMBA	Advanced Microcontroller Bus Architecture.
SSMC	Synchronous Static Memory Controller.
Mux	Multiplexer
DBI	Internal Data Bus Interface
TSM	Transfer State Machine
EBI	External Bus Interface

TIC	Test Interface Controller

1.4 Conventions used in this document

1.4.1 Signals

- Signal names are shown in bold font type.
- Active low signals are prefixed by a lower case ‘n’. In the case of AHB signals by ‘Hn’, or postfixed by ‘n’.
- AHB signals are prefixed by an upper case ‘H’.
- When a signal is described as being ‘Asserted’, the actual level depends on whether the signal is active HIGH or active LOW. ‘Asserted’ means HIGH for active high signals and LOW for active low signals.

1.4.2 Bytes, Half-words and Words

- A byte is 8 bits.
- A half-word is two bytes i.e. 16 bits.
- A word is 4 bytes i.e. 32 bits.

1.4.3 Bits, bytes, K and M

- The suffix ‘B’ (upper case) is used to indicate BYTES.
- The suffix ‘b’ (lower case) is used to indicate BITS.
- When used to indicate an amount of memory, the suffix ‘k’ means 1024.
- When used to indicate a frequency, the suffix ‘k’ means 1000.
- When used to indicate an amount of memory, the suffix ‘M’ means 1024^2 = 1048576.
- When used to indicate a frequency, the suffix ‘M’ means 1,000,000.

2 SCOPE

This document describes the design and implementation of the ARM PrimeCell PL093 SSMC (Synchronous Static Memory Controller). The design description is split between three sections - Section 4, Section 5 and Section 6. The testbenches and the test programs for functional verification and integration testing are described in Section 7 and Section 8.

Section 4 presents an overview of the SSMC design, describing the functional partitioning of the SSMC and the signal interconnections. Section 5 presents detailed descriptions on the SSMC design. Section 6 describes the design assumptions made. Section 7 describes the testbench and the test programs used to generate test vectors for functional verification of the SSMC. Section 8 describes the testbenches and the test programs used to generate test vectors for Integration Testing of the SSMC.

3 INTRODUCTION

The SSMC is a part of a library of reusable IP blocks, which can be more easily integrated into a typical ASIC design flow.

For further information on this block refer to the ARM SSMC (PL093) Technical Reference Manual [Ref.2]. The ARM PrimeCell SSMC (PL093) Integration Manual [Ref.3] describes how the block may be integrated into a typical industry standard ASIC design flow.

4 SSMC DESIGN OVERVIEW

The Synchronous Static Memory Controller (SSMC) is an Advanced Microcontroller Bus Architecture (AMBA) block that connects to the Advanced High-performance Bus (AHB). There is one AHB slave interface for programming the SSMC and one AHB slave interfaces for accessing the memory devices that are connected to the SSMC. The SSMC (PL093) controls memory devices connected to 8 chip selects.

4.1 Signal Description

The following tables describe the synchronous static memory controller interface signals. ***Tables 4.1-1*** and ***Table 4.1-2*** list the AHB slave memory interface and register interface signals, while ***Tables 4.1-3*** and ***Table 4.1-4*** list the master interface signals. ***Table 4.1-5*** lists the clocks in the SSMC. ***Tables 4.1-6*** and ***Table 4.1-7*** list the SSMC Pad Interface and Pad Control signals. ***Tables 4.1-8*** and ***Table 4.1-9*** list the miscellaneous signals and the scan test related signals in the SSMC.

For more information on the AHB, please see AMBA Specification Rev2.0 [Ref. 1].

Signal Name	Type	Source / Destination	Description
HCLK Bus clock	Input	Clock Source	This clock times all bus transfers. All signal timings are related to the rising edge of HCLK.
HRESETn Reset	Input	Reset controller	The Bus reset signal is active LOW and is used to reset the system and the bus. This is the only active LOW signal.
HADDRSMC[25:0] Address bus	Input	AHB Master	The AHB address bus to the AHB Memory interface.
HWRITESMC Transfer direction	Input	AHB Master	When HIGH this signal indicates a write transfer and when LOW, a read transfer to the memory.
HSIZESMC[2:0] Transfer size	Input	AHB Master	Indicates the size of the transfer to the Memory. The SSMC AHB Slave Memory Interface supports only 8-bit, 16-bit and 32-bit accesses '000' represents 8-bit '001' represents 16-bit '010' represents 32-bit. If HSIZEx[2:0] indicates a transfer width larger than 32-bits, the SSMC returns an ERROR response on HRESPSMC[1:0]
HTRANSSMC[1:0] Transfer type	Input	AHB Master	Indicates whether a data-transaction is to be done for the current transfer to Memory. '00' represents IDLE. '01' represents BUSY. '10' represents NSEQ '11' represents SEQ
HBURSTSMC[2:0] Burst Type	Input	AHB Master	Indicates if the transfer forms part of a burst. 4, 8 and 16 beat bursts are supported and the burst can be either incrementing or wrapping.
HREADYINSMC Transfer done	Input	AHB Slave	When HIGH the HREADYIN signal indicates that a transfer has finished on the bus. When LOW, it indicates that the AHB Slave has introduced a wait state.
HSELSMC[7:0] Chip select-specific select	Input	AHB Decoder	Each SSMC AHB slave memory interface has its own set of chipselect -specific select signals. When HIGH, the relevant bit of HSELSSM [7:0] indicates that the current transfer is intended for a memory chip connected to a particular chip select output of the SSMC corresponding to that bit. For example, if bit 0 is asserted during an AHB transfer, it indicates

Signal Name	Type	Source / Destination	Description
			that the transfer is intended for chip select 0. This signal is a combinatorial decode of the address bus.
HWDATASMC[31:0] Write data bus	Input	AHB Master	The write data bus is used to transfer data from the master to the SSMC AHB Slave Memory Interface during write operations. The SSMC AHB Slave Memory Interface supports only 8-bit, 16-bit and 32-bit accesses.
HRDATASMC[31:0] Read data bus	Output	AHB Master	The read data bus is used to read data from the SSMC AHB Slave Memory Interface. The SSMC AHB Slave Memory Interface supports only 8, 16 and 32-bit accesses.
HREADYOUTSMC Transfer done	Output	AHB Master	When HIGH the HREADYOUTSMC signal indicates that a transfer targeting the SSMC AHB Slave Memory Interface has finished on the bus. When LOW, it indicates that a wait state has been introduced by the AHB Slave
HRESPSMC[1:0] Transfer response	Output	AHB Slave	The transfer response provides additional information on the status of a transfer. Defined responses from AHB slave are, OKAY, ERROR, RETRY and SPLIT. The SSMC AHB Slave Memory Interface only drives OKAY or ERROR response.

Table 4.1-1 AHB Slave Memory Interface signals

Signal Name	Type	Source / Destination	Description
HADDRREG[11:2] Address bus	Input	AHB Master	The system address bus to the AHB register interface.
HWRITEREG Transfer direction	Input	AHB Master	When HIGH this signal indicates a write transfer and when LOW, a read transfer to the SSMC registers.
HSIZEREG[2:0] Transfer size	Input	AHB Master	Indicates the size of the transfer. **The SSMC supports only 32 bit WORD accesses to all its registers.** So all transfers to and from the SSMC controller must be 32-bit (HSIZE [2:0] = '010'). Transfer sizes other than 32-bit generate an ERROR response.
HTRANSREG Transfer type	Input	AHB Master	Indicates whether a data-transaction is to be done for the current transfer to the SSMC Registers. HTRANS [1] on the AHB bus must be connected to the single-bit-wide HTRANSREG input of the SSMC. '1' represents NSEQ or SEQ and thus a valid data transfer. '0' represents BUSY or IDLE.
HREADYINREG Transfer done	Input	AHB Slave	When HIGH the HREADYINREG signal indicates that a transfer has finished on the bus. When LOW, it indicates that a wait state has been introduced by the AHB Slave
HSELREG AHB Slave select	Input	AHB Decoder	The SSMC AHB slave register interface has its own slave select signal. This signal indicates that the current transfer is intended for the registers of the SSMC. This signal is a combinatorial decode of the address bus.
HWDATAREG[31:0] Write data bus	Input	AHB Master	The write data bus is used to transfer data from the master to the SSMC AHB Slave Register Interface during write operations. The SSMC AHB Slave Register Interface supports only 32-bit accesses.
HRDATAREG[31:0] Read data bus	Output	AHB Master	The read data bus is used to read data from the SSMC AHB Slave Register Interface. The SSMC AHB Slave Register Interface supports only 32-bit accesses

Signal Name	Type	Source / Destination	Description
HREADYOUTREG Transfer done	Output	AHB Master	When HIGH the HREADYOUTREG signal indicates that a transfer targeting the SSMC AHB Slave Register Interface has finished on the bus. When LOW, it indicates that a wait state has been introduced by the AHB Slave
HRESPREG[1:0] Transfer response	Output	AHB Slave	The transfer response provides additional information on the status of a transfer. Defined responses from AHB slave are, OKAY, ERROR, RETRY and SPLIT. The SSMC AHB Slave Register Interface only drives OKAY or ERROR response.

Table 4.1-2 AHB Slave Register Interface signals

Signal Name	Type	Source / Destination	Description
HADDRTIC[31:0] Address bus	Output	AHB Slave	The 32-bit system address bus.
HTRANSTIC[1:0] Transfer type	Output	AHB Slave	Indicates the type of the current transfer, which can be NONSEQ, SEQ, IDLE or BUSY.
HWRITETIC Transfer direction	Output	AHB Slave	When HIGH this signal indicates a write transfer and when LOW a read transfer.
HSIZETIC[2:0] Transfer size	Output	AHB Slave	Indicates the size of the transfer. The SSMC's TIC allows 8, 16 and 32-bit transfer widths. 8-bit: HSIZE[2:0] = '000' 16-bit: HSIZE[2:0] = '001' 32-bit: HSIZE[2:0] = '010'
HPROTTIC[3:0] Protection control	Output	AHB Slave	Provides information about a bus access.
HLOCKTIC Lock information for locked transfers	Output	AHB Arbiter	This signal is used to indicate to the arbiter that the requesting master is going to perform a number of indivisible transfers and the arbiter must not grant the bus to another bus master once the locked transfer has commenced.
HBURSTTIC[2:0] Burst type	Output	AHB Slave	Indicates the type of burst transfer. **Only INCR accesses are supported by the TIC Interface.** So HBURSTTIC [2:0] will always have a value of "001" in the SSMC.
HWDATATIC[31:0] Write data bus	Output	AHB Slave	The write data bus is used to transfer data from the master to the bus slaves during write operations.

Signal Name	Type	Source / Destination	Description
HRDATATIC[31:0] Read data bus	Input	AHB Slave	The read data bus is used to transfer data from bus slaves to the bus master during read operations.
HREADYINTIC Transfer done	Input	AHB Slave	When HIGH the HREADYINTIC signal indicates that a transfer has finished on the bus. This signal may be driven LOW to extend a transfer, by the AHB slave.
HRESPTIC[1:0] Transfer response	Input	AHB Slave	The transfer response provides additional information on the status of a transfer. Four different responses are supported: OKAY, ERROR, RETRY and SPLIT. To indicate that the transfer has completed successfully, the slave uses the OKAY response. To indicate that an error has occurred, the slave uses the ERROR response. To indicate that the data is not ready, the slave uses the RETRY and SPLIT responses.

Table 4.1-3 AHB Master signals from the TIC

Signal Name	Type	Source / Destination	Description
HBUSREQTIC AHB bus request	Output	AHB arbiter	Bus request signal used by the Test Interface controller to request access to the AHB bus. When HIGH, this signal indicates that the SSMC is requesting the ownership of the bus.
HGRANTTIC AHB bus grant	Input	AHB arbiter	Grant signal generated by the arbiter. This signal indicates that the SSMC is currently the highest priority master requesting the bus. The SSMC gains bus ownership when HGRANTTIC and HREADYINTIC are high on the rising edge of HCLK.

Table 4.1-4 AHB Master bus request signals

Signal Name	Type	Source / Destination	Description

Signal Name	Type	Source / Destination	Description
SMMEMCLK Memory clock	Input	Clock Source	SSMC Memory Clock. SSMCCLK times all external memory transfers. SSMCCLK must be synchronous to HCLK. SSMCCLK can operate at the following ratios with reference to HCLK: 1:1 2:1. 3:1. Used for Synchronous Memory devices.
nSMMEMCLK	Input	Clock Source	Inverted memory clock.
SMMEMCLKDELAY	Input	Clock Source	Delayed version of SSMC Memory Clock. This clock is used to drive out the control and data signals for Synchronous Memory.
SSMCFBCLKIN3...0 Fed-back clock in	Input	Clock Source	Fed-back clocks from the Synchronous Memory devices. There are 4 such clocks – one for each byte lane. Each clock is used to register one byte of the 32-bit data returned by the Synchronous Memory device.
SMCLK[3:0] Synchronous Memory clock out	Output	Synchronous memory devices	Memory clock output. Each bit of SMCLK[3:0] is to be connected to the clock input of Synchronous Memory devices.

Table 4.1-5 SSMC clock signals

Signal Name	Type	Source / Destination	Description
nSMBLS[3:0] SSMC byte lane select	Output	Pad	Byte lanes select signals. Active LOW. The signal nSMBLS[3:0] select byte lanes [31:24], [23:16], [15:8] and [7:0] on the data bus.
nSMOEN SSMC output enable	Output	Pad	Output enable for static memories. Active LOW.
nSMWEN SSMC write enable	Output	Pad	Write enable for memories. Active LOW.
nSMCS1…7 Static memory chip select	Output	Pad	Static memory chip selects. Default active LOW.
SMCS1…7 Static memory chip select	Output	Pad	Static memory chip selects. Default active HIGH.
SMADDR[25:0] Memory address	Output	Pad	Memory Address output.
SMBAA Address advance	Output	Pad	External burst address advance signal. Used to advance address in Synchronous burst memory device.
SMADDRVALID Address valid	Output	Pad	Address valid indication for Synchronous memories.
SMDATAOUT[31:0] Write data	Output	Pad	Data output to memory. Used for the static memory controller, the synchronous memory controller and the Test Interface

Signal Name	Type	Source / Destination	Description
			Controller (TIC).
SMDATAIN[31:0] Read data	Input	Pad	Read data from memory. Used for the static memory controller, the synchronous memory controller and the Test Interface Controller (TIC).
SMTESTREQA Test bus request A in test mode	Input	TicBox	During test, this signal is used to indicate the type of test vector that will be applied in the following cycle. The signal gives indication that the test bus has been granted and completion of a test access. When SMTESTACK is LOW, the current test vector must be extended until SMTESTACK becomes HIGH.
SMTESTREQB Test bus request B in test mode	Input	TicBox	During test, this signal is used, in combination with SMTESTREQA, to indicate the type of test vector that will be applied in the following cycle. The signal gives indication that the test bus has been granted and completion of a test access. When SMTESTACK is LOW, the current test vector must be extended until SMTESTACK becomes HIGH.
SMTESTACK Test Mode	Output	Pad	Test bus acknowledge.

Table 4.1-6 Pad Interface signals

Signal Name	Type	Source / Destination	Description
nSSMCDATAEN[3:0] Data enable	Output	Pad control	Tri-State I/O pad enables for the byte lanes of the external memory data bus SSMCDATA[31:0]. Active LOW. Each bit enables the byte lanes [31:24], [23:16], [15:8] and [7:0] of the data bus independently. Used for the static memory controller and the Test Interface Controller (TIC).

Table 4.1-7 Pad Control Signals

Signal Name	Type	Source / Destination	Description
SMBIGENDIAN	Input	Pad	Static pin indicating endianness of the system. When HIGH, it indicates Big-Endian mode. When LOW, it indicates Little-Endian mode.
SMMWCS7[1:0]	Input	Pad	Memory width of CS7. This power on reset value can be over written through the register interface. “00” indicates 8-bits, “01” indicates 16-bits and “10” indicates 32-bits.
SMBLSPOL	Input	Pad	Static input pin used to program the polarity of SMBLS bit field in Bank7 control register.
SMMEMCLKRATIO[1:0]	Input	Pad	Static pin indicating the Clock ratios between HCLK and SMMEMCLK. This value can be overwritten by register write.

Signal Name	Type	Source / Destination	Description
SMBUSGNTEBI	Input	EBI	External bus granted for memory transfer.
SMTICBUSGNTEBI	Input	EBI	External bus granted for TIC transfer.
SMBUSBACKOFFEBI	Input	EBI	Backoff indication from EBI for memory accesses.
SMEXTBUSMUX	Input	EBI	Static External Bus select pin. High indicates that EBI needs to be used. Low indicates that internal DBI needs to be used for arbitration.
SMBUSREQEBI	Output	EBI	Request EBI for Memory transfer.
SMTICBUSREQEBI	Output	EBI	Request EBI for TIC transfer.
SMWAIT	Input	Pad	Asynchronous wait signal from external memory controller to delay the transfer.
SMCANCELWAIT	Input	Pad	Asynchronous wait signal from external memory controller, to signal that SMWAIT is timed out.
nSMBURSTWAIT[7:0]	Input	Pad	Synchronous burst wait signal from external memory to delay the transfer.

Table 4.1-8 Miscellaneous signals

Signal Name	Type	Source / Destination	Description
SCANENABLE	Input	Test Controller	Scan Enable signal. Indicates that the circuit is in scan-test mode.
SCANINHCLK	Input	Test Controller	HCLK domain Scan data input for clocking in the test pattern in scan-test mode.
SCANINSMMEMCLK	Input	Test Controller	SMMEMCLK domain Scan data input for clocking in the test pattern in scan-test mode.
SCANINnSMMEMCLK	Input	Test Controller	nSMMEMCLK domain Scan data input for clocking in the test pattern in scan-test mode.
SCANINFBCLKIN0	Input	Test Controller	SSMCFBCLKIN0 domain Scan data input for clocking in the test pattern in scan-test mode.
SCANINFBCLKIN1	Input	Test Controller	SSMCFBCLKIN1 domain Scan data input for clocking in the test pattern in scan-test mode.
SCANINFBCLKIN2	Input	Test Controller	SSMCFBCLKIN2 domain Scan data input for clocking in the test pattern in scan-test mode.
SCANINFBCLKIN3	Input	Test Controller	SSMCFBCLKIN3 domain Scan data input for clocking in the test pattern in scan-test mode.
SCANINCLKDELAY	Input	Test Controller	SSMCCLKDELAY domain Scan data input for clocking in the test pattern in scan-test mode.
SCANOUTHCLK	Output	Test Controller	HCLK domain Scan data output for observing the flip-flop contents in scan test mode.

Signal Name	Type	Source / Destination	Description
SCANOUTSMMEMCLK	Output	Test Controller	SMMEMCLKdomain Scan data output for observing the flip-flop contents in scan test mode.
SCANOUTnSMMEMCLK	Output	Test Controller	nSMMEMCLK domain Scan data output for observing the flip-flop contents in scan test mode.
SCANOUTFBCLKIN0	Output	Test Controller	SSMCFBCLKIN0 domain Scan data output for observing the flip-flop contents in scan test mode.
SCANOUTFBCLKIN1	Output	Test Controller	SSMCFBCLKIN1 domain Scan data output for observing the flip-flop contents in scan test mode.
SCANOUTFBCLKIN2	Output	Test Controller	SSMCFBCLKIN2 domain Scan data output for observing the flip-flop contents in scan test mode.
SCANOUTFBCLKIN3	Output	Test Controller	SSMCFBCLKIN3 domain Scan data output for observing the flip-flop contents in scan test mode.
SCANOUTCLKDELAY	Output	Test Controller	SMMEMCLKDELAY domain Scan data output for observing the flip-flop contents in scan test mode.

Table 4.1-9 Scan Test related signals

4.2 Programmer's Model

In this section, information about all the registers in the SSMC (PL093) is given.

Address	Type	Width	Reset Value HRESETn	Register Name	Description
SSMC RegBase + 0x00	Read/ Write	4	0xF	SMBIDCYR0	Idle cycle control register for memory bank 0
SSMC RegBase + 0x04	Read/ Write	5	0x1F	SMBWSTRDR0	Read wait state control register for memory bank 0
SSMC RegBase + 0x08	Read/ Write	5	0x1F	SMBWSTWRR0	Write wait state control register for memory bank 0
SSMC RegBase + 0x0C	Read/ Write	4	0x0	SMBWSTOENR0	Output enable assertion delay control register for memory bank 0
SSMC RegBase + 0x10	Read/ Write	4	0x1	SMBWSTWENR0	Write enable assertion delay control register for memory bank 0
SSMC RegBase +	Read/ Write	22	0x303020	SMBCR0	Control register for memory bank 0

Address	Type	Width	Reset Value HRESETn	Register Name	Description
0x14					
SSMC RegBase + 0x18	Read/ Write	1	0x0	SMBSR0	Status register for memory bank 0
SSMC RegBase + 0x1C	Read/ Write	5	0x1F	SMBWSTBRDR0	Burst read wait state control register for memory bank 0
SSMC RegBase + 0x20	Read/ Write	4	0xF	SMBIDCYR1	Idle cycle control register for memory bank 1
SSMC RegBase + 0x24	Read/ Write	5	0x1F	SMBWSTRDR1	Read wait state control register for memory bank 1
SSMC RegBase + 0x28	Read/ Write	5	0x1F	SMBWSTWRR1	Write wait state control register for memory bank 1
SSMC RegBase + 0x2C	Read/ Write	4	0x0	SMBWSTOENR1	Output enable assertion delay control register for memory bank 1
SSMC RegBase + 0x30	Read/ Write	4	0x1	SMBWSTWENR1	Write enable assertion delay control register for memory bank 1
SSMC RegBase + 0x34	Read/ Write	22	0x303000	SMBCR1	Control register for memory bank 1
SSMC RegBase + 0x38	Read/ Write	1	0x0	SMBSR1	Status register for memory bank 1
SSMC RegBase + 0x3C	Read/ Write	5	0x1F	SMBWSTBRDR1	Burst read wait state control register for memory bank 1
SSMC RegBase + 0x40	Read/ Write	4	0xF	SMBIDCYR2	Idle cycle control register for memory bank 2
SSMC RegBase + 0x44	Read/ Write	5	0x1F	SMBWSTRDR2	Read wait state control register for memory bank 2
SSMC RegBase + 0x48	Read/ Write	5	0x1F	SMBWSTWRR2	Write wait state control register for memory bank 2
SSMC RegBase + 0x4C	Read/ Write	4	0x0	SMBWSTOENR2	Output enable assertion delay control register for memory bank 2
SSMC RegBase + 0x50	Read/ Write	4	0x1	SMBWSTWENR2	Write enable assertion delay control register for memory bank 2

Address	Type	Width	Reset Value HRESETn	Register Name	Description
SSMC RegBase + 0x54	Read/ Write	22	0x303010	SMBCR2	Control register for memory bank 2
SSMC RegBase + 0x58	Read/ Write	1	0x0	SMBSR2	Status register for memory bank 2
SSMC RegBase + 0x5C	Read/ Write	5	0x1F	SMBWSTBRDR2	Burst read wait state control register for memory bank 2
SSMC RegBase + 0x60	Read/ Write	4	0xF	SMBIDCYR3	Idle cycle control register for memory bank 3
SSMC RegBase + 0x64	Read/ Write	5	0x1F	SMBWSTRDR3	Read wait state control register for memory bank 3
SSMC RegBase + 0x68	Read/ Write	5	0x1F	SMBWSTWRR3	Write wait state control register for memory bank 3
SSMC RegBase + 0x6C	Read/ Write	4	0x0	SMBWSTOENR3	Output enable assertion delay control register for memory bank 3
SSMC RegBase + 0x70	Read/ Write	4	0x1	SMBWSTWENR3	Write enable assertion delay control register for memory bank 3
SSMC RegBase + 0x74	Read/ Write	22	0x303000	SMBCR3	Control register for memory bank 3
SSMC RegBase + 0x78	Read/ Write	1	0x0	SMBSR3	Status register for memory bank 3
SSMC RegBase + 0x7C	Read/ Write	5	0x1F	SMBWSTBRDR3	Burst read wait state control register for memory bank 3
SSMC RegBase + 0x80	Read/ Write	4	0xF	SMBIDCYR4	Idle cycle control register for memory bank 4
SSMC RegBase + 0x84	Read/ Write	5	0x1F	SMBWSTRDR4	Read wait state control register for memory bank 4
SSMC RegBase + 0x88	Read/ Write	5	0x1F	SMBWSTWRR4	Write wait state control register for memory bank 4
SSMC RegBase + 0x8C	Read/ Write	4	0x0	SMBWSTOENR4	Output enable assertion delay control register for memory bank 4

Address	Type	Width	Reset Value HRESETn	Register Name	Description
SSMC RegBase + 0x90	Read/ Write	4	0x1	SMBWSTWENR4	Write enable assertion delay control register for memory bank 4
SSMC RegBase + 0x94	Read/ Write	22	0x303020	SMBCR4	Control register for memory bank 4
SSMC RegBase + 0x98	Read/ Write	1	0x0	SMBSR4	Status register for memory bank 4
SSMC RegBase + 0x9C	Read/ Write	5	0x1F	SMBWSTBRDR4	Burst read wait state control register for memory bank 4
SSMC RegBase + 0xA0	Read/ Write	4	0xF	SMBIDCYR5	Idle cycle control register for memory bank 5
SSMC RegBase + 0xA4	Read/ Write	5	0x1F	SMBWSTRDR5	Read wait state control register for memory bank 5
SSMC RegBase + 0xA8	Read/ Write	5	0x1F	SMBWSTWRR5	Write wait state control register for memory bank 5
SSMC RegBase + 0xAC	Read/ Write	4	0x0	SMBWSTOENR5	Output enable assertion delay control register for memory bank 5
SSMC RegBase + 0xB0	Read/ Write	4	0x1	SMBWSTWENR5	Write enable assertion delay control register for memory bank 5
SSMC RegBase + 0xB4	Read/ Write	22	0x303020	SMBCR5	Control register for memory bank 5
SSMC RegBase + 0xB8	Read/ Write	1	0x0	SMBSR5	Status register for memory bank 5
SSMC RegBase + 0xBC	Read/ Write	5	0x1F	SMBWSTBRDR5	Burst read wait state control register for memory bank 5
SSMC RegBase + 0xC0	Read/ Write	4	0xF	SMBIDCYR6	Idle cycle control register for memory bank 6
SSMC RegBase + 0xC4	Read/ Write	5	0x1F	SMBWSTRDR6	Read wait state control register for memory bank 6
SSMC RegBase + 0xC8	Read/ Write	5	0x1F	SMBWSTWRR6	Write wait state control register for memory bank 6

Address	Type	Width	Reset Value HRESETn	Register Name	Description
SSMC RegBase + 0xCC	Read/ Write	4	0x0	SMBWSTOENR6	Output enable assertion delay control register for memory bank 6
SSMC RegBase + 0xD0	Read/ Write	4	0x1	SMBWSTWENR6	Write enable assertion delay control register for memory bank 6
SSMC RegBase + 0xD4	Read/ Write	22	0x303010	SMBCR6	Control register for memory bank 6
SSMC RegBase + 0xD8	Read/ Write	1	0x0	SMBSR6	Status register for memory bank 6
SSMC RegBase + 0xDC	Read/ Write	5	0x1F	SMBWSTBRDR6	Burst read wait state control register for memory bank 6
SSMC RegBase + 0xE0	Read/ Write	4	0xF	SMBIDCYR7	Idle cycle control register for memory bank 7
SSMC RegBase + 0xE4	Read/ Write	5	0x1F	SMBWSTRDR7	Read wait state control register for memory bank 7
SSMC RegBase + 0xE8	Read/ Write	5	0x1F	SMBWSTWRR7	Write wait state control register for memory bank 7
SSMC RegBase + 0xEC	Read/ Write	4	0x0	SMBWSTOENR7	Output enable assertion delay control register for memory bank 7
SSMC RegBase + 0xF0	Read/ Write	4	0x1	SMBWSTWENR7	Write enable assertion delay control register for memory bank 7
SSMC RegBase + 0xF4	Read/ Write	22	0x303000	SMBCR7	Control register for memory bank 7
SSMC RegBase + 0xF8	Read/ Write	1	0x0	SMBSR7	Status register for memory bank 7
SSMC RegBase + 0xFC	Read/ Write	5	0x1F	SMBWSTBRDR7	Burst read wait state control register for memory bank 7
SSMC RegBase + 0x100 - 0x1FC	**Reserved**		**Undefined**	**Reserved**	**Reserved**
SSMC RegBase + 0x200	Read	1	0x0	SSMCSR	External Wait Status register for all memory banks

Address	Type	Width	Reset Value HRESETn	Register Name	Description
SSMC RegBase + 0x204	Read/ Write	3	0x1	SSMCCR	SSMC Control Register. Controls all banks
SSMC RegBase + 0x208	Read/ Write	1	0x0	SSMCITCR	SSMC Test Control Register.
SSMC RegBase + 0x20C	Read/ Write	7	System Dependant	SSMCITIP	SSMC Test Input Register.
SSMC RegBase + 0x210	Read/ Write	2	System Dependant	SSMCITOP	SSMC Test Output Register.
SSMC RegBase + 0x214 - 0xFDC	**Reserved**		**Undefined**	**Reserved**	**Reserved**
SSMC RegBase + 0xFE0	Read Only	8	0x93	SSMCPeriphID0	Peripheral ID register Bits 7:0
SSMC RegBase + 0xFE4	Read Only	8	0x10	SSMCPeriphID1	Peripheral ID register Bits 15:8
SSMC RegBase + 0xFE8	Read Only	8	0x14	SSMCPeriphID2	Peripheral ID register Bits 23:16
SSMC RegBase + 0xFEC	Read Only	8	0x00	SSMCPeriphID3	Peripheral ID register Bits 31:24
SSMC RegBase + 0xFF0	Read Only	8	0x0D	SSMCPCellID0	PrimeCell ID register Bits 7:0
SSMC RegBase + 0xFF4	Read Only	8	0xF0	SSMCPCellID1	PrimeCell ID register Bits 15:8
SSMC RegBase + 0xFF8	Read Only	8	0x05	SSMCPCellID2	PrimeCell ID register Bits 23:16
SSMC RegBase + 0xFFC	Read Only	8	0xB1	SSMCPCellID3	PrimeCell ID register Bits 31:24

Table 4.2-1 Memory map for the SSMC registers

NOTE: **Writes to a read-only register will not change the contents of the register. However, the SSMC AHB slave register interface will return an OKAY response on the AHB for such accesses.**

4.3 Design Hierarchy

The hierarchy in the SSMC is shown in ***Figure 4.3-1***. A short description about the functionality of each of the sub-modules is also given within brackets.

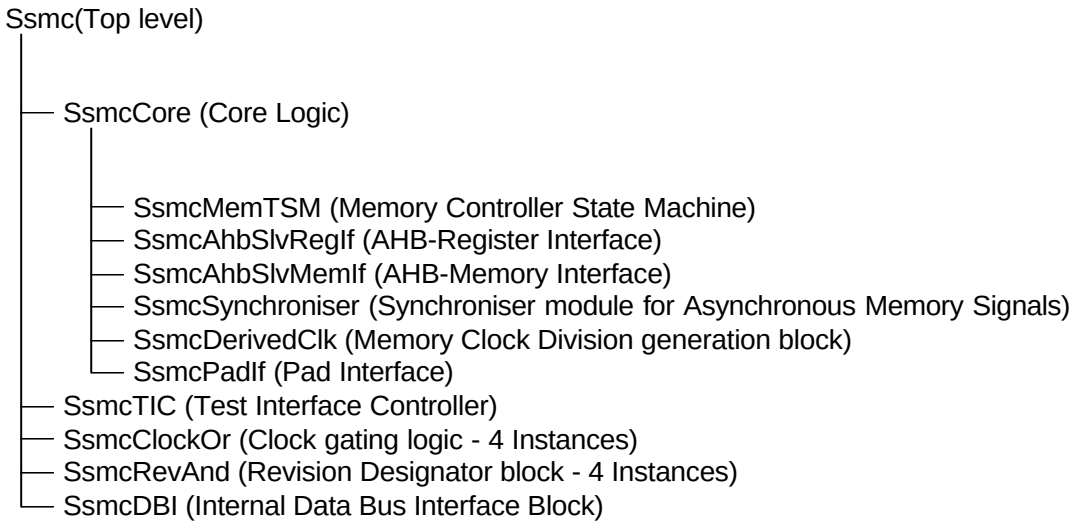


Figure 4.3-1 SSMC Design Hierarchy

4.4 SSMC Block Partitioning

At the topmost level, the SSMC consists of the following blocks:

- SSMC Core Logic (SsmcCore)
- SSMC TIC Interface (SsmcTIC)
- SSMC DBI Interface (SsmcDBI)
- SSMC Clock Gating Logic (SsmcClockOr)
- SSMC Revision Designator Block (SsmcRevAnd)

The SSMC Core logic block contains the main functionality of the SSMC. It contains the memory controller, AHB interfaces and the Pad Interface blocks. The SSMC TIC block contains the test interface controller functionality of the SSMC. The SSMC Core logic and the SSMC TIC Interface share some of the output pins at the topmost level of the SSMC. The logic to combine the outputs from the two blocks is within the Pad Interface block in the SSMC Core Logic. The clock gating logic module of the SSMC (SsmcClockOr) is instantiated four times to generate the four output clocks, SMCLK [3:0]. The revision designator block (SsmcRevAnd) is also instantiated four times to generate the four bits of the revision number of the SSMC. The DBI (SsmcDBI) block is responsible for arbitration between Memory request and TIC request when EXTBUSMUX is disabled.

A block schematic of the ARM PrimeCell Synchronous static Memory Controller (PL093) is shown in ***Figure 4.4-1.***

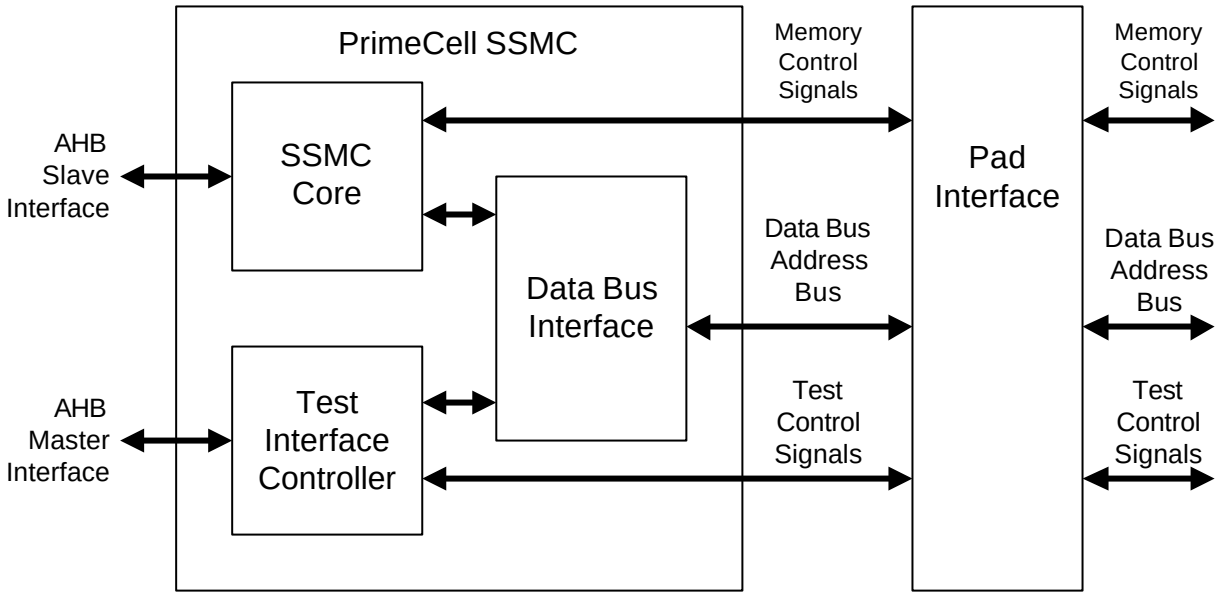


Figure 4.4-1 ARM PrimeCell Synchronous Static Memory Controller (PL093) block diagram

The SSMC Core logic consists of the following major sub-blocks:

- AHB Memory Interfaces top level block to provide the interface between the SSMC and the AHB ports
- AHB Register Interface to provide the interface between the SSMC registers and the AHB register port
- Memory controller block containing the Static memory controller logic.
- Pad Interface to suitably Mux out the control, address and data signals onto the memory.

A block schematic of the SSMC Core logic is shown in ***Figure 4.4-2.***

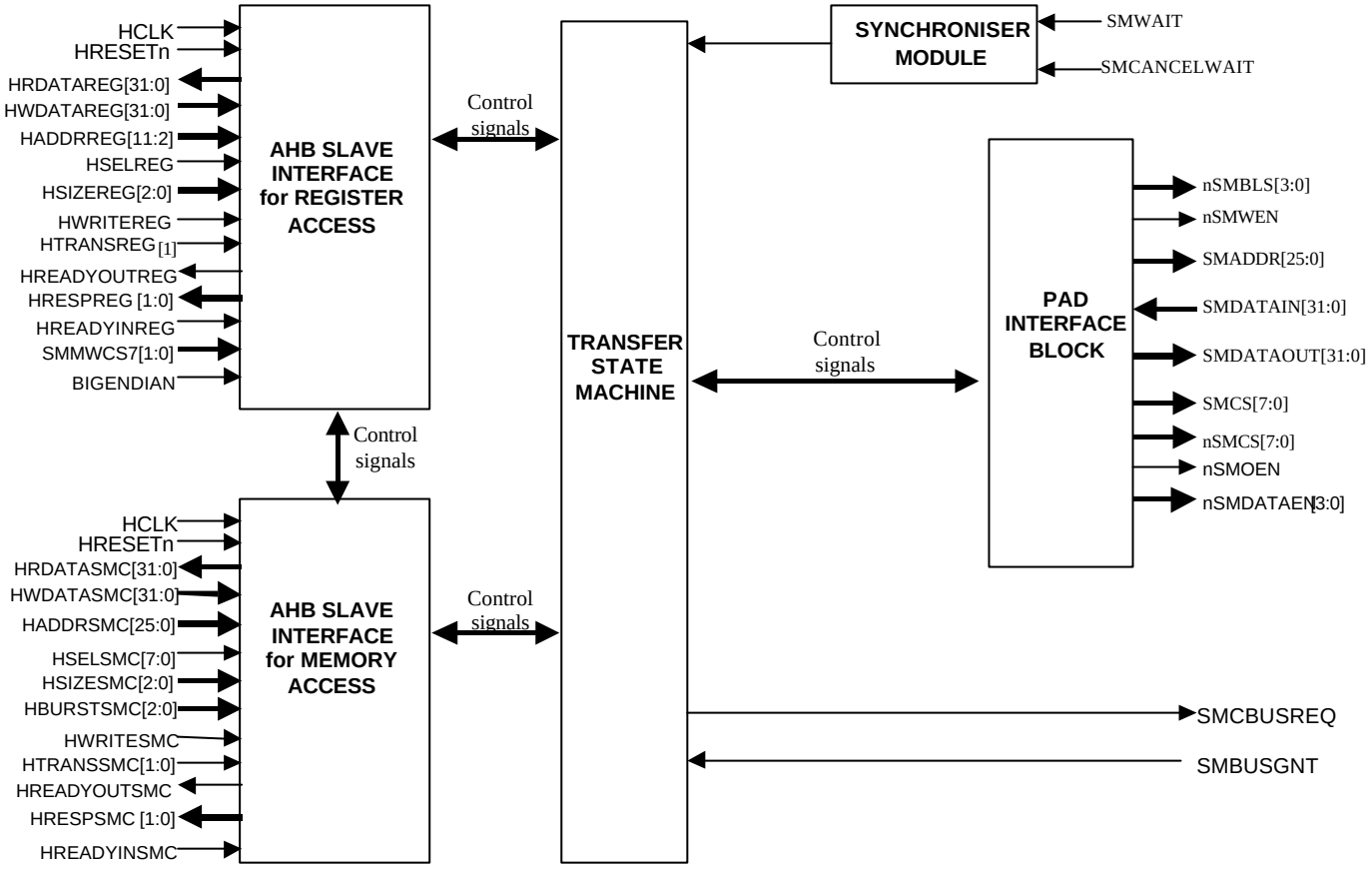


Figure 4.4-2 Block diagram of the sub-blocks in the SSMC Core logic

4.5 AHB Register Interface (SsmcAhbSlvRegIf)

The port level connections of the AHB memory interface (SsmcAhbRegIf) are shown in ***Figure 4.5-1.***

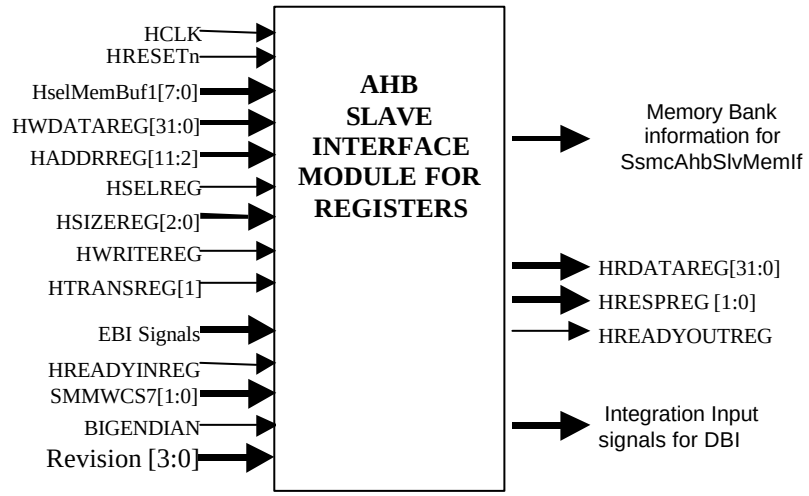


Figure 4.5-1 Block Inputs and Outputs of SsmcAhbSlvRegIf

The AHB Register Interface provides the interface between the AHB and the registers of the SSMC. This module also

contains all the common (non-chip-select-specific) registers, the integration test registers, the peripheral ID registers and the PrimeCell Id registers. This module contains the logic to generate the appropriate responses to the register access transfers on the AHB.

4.6 AHB Memory Interface (SsmcAhbSlvMemIf)

The port level connections of the AHB memory interface (SsmcAhbSlvMemIf) are shown in ***Figure 4.6-1.***

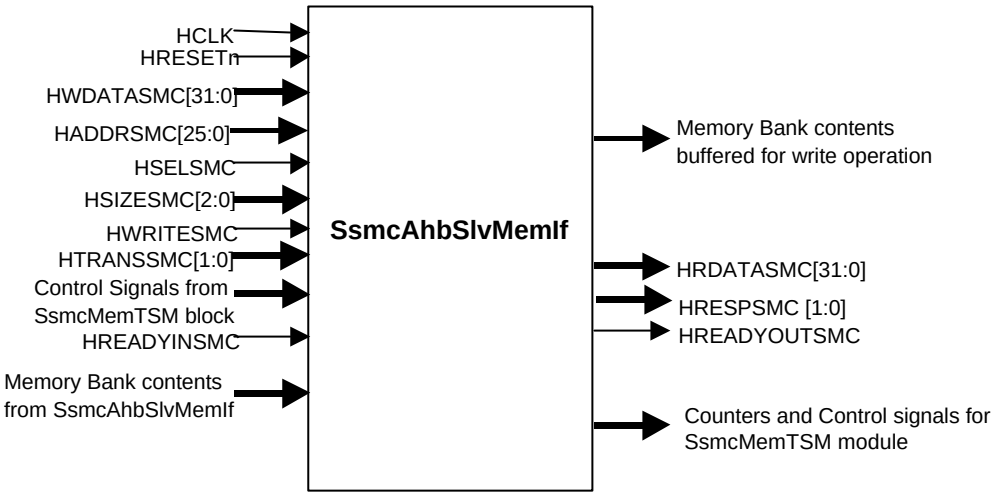


Figure 4.6-1 Block Inputs and Outputs of SsmcAhbSlvMemIf

Following functionality is implemented in this block

- AHB Address and control signals registering
- Memory transfer request generation
- Predictive address generation for sequential transfers
- Valid byte indicator generation
- Read data selection and packing based on Endian mode
- Error monitoring to detect the transfers that result in an error response.

4.7 Memory Interface (SsmcMemTSM)

The Static memory Transfer State Machine (SsmcMemTSM) block controls all the transactions of the static memory controller block to the external memory device. It generates the bus request signal for the control of the external data bus lines. External bus turnaround cycles are initiated appropriately after a read transfer completion.

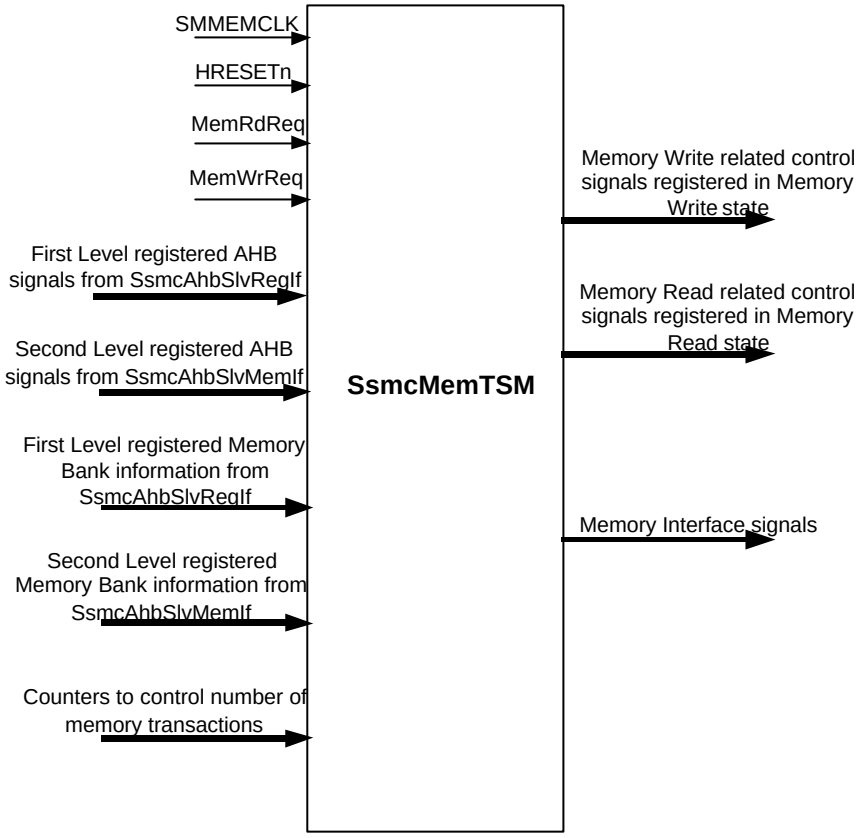


Figure 4.7-1 Block Inputs and Outputs of SsmcMemTSM

4.8 Pad Interface (SsmcPadIf)

The pad interface (SsmcPadIf) block is responsible for routing the control, address and data signals from SsmcCore to the Memory. This block is also responsible for clocking in the SMDATAIN from memory in case of synchronous accesses.

The port level connections of the SsmcPadIf are shown in ***Fig 4.8-1***.

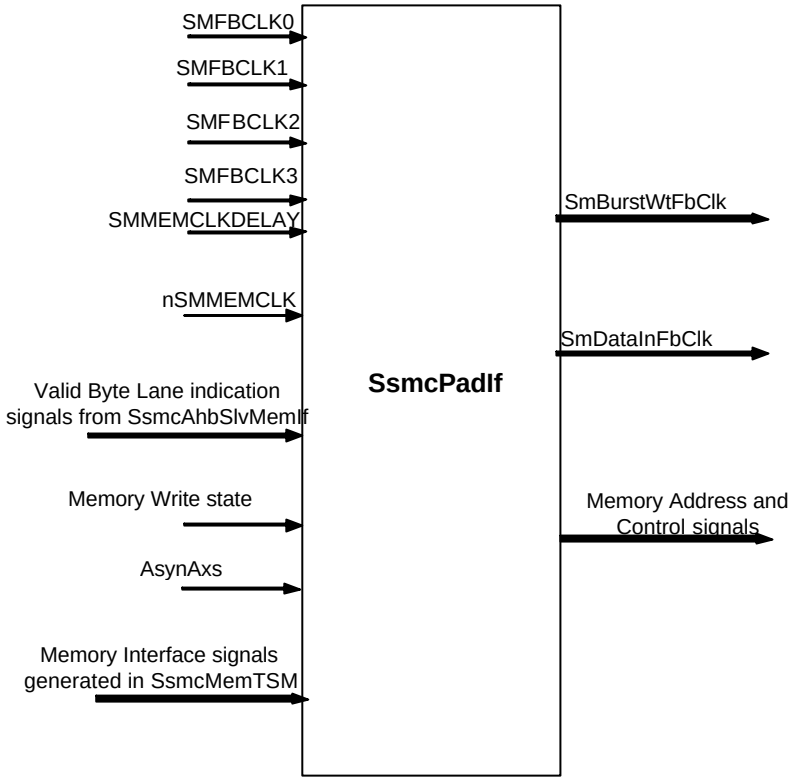


Figure 4.8-1 Block Inputs and Outputs of SsmcPadIf

4.9 Synchroniser block (SsmcSynchroniser)

The asynchronous input signal SMWAIT and SMCANCELWAIT are passed through double synchronizers before using internally in the logic. The outputs of this block are the signals SMWaitSync and SMCANCELWAITSync.

The block schematic of the Synchronizer Block is shown in the ***Figure 4.9-1***.

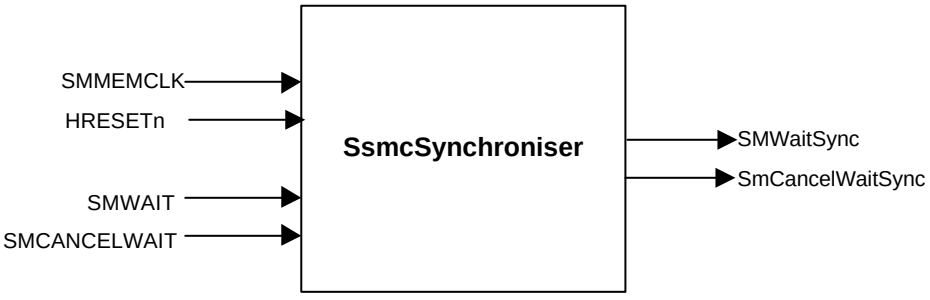


Figure 4.9-1 Synchroniser Block

4.10 Internal Data Bus Interface block (SsmcDBI)

This block implements arbitration logic for the Data Bus and Address Bus Interface between SsmcCore and TIC block. The TIC has the highest priority and the SsmcCore has the lowest priority.

Support is also provided in the DBI so that the SsmcCore can be interfaced with an external Bus interface arbiter block. Bypass logic has been implemented which will cater to the requirement when the SSMC needs to interface with the EbiSdram. In this scenario the DBI becomes redundant as the External bus request and bus grant pins become active and these get connected to the SsmcCore block. If the internal DBI is bypassed for using the external EbiSdram, then the bus-request and bus-grant pins of the SsmcCore and TIC, along with TIC's output data lines and data enable pin [all these are pins of the SSMC] will become active.

The block schematic shown in the **Figure 4.10-1** represents the diagram of the DBI with its I/O pins.

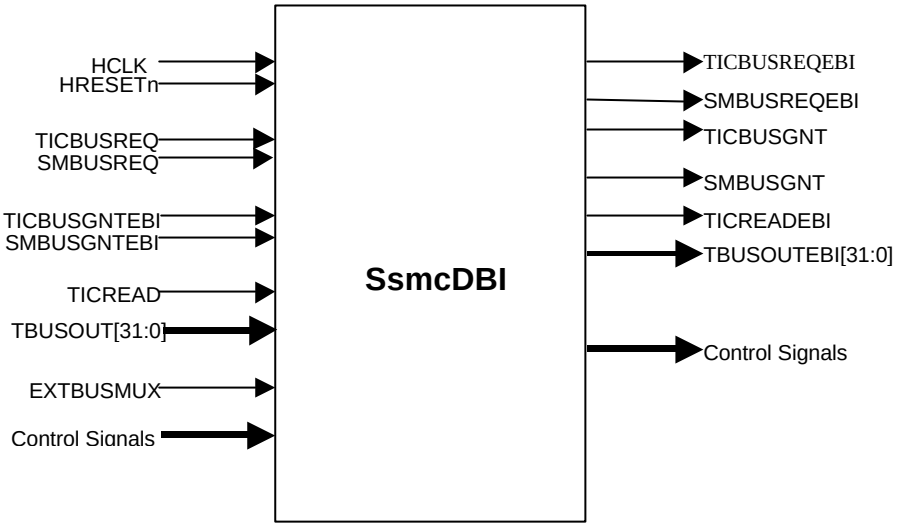


Figure 4.10-1 Data Bus Interface [DBI] Block

4.11 SlowClkM generation block (SsmcDerivedClk)

SlowClkM is used to indicate the slower of SMMEMCLK and HCLK to the Memory Controller. SlowClkM is used in the Memory Controller to control the timing of assertion of AHB and Memory signals. This signal becomes active when frequency ratios is SMMEMCLK: HCLK = 2: 1, SMMEMCLK: HCLK = 3: 1.

Figure 4.11-1 Block Inputs and Outputs for SsmcDerivedClk

4.12 Clock gating logic (SsmcClockOr)

This module (SsmcClockOr) contains a single OR gate to be used for clock gating to control the enabling of the clock output to the synchronous memories. The clocks to the memory devices are pulled high when they are not being accessed in order to save power. The ClkStpd signal from the SsmcCore is used to control the output clocks.

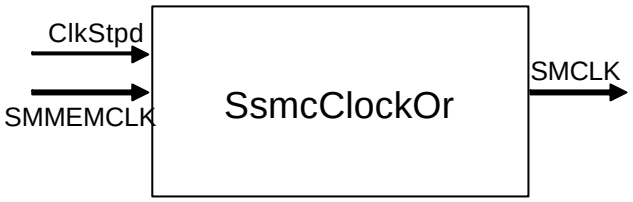


Figure 4.12-1 Block Inputs and Outputs for SsmcClockOr

4.13 Revision Designator Block (SsmcRevAnd)

This module (SsmcRevAnd) contains a single AND gate to be used as a placeholder cell to mark the Revision of the controller. The 2 input pins are tied-off at the top level of the hierarchy. These "TieOffs" can be identified during layout and re-wired to "VDD" or "VSS" if needed.

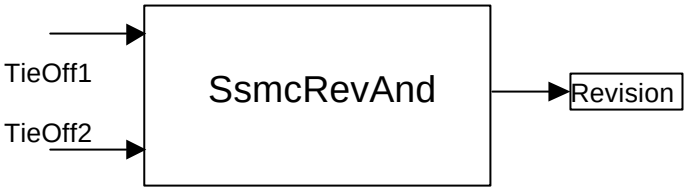


Figure 4.13-1 Block Inputs and Outputs for SsmcRevAnd

4.14 Test Interface Controller(SsmcTIC)

The SSMC TIC bus master module (SsmcTIC) helps to allow externally applied test vectors to be converted into internal AHB transfers with the appropriate sequence. The TIC controls the External Test Bus and AHB data buses HRDATATIC and HWDATATIC during the application of Test Interface Format (TIF) test vectors

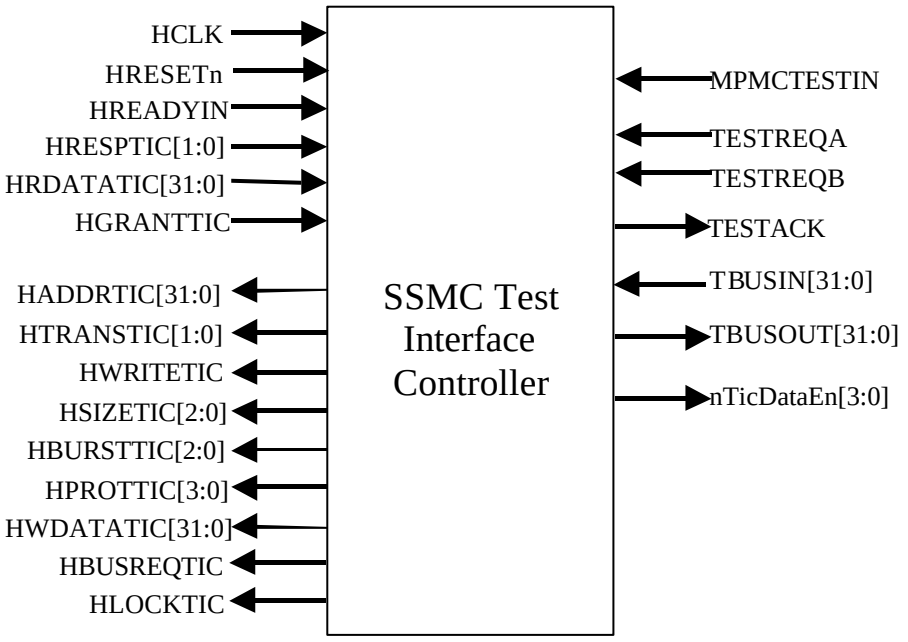


Figure 4.14-1 Block Inputs and Outputs of SsmcTIC

5 SSMC IMPLEMENTATION DETAILS

In this section, the implementation details of the blocks listed in section 4 are provided.

5.1 SSMC Top Level Module (Ssmc)

The top-level module is a structural block which instantiates and interconnects the two main functional blocks in the SSMC - the SSMC Core logic and the SSMC TIC interface. SSMC also contains the revision designator block and the clock gating logic for the output clock to the synchronous memories. It also contains Internal Data bus interface to arbitrate between SsmcCore and SsmcTIC requests. All the functional blocks in the SSMC are described in the following sections.

5.2 SSMC Core (SsmcCore)

The SSMC Core logic block is a structural block containing the blocks that facilitate the SSMC's memory controller functionality. It contains the AHB slave interfaces for memory accesses and the AHB slave interface for accesses to the SSMC registers. This block also controls the flow of Read and Write transfers between AHB and Memory. There is a Pad interface, which is responsible for routing the control signals on appropriate lines for Memory. The data from the memory is clocked with respect to FBCLK in case of synchronous memories.

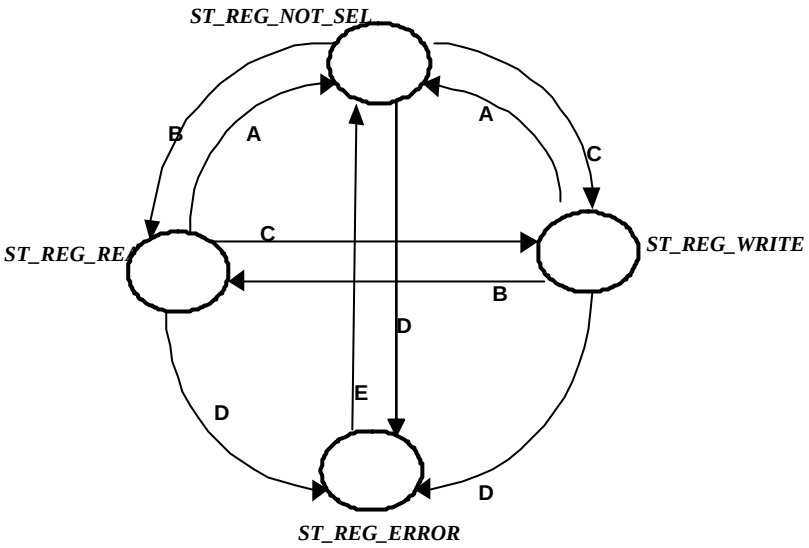
5.3 SSMC AHB Register Interface (SsmcAhbSlvRegIf)

This module provides the interfacing to AMBA AHB buses. The slave interface is divided into two Slave modules, one for Register access and other for Memory access. Each of these modules has AHB response generation logic. All of the configuration, control and status registers corresponding to the external memory banks are situated in SsmcAhbSlvRegIf module. In addition to these registers the PrimeCell and Peripheral Identification registers are situated in SsmcAhbSlvRegIf module. All Memory accesses are controlled by SsmcAhbSlvMemIf module. ***Figure 5.3-1*** shows the various states the State Machine (SM) transitions through while accessing the slave registers.

The default state is ST_REG_NOT_SEL and the SM continues to be in this state until the HSELREG is sampled asserted at the end of any given bus cycle. If in any given state the HSELREG is sampled de-asserted, or if HTRANSREG [1] is sampled 0, at the end of a bus cycle the SM transitions to the ST_REG_NOT_SEL State.

The SM makes a state transition to ST_REG_READ state from any other state whenever a 32 bit, read access data transfer cycle is initiated on the SSMC at the end of any given bus cycle.

The SM makes a state transition to ST_REG_WRITE state from any other state whenever a 32 bit, write access data transfer cycle is initiated on the SSMC at the end of any given bus cycle.



Condition A : HREADYINREG && (!HSELREG or (HTRANSREG = 0))
Condition B : HSELREG && HREADYINREG && (HTRANSREG = 1) && !HWRITEREG
Condition C : HSELREG && HREADYINREG && (HTRANSREG = 1) && HWRITEREG
Condition D : HSIZEREG != WORD
Condition E : Unconditional transition.

Figure 5.3-1 AHB Register Slave State Machine

The logic has a separate address buffer that latches in the address on the HADDRREG lines at the end of every bus cycle. This latched address is used for further decoding. The data transfers from registers to the AHB bus occur only in the ST_REG_READ state. The data transfers to registers from the AHB bus occur only in the ST_REG_WRITE state.

The buffered address is decoded and if the access is a Write to registers a combinatorial decode of ST_REG_WRITE state bit and the buffered address directly forms the Write Enable for the registers. The Writes to registers terminate with one clock cycle response.

Whenever a read/write access is initiated on SSMC, the SM always inserts a Wait state for the cycle. This Wait period is of only one clock. At the end of the Wait period the SM drives HREADYOUTREG high with OKAY response as the SM assumes that a valid data is present on HRDATAREG flip-flops and thus can ideally terminate the bus cycle with one clock response. Further details about the internal registers can be found in the *ARM PrimeCell Synchronous Static Memory Controller (PL093) Technical Reference Manual [Ref. 1], section 5 Programmer's Model*.

The SM enters the ST_REG_ERROR state when there is an ERROR access i.e. when HSIZEREG is not equal to WORD access. In this case HRESPREG is driven with ERROR response and two cycle ERROR response is driven on HREADYOUTREG (First cycle HREADYOUTREG is de-asserted and in second cycle it is asserted) line.

This block also updates the status register. The status indicated is WaitToutErr for eight Memory Banks.

Register Write Logic

Write to the register is done in 2 stages. In the First clock the Write enables for the register is generated. These write enables are direct decode of the HADDRREG, HSELREG and HREADYINREG. In the second clock cycle the generated write enables are used as the qualifying condition to write into the actual register. Hence register write takes 2 clock cycles.

Register Read Logic

Read to the register takes 2 clock cycles. In the first clock cycle the data from the registers are multiplexed and the output of the Mux is clocked into 32-bit register. This is done because the Mux is very huge to accommodate in 40% of HCLK = 3 ns. In the second clock cycle the registered contents is driven out as HRDATAREG, and HREADYOUTREG is asserted.

Routing the Memory Bank information

At a time only one memory bank can be active. So appropriate memory bank has to be selected. There is a Mux with HselMemBuf1 as the select input. Depending on HselMemBuf1 value appropriate memory banks information is routed on to memory controller state machine (SsmcMemTSM). HselMemBuf1 is nothing but HSELSMC registered version when a valid read or writes is initiated by the master to the memory through the controller.

Integration Test Registers

The SSMC integration test registers allows the integrator to verify that the SSMC has been connected into the target system correctly. The T bit in the SSMCITCR register is used for configuring the SSMC in integration test mode. T is low for normal operation. When T is set high by software, multiplexes on the inputs and outputs are switched to use the test mode values.

Figure 5.3-2 Integration Intrachip Input Register explains the implementation details of the input integration test harness. The SSMCITIP is a 7-bit register. The T bit is used as the control bit for the Multiplexer, which is used in the read path of the intra-chip inputs SMBUSBACKOFFEBI, SMTICBUSGNTEBI, SMBUSGNTEBI, SMEXTBUSMUX, SMMEMCLKRATIO and SMBIGENDIAN. If the T control bit is de-asserted, intra-chip inputs are routed to the core logic, else the stored register value is driven on the internal line.

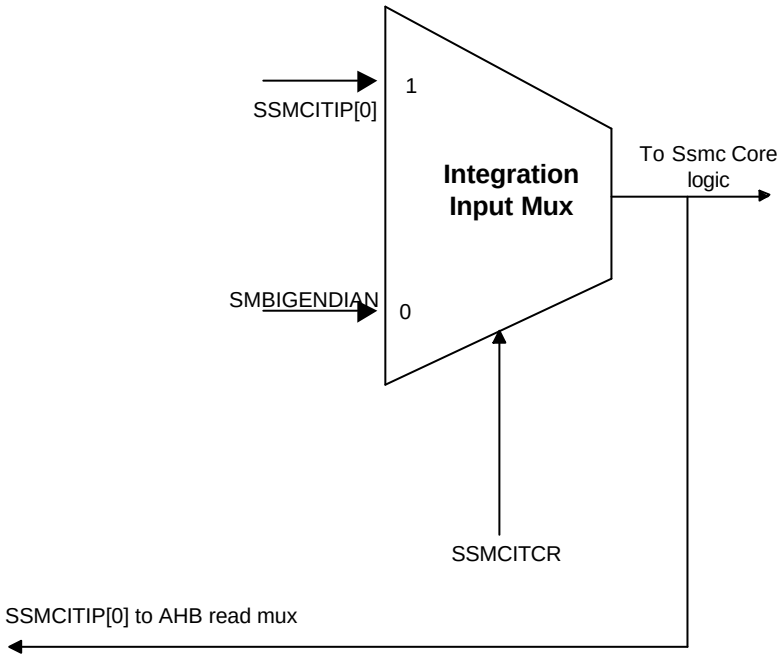


Figure 5.3-2 Integration Intrachip Input Register

Figure 5.3-3 Integration Intrachip Output Register illustrates the implementation details of the output integration test harness in the case of intra-chip outputs. SSMCITOP is a 2-bit register. If the T bit is set, SSMCITOP [1:0] is connected with the output pins SMTICBUSREQEBI and SMBUSREQEBI else these pins are driven by core logic.

-

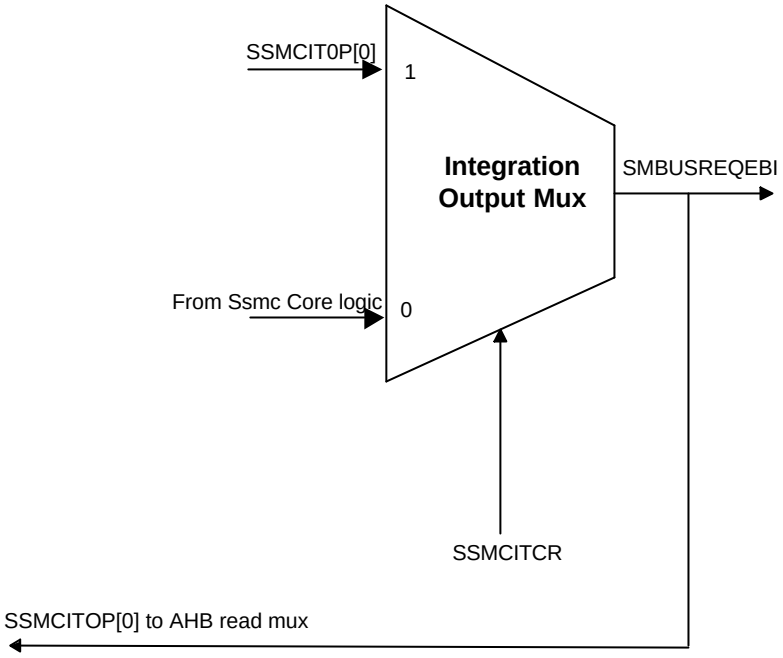


Figure 5.3-3 Integration Intrachip Output Register

5.4 SSMC AHB Memory Interface (SsmcAhbSlvMemIf)

Following functionality is implemented in this block:

- Address and control signal registering: The AHB address and control signals are registered on the rising edge of HCLK in this block and the registered signals are used for generating the memory access requests.
- Memory access request generation: The memory access requests from the AHB are processed and the valid requests are passed on to the memory control SM.
- Valid byte indicator logic: The valid bytes for a write access to the memory are generated based on the Endian mode of the SSMC in the case when AHB Width is greater than memory size.
- Read data selection and packing: The read data is selected and packed depending on the Endian mode.
- Error monitoring: The error generating conditions are checked and errors are flagged to the AHB master.

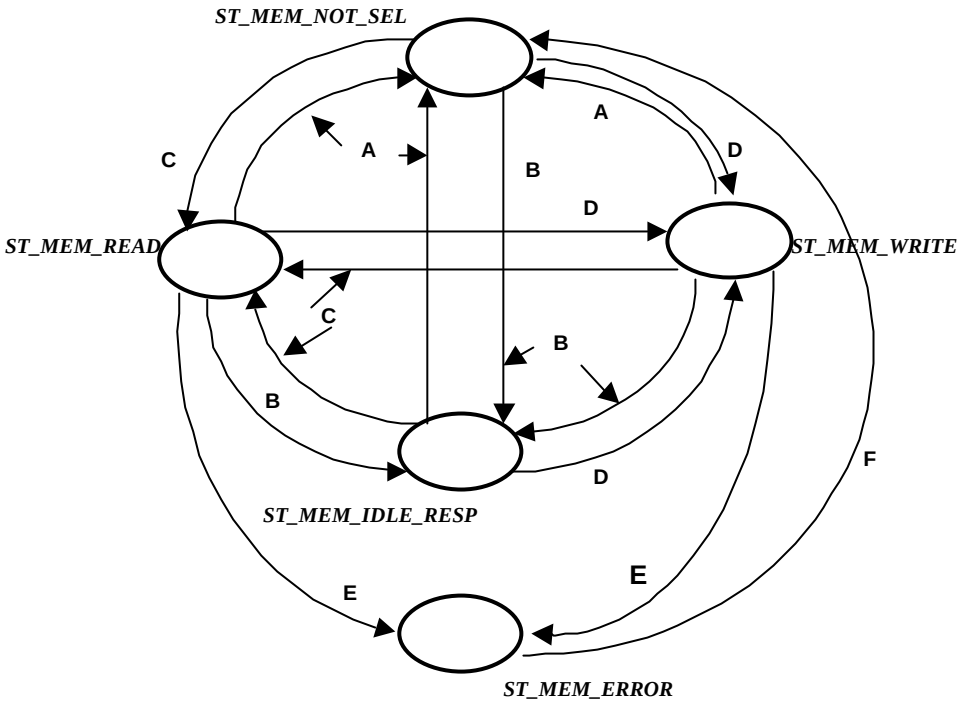
The ***Figure 5.4-1*** shows the various states the slave memory access SM transitions through.

The default state is ST_MEM_NOT_SEL and the SM continues to be in this state until the HSELMEM is sampled asserted at the end of any given bus cycle. If in any given state the HSELMEM is sampled de-asserted at the end of a bus cycle the SM transitions to the ST_MEM_NOT_SEL State.

The SM transitions to ST_MEM_IDLE_RESP at the end of any bus cycle, when IDLE or BUSY is sampled on HTRANSSMC lines. This transition can happen from any of the states in the SM and will happen regardless of the width and nature of the transfer.

The SM makes a state transition to ST_MEM_READ state from any other state whenever a, read access data transfer cycle is initiated on the SSMC at the end of any given bus cycle.

The SM makes a state transition to ST_MEM_WRITE state from any other state whenever a, write access data transfer cycle is initiated on the SSMC at the end of any given bus cycle.



Condition A: !HSELSMC && HREADYINSMC
Condition B: HSELSMC && HREADYINSMC && (HTRANSSMC != NSEQ/SEQ)
Condition C: HSELSMC && HREADYINSMC && (HTRANSSMC = NSEQ/SEQ) && !HWRITESMC
Condition D: HSELSMC && HREADYINSMC && (HTRANSSMC == NSEQ/SEQ) && HWRITESMC
Condition E : HSIZESMC > WORD || WaitToutErr == 1 || WriteProtErr == 1
Condition F : Unconditional transition.

Figure 5.4-1 AHB Memory Slave State Machine

The logic has a separate address buffer that latches in the address on the HADDRSMC lines at the end of every bus cycle. This latched address is used for further decoding. The data transfers from memory to the AHB bus occur only in the ST_MEM_READ state. The data transfers to memory from the AHB bus occur only in the ST_MEM_WRITE state.

The buffered address is decoded and if the access is a Write to memory a combinatorial decode of ST_MEM_WRITE state bit forms the MemWrReq (Write Request for Memory access) for the memory. The Writes to memory terminate with minimum of two-clock cycle response. If the Memory write is taking longer than two cycles to complete on the Memory side then a signal WaitWrCycMux will hold the HREADYOUTSMC signal de-asserted till the write is over.

Whenever a read access is initiated with a NSEQ transfer on the SSMC, the SM always inserts a Wait state for the cycle. This Wait period is generally of only one clock but can be controlled with a signal "WaitRdCycMux" depending on the time it take for the source data to reach to HRDATASMC output flip-flops through a Multiplexer that is controlled by the endianization logic. At the end of the Wait period the SM drives HREADYOUTSMC high with OKAY response. Whenever a read cycle is SEQ type the SM assumes that a valid data is present on HRDATASMC flip-flops and thus can ideally terminate the bus cycle with one clock response provided the source of data has not asserted the "WaitRdCycMux" signal. WaitRdCycMux signal is an indication to the AHB Memory SM, to assert HREADYOUTSMC. When the Read access on the Memory side is waited then this signal is asserted. SM asserts HREADYOUTSMC when it samples WaitRdCycMux de-asserted, with an OKAY response.

The SM enters the ST_MEM_ERROR state when there is an ERROR access. The ERROR access can happen on following conditions.

- During a transfer of size greater than 32-bits to external memory.
- If a write transfer is attempted to a Write-Protected Memory device.

- After an externally waited transfer has timed out.
- When an AHB master tries to initiate a new transfer in the WaitToutErr case and the SMWAIT input is still in the asserted state due to the previous transfer.

In the above cases HRESPSMC is driven with ERROR response and two cycle ERROR response is driven on HREADYOUTSMC (i.e. First cycle HREADYOUTSMC is de-asserted and in second cycle it is asserted) line.

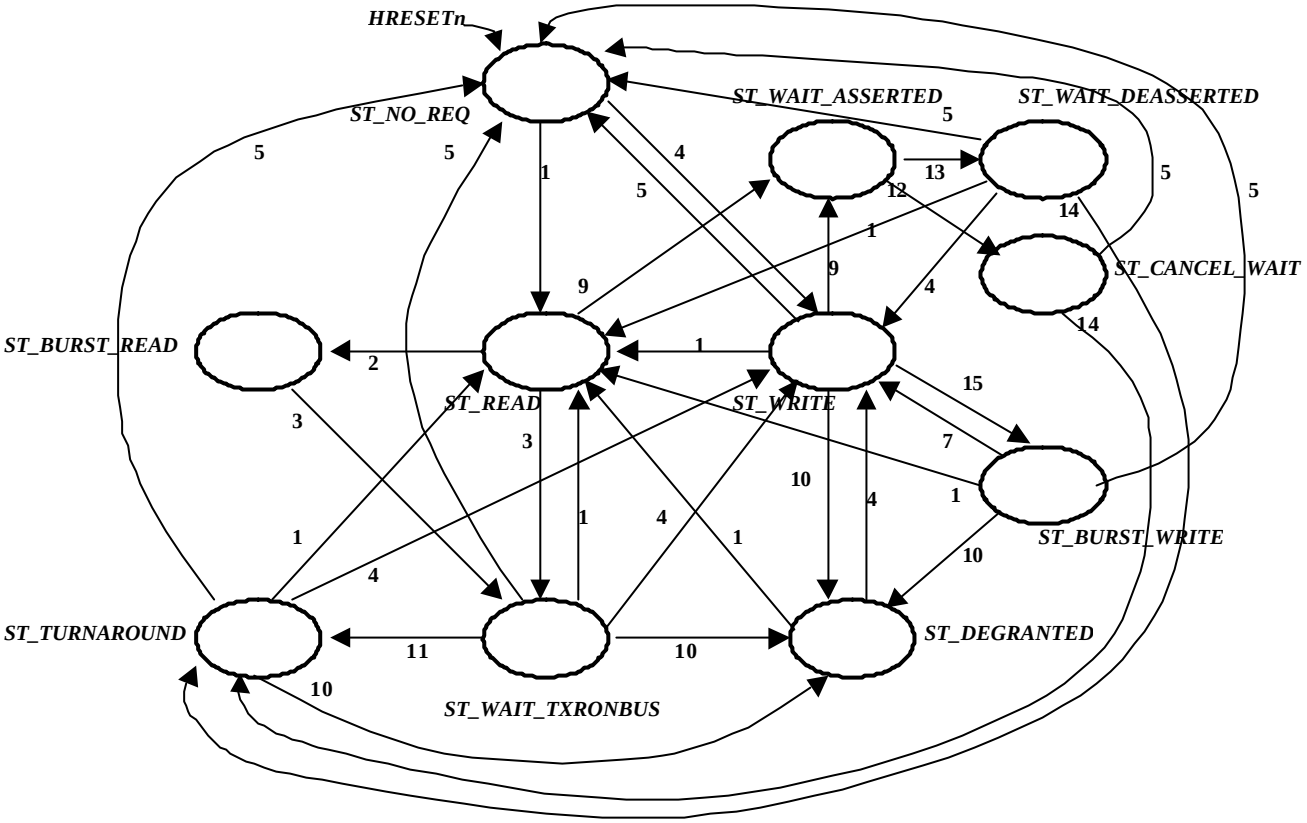
The SMBIGENDIAN input is used to configure the system for Big-Endian or Little-Endian operation, and is assumed to be a static input pin. The SsmcCore does not support dynamic Endianness. This signal is used in the External Interface block to control the routing of data to the proper byte lanes used during data transfers.

The SMMWCS7 [1:0] inputs are used to configure the size for memory bank seven immediately after reset.

Control signals MemRdReq and MemWrReq, for the memory read and write transfers are decoded and sent to the memory transfer SM.
During write transfer for the cases when HSIZESMC < MSIZE a read-write buffer is used to collect the partial data before the external transfer is performed. A counter is implemented, which keeps track of the number of AHB transfers required to start the external memory transfer, these counters are routed to other blocks. In cases where HSIZESMC = MSIZE, this buffer is bypassed and the HWDATASMC is directly routed to the external data out register SMDATAOUT.
TurnAround is the signal, which indicates whether any turn around cycles are needed. The conditions on which this signal is asserted depends on IDCYC value programmed in the register.
Write protection checking and AHB transfer size error checking are performed in this block. Also during reads, optimization is done for transfers with HSIZESMC < MSIZE condition, by caching read data. After the first read the subsequent data's are driven from the internal buffer with zero wait cycles and without any extra memory accesses.

5.5 Memory Control State Machine (SsmcMemTSM)

This block is used to control the passing of signals between the AHB Interface and the External Interface Block. It consists of a SM as shown in ***Fig 5.5-1.*** The SM state transition conditions are also depicted in the Figure.



Condition 1: (MemRdReq = 1) && (SMBUSGNT = 1)
Condition 2: (MemBrstRdReq = 1) && (BM = 1) && (SMBUSGNT = 1)
Condition 3: (SMCS = 1)
Condition 4: (MemWrReq = 1) && (SMBUSGNT = 1)
Condition 5: (MemWrReq = 0) && (MemRdReq = 0)

Condition 7: (NewBurst = 1) && (TurnAround = 0) && (MemWrReq = 1) && (SMBUSGNT = 1)

Condition 9: (WaitEn = 1) && (nSMWait = 0)
Condition 10: (SMBUSGNT = 0) && ((MemRdReq = 1) || (MemWrReq = 1))
Condition 11: (TurnAround = 1) || (IDCYC /= "0000")
Condition 12: (SMCANCELWAIT = 1)
Condition 13: (nSMWAIT = 1)
Condition 14: (TurnAround = 1)
Condition 15: (SyncEnWrite = 1) && (BMWrite = 1) && (SMBUSGNT = 1)

Figure 5.5-1 Memory Transfer State Machine

The SM States are described below.

ST_NO_REQ State:

The SM enters this state on Reset. The SM makes a transition from this state to either ST_READ or ST_WRITE depending on the Memory Read access from AHB or Memory Write Access from AHB, provided SMBUSGNT is sampled asserted. While making a transition to either of two states the Counters (WstBrstRd, WstLongRd, WstWrCnt, WstWrEnCnt, WstOenCnt, and BeatCount) are loaded. While making a transition from this state nSMCS is asserted and SMADDR are driven with the registered value in AHB Memory SM. If the WSTOEN is programmed as zero, then nSMOEN is asserted while making a state transition to ST_READ state along with nSMCS. If the WSTWEN is programmed as zero, then nSMWEN is asserted while making a state transition to ST_WRITE state along with nSMCS. In this state SMBUSREQ is raised.

ST_READ State:

From this state the SM can make a transition to ST_BURST_READ state, ST_WAITASSERTED or ST_WAIT_TXRONBUS state. It will make a transition to ST_BURST_READ state if BM bit is set, provided SMBUSGNT is sampled asserted. The SM will make a transition to ST_WAIT_TXRONBUS if MemRdReq is sampled de-asserted or if the BM bit is not set. Transitions to these states are done after ensuring that the WstLongRd counter had expired. While making a transition to ST_WAIT_TXRONBUS state the nSMCS should have been sampled de-asserted in this state. While making a state transition to ST_BURST_READ state the nSMCS is continued to be asserted. After the First long read the BeatCount (This counter indicates the number of access possible, this restriction might be for memory boundary) counter is decremented.

If it is a wait enabled transfer and if the Wait is asserted before the WstLongRd counter expires then state transition will happen to ST_WAIT_ASSERTED state.

ST_BURST_READ State:

In this state following condition can happen while burst is progressing

- Busy Insertion from Master:

  In this case the SMADDR would have been incremented, so by the time the SM sees the registered BUSY signal, the SMADDR would have progressed ahead of HADDRSMC, the BeatCount would have decremented. Hence if HREADYOUTSMC is asserted immediately for the SEQ transfer following BUSY access, there will be wrong data on the HRDATASMC lines. To prevent this, in the AHB Memory SM care is taken such that immediately after BUSY access if a SEQ access happens then HREADYOUTSMC is de-asserted. Also, the Counters are not updated when the registered BUSY is seen and clocked SMDATAIN is not consumed by controller.

- Break in Burst initiated by Master:

  When the NewBurst indication is sampled asserted in this state then Turnaround is done, before honoring the new burst, provided TurnAround signal is sampled asserted.

- Memory Bus is Degranted:

  When the Memory Bus is degranted in the middle of the Burst then state transition will happen to ST_WAIT_TXRONBUS state, from there depending on the IDCYC value programmed SM will make a transition to ST_DEGRANTED state or ST_TURNAROUND state.

For every completion of one beat of the ongoing burst, the BeatCount counter is decremented. When the BeatCount has down-counted to a value 1 and if the WaitRdCyc (Indication as to when the AHB Memory SM should drive HREADYOUTSMC high) is de-asserted, then state transition will happen to ST_WAIT_TXRONBUS state.

ST_WRITE State:

The SM enters this state after asserting nSMCS, and driving out SMADDR. nSMWEN is asserted depending on the delay value programmed in the WSTWEN register. When the write is happening to Asynchronous devices the nSMWEN signal is de-asserted after every write. This signal is asserted again based on further MemWrReq and delay value

programmed in WSTWEN register. The state transitions happen after the de-assertion of nSMWEN signal. State transition can happen to ST_BURST_WRITE state if there is further MemWrReq from AHB side and BMWrite bit and SyncEnWrite is set.

In this state the signal WaitWrCycVer will decide as to when the HREADYOUTSMC has to be driven high for the current Write access.

If the SMBUSGNT is sampled de-asserted after the Write access to Memory then the SM will make a state transition to ST_DEGRANTED state provided AHB had requested for MemWrReq or MemRdReq.

In this state if MemRdReq is sampled asserted after the current write access then the SM will make a state transition to ST_READ state.

In this state if there are no further memory access requests then the state transition happens to ST_NO_REQ state.

If it is a wait enabled transfer and if the Wait is asserted before the WstWrCnt counter expires then state transition will happen to ST_WAIT_ASSERTED state.

ST_BURST_WRITE State:

The SM enters this state if a burst write has to be performed for Synchronous Burst devices. In this state the HREADYOUTSMC is driven high on every clock when WaitWrCycVer is de-asserted. This is possible until registered nSMBURSTWAIT signal is not sampled high. When registered nSMBURSTWAIT signal is sampled asserted then WaitWrCycVer is asserted and HREADYOUTSMC is driven low.

When a NewBurst signal is encountered then state transition can happen to ST_WRITE state or ST_READ state depending on whether it is a MemWrReq to the same memory bank or MemRdReq. If no Memory access is requested then state transition will happen to ST_NO_REQ state.

In this state if the SMBUSGNT is sampled de-asserted and there are further MemRdReq or MemWrReq, then state transition will happen to ST_DEGRANTED state.

ST_WAIT_TXRONBUS State:

The SM enters enter this state on following conditions,

- After finishing single memory read access from ST_READ state.
- After finishing burst memory read access from ST_BURST_READ state.
- When SMBUSGNT is de-asserted in the middle of the burst read in ST_BURST_READ state.
- When boundary has reached (BeatCount expiring) then remaining read access has to be performed.

This state was added so that after the current read access the pipelined access is registered by the AHB Memory SM, is sampled in this state, and based on the turnaround information SM can make a state transition to ST_READ, ST_WRITE or ST_TURNAROUND states.

In this state if the SMBUSGNT is sampled de-asserted and there are further MemRdReq or MemWrReq, then state transition will happen to ST_DEGRANTED state.

If no Memory access is requested then state transition will happen to ST_NO_REQ state.

ST_TURNAROUND State:

Turnaround is done on following conditions,

- Read followed by Write to same Bank.
- Read followed by Write to different Bank.
- Read followed by Read to different Bank.
- After Single Read and if IDCYC values are programmed (before de-asserting SMBUSREQ, as there is no further memory accesses).
- After Single Burst Read and if IDCYC values are programmed (before de-asserting SMBUSREQ, as there is no further memory accesses).
- When TimeOutErr has occurred for Read access and if IDCYC values are programmed.
- When WaitEnabled transfer has completed successfully for Read access and if IDCYC values are programmed (before de-asserting SMBUSREQ, as there is no further memory accesses).

From this state SM will make a state transition to ST_DEGRANTED state if SMBUSGNT is sampled de-asserted, and there are further memory access pending. State transition will happen to ST_READ state if MemRdReq is sampled asserted. State transition will happen to ST_WRITE state if MemWrReq is sampled asserted, else state transition will happen to ST_NO_REQ state.

ST_DEGRANTED State:

SM enters this state only if there is some Memory access pending and SMBUSGNT was sampled de-asserted in previous states. In this state SMBUSREQ is asserted and SM makes a transition from this state only when it samples SMBUSGNT asserted. SM will make a state transition to either ST_READ if SMBUSGNT and MemRdReq are sampled asserted, or to ST_WRITE state if SMBUSGNT and MemWrReq is sampled asserted.

ST_WAIT_ASSERTED State:

SM enters this state in case of wait enabled transfers and if the SMWAIT is sampled asserted before WSTRD counter expires in case of Read transfer and WSTWR counter in case of Write transfer. Once the SMWAIT is asserted then SM will wait in this state till SMWAIT is de-asserted. When SMWAIT is sampled de-asserted, SM makes a transition to ST_WAIT_DEASSERTED state. In this state if SMCANCELWAIT is sampled asserted, state transition will happen to ST_CANCEL_WAIT state.

ST_WAIT_DEASSERTED State:

From this state transition will happen to ST_TURNAROUND state if there was no further read request pending, the current waited transfer was read and IDCYC value was programmed for non-zero value. SM will make a state transition to ST_READ state if there is pending read request. SM will make a state transition to ST_WRITE state if there is pending write request.

ST_CANCEL_WAIT State:

From this state SM transition will happen to ST_TURNAROUND state if there was TimeOutErr for Read access and IDCYC register was programmed with a non-zero value, else SM will make a state transition to ST_NO_REQ state.

The SsmcCore uses the Data Bus Interface (DBI) to access the External Memory Data lines. When there is a valid read or a write transaction, the SMBUSREQ signal is asserted, indicating to the DBI that use of the bus is required. The DBI uses the SMBUSGNT line to indicate when there is a successful grant of the bus to the SSMC.

5.6 Pad Interface (SsmcPadIf)

This module is responsible for routing out the Memory signals on to memory. This module is also responsible for accepting the read data from Memory. The read data is bypassed if the access is an asynchronous read, otherwise it is clocked with respect to the fedback clock. Similarly when the Address, control and write data signals are driven out from the SsmcMemTSM, is bypassed if it is for asynchronous access otherwise it is driven out with respect to SMMEMCLKDELAY synchronous access. nSMWEN is either driven out w.r.t nSMMEMCLK for Asynchronous access or it is driven w.r.t SMMEMCLKDELAY for Synchronous access. Similarly nSMBLS follows nSMWEN when RBLE bit is set as 1.

5.7 Clock gating logic (SsmcClkOr)

This module contains the clock gating logic for the synchronous memory clocks that are driven on the output ports, SMCLK [3:0] of the SSMC. A single OR gate implements this. The signal ClkStpd, which is driven by the memory controller, is used for controlling the SMCLK [3:0] during the idle time of the memories. If ClkStpd is low, SMCLK follows SMMEMCLK. If ClkStpd is high, SMCLK goes to HIGH and the clock to the memories is thus disabled.

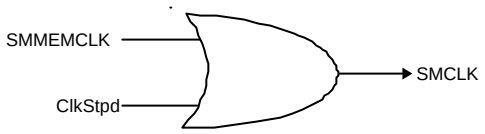


Figure. 5.7-1Clock gating component

This OR gate is instantiated four times at the top level, since 4 clocks are to be driven out on SMCLK [3:0].

5.8 Revision Designator block (SsmcRevAnd)

This module contains a single AND gate to be used as a placeholder cell to mark the Revision of the controller. The 2 input pins are tied-off at the top level of the hierarchy. These "TieOffs" can be identified during layout and re-wired to "VDD" or "VSS" if needed.

This AND gate is instantiated four times at the top level, since the revision number is a four bit quantity. In the present version of the SSMC, all the inputs are tied to zero at the top level. Outputs of all AND gates are concatenated and connected to the output read multiplexer of the AHB slave interface logic. This helps software to identify the revision number through an AHB read.

5.9 SSMC Test Interface Controller (SsmcTIC)

The SSMC TIC bus master module helps to allow externally applied test vectors to be converted into internal AHB transfers.

The TIC is a State Machine that provides an AMBA bus master for system test. It controls the External Test Bus and AHB data buses HRDATATIC and HWDATATIC during the application of Test Interface Format (TIF) test vectors.

The TIC consists of three main blocks:

- Test Vector State Machine - Uses the SMTESTREQA and SMTESTREQB signals to decide which type of vector is being applied on the external TESTBUS.
- Address and Control Registers - Holds values for the address and control signals and includes an address

incrementer for burst accesses.

- AMBA Bus Master Interface - Includes the AMBA bus master State Machine and the control of the AMBA bus signals.

The details of the TIC can be referred to, in the ARM PrimeCell Multiport Memory Controller (PL093) Technical Reference Manual [Ref. 1] and chapter 6, of the AMBA Specification (Rev 2.0) [Ref. 3].

5.10 Clock Divider (SsmcDerivedClk)

This module is responsible for generating SlowClkM signal, which is used to synchronise the signals traversing from different clock domain.

SlowClkM is used to indicate the slower of SSMCCLK and HCLK to the memory controller. Generating a signal called SSMCCLKDiv2, which is at half the frequency of HCLK, generates this signal. SSMCCLKDiv2 is driven on SlowClkM if the ClockRatio vector from the AHB register interface indicates that the HCLK: SMMEMCLKfrequency ratio is 1:2. Else, a '1' is driven on SlowClkM. Similarly the signal SSMCCLKDiv3 is generated, which is one-third the frequency of HCLK. SSMCCLKDiv3 is driven on SlowClkM if the ClockRatio vector from the AHB register interface indicates that the HCLK: SMMEMCLK frequency ratio is 1:3. Else, a '1' is driven on SlowClkM.

6 DESIGN ASSUMPTIONS

Some assumptions have been made in the design of the SSMC (PL093). The user is required to keep the system assumptions in mind while connecting the SSMC (PL093) in an AHB system. The software assumptions are to be kept in mind while programming the SSMC and while accessing the memory through the SSMC.

6.1 Software Assumptions

The following software assumptions have been made regarding programming of the SSMC:

1. The content of SMBWSTOENRx should be less than or equal to SMBWSTRDx value.
2. The content of SMBWSTWENRx should be less than or equal to SMBWSTWRx value.
3. nSMBURSTWAIT from memory should be assertted 1 clock earlier for the memory access it needs to wait.
4. Register programming should be done when memory access is not happening.

6.2 System Assumptions

6.2.1 Power-on reset values of Input pins

- On power on reset, the static memory bank7 memory width pins (SMMWCS7) should be driven to "00" for 8-bit, "01" for 16-bit and "10" for 32-bit databus width This value will be used until the SMBCR7 register is written into. Once the register is programmed, the memory width bits of the register will be used to determine the memory width.
- On power on reset, the static memory byte lane polarity pins (SMBLS7POL) should indicate the polarity of the byte lane selects (0 for active LOW and 1 for active HIGH). These values will be used until the SMBCRx register is written into.

- On power on reset, the endian mode of the SSMC should be driven on the SMBIGENDIAN pin.
- On power on reset, external bus interface indicator of SSMC should be driven on the SMEXTBUSMUX pin.

6.2.2 Write enable connection of static memory devices

- For memory banks constructed from 8-bit or non byte-partitioned memory devices, the nSMBLS[3:0] signals should be connected to write enable (nWE) inputs of each 8-bit memory. The nSMWEN signal from the SSMC should not be used.
- For memory banks constructed from 16-bit or 32-bit memory devices, the nSMBLS[3:0] signals should be connected to the byte select inputs of each memory device. The nSMWEN signal from the SSMC should be connected to the write enable inputs of each memory device.

6.2.3 Address connection of static memory device

When external memory width is 8-bits or 16-bits or 32-bits, SMADDR [25:0] should be connected to the A[25:0] pins of the static memory device.

6.2.4 Entry into Test mode

- On the AHB, it is assumed that the processor is always requesting access to the bus, but the TIC has the highest priority to bus access.
- Once the SSMC has entered test mode, it is assumed that there will be no normal mode accesses. The SSMC will enter test mode only if the sequence of events as indicated below is followed. The following sequence of events has to be used to use the SSMC's TIC to apply test patterns:
  - Apply reset (HRESETn) asynchronously.
  - When the reset is removed asynchronously, the SSMC enters TIC test mode, if the SMITCR is programmed for test mode.
  - The TIC block in the SSMC requests for the access to the external bus through the assertion of the SMTICBUSREQEBI signal. Once SMTICBUSGNTEBI is sampled asserted, the TIC block takes control of the external bus and relinquishes the bus only after it has completed all the transfers on the AHB master interface.
  - The TIC block asserts its HBUSREQTIC signal to the arbiter, requesting access to the AHB bus. Because the TIC is the highest priority, HGRANTTIC is asserted and the granted TIC takes ownership of the AHB bus.

7 SSMC VERIFICATION TESTBENCH

The SSMC Functional Verification testbench is an AHB BusTalk testbench. The VHDL and Verilog BusTalk testbench files reside in the ***verification/vhdl*** and ***verification/verilog*** directories respectively.

7.1 SSMC VHDL Verification Testbench

The ***Figure 7.1-1*** shows the test environment of SSMC. The Testbench contains instantiation of TrickMem block, which serves the purpose of emulating memory models. These will be used for the verification of the SSMC. The Testbench also has a SSMC Trickbox block, which is used to verify the non-memory related signals. It is possible to access the Memory Models (TrickMemory, which is being instantiated eight times in TrickMem), both from the SSMC and the AHB. The slave Testbench is responsible for programming the slave registers in Trickbox, TrickMem and UUT (SSMC). The Trickbox also does all the protocol checking and drives the non-AMBA signals to the SSMC.

Figure 7.1-1 illustrates the Bustalk VHDL testbench for running test vectors.

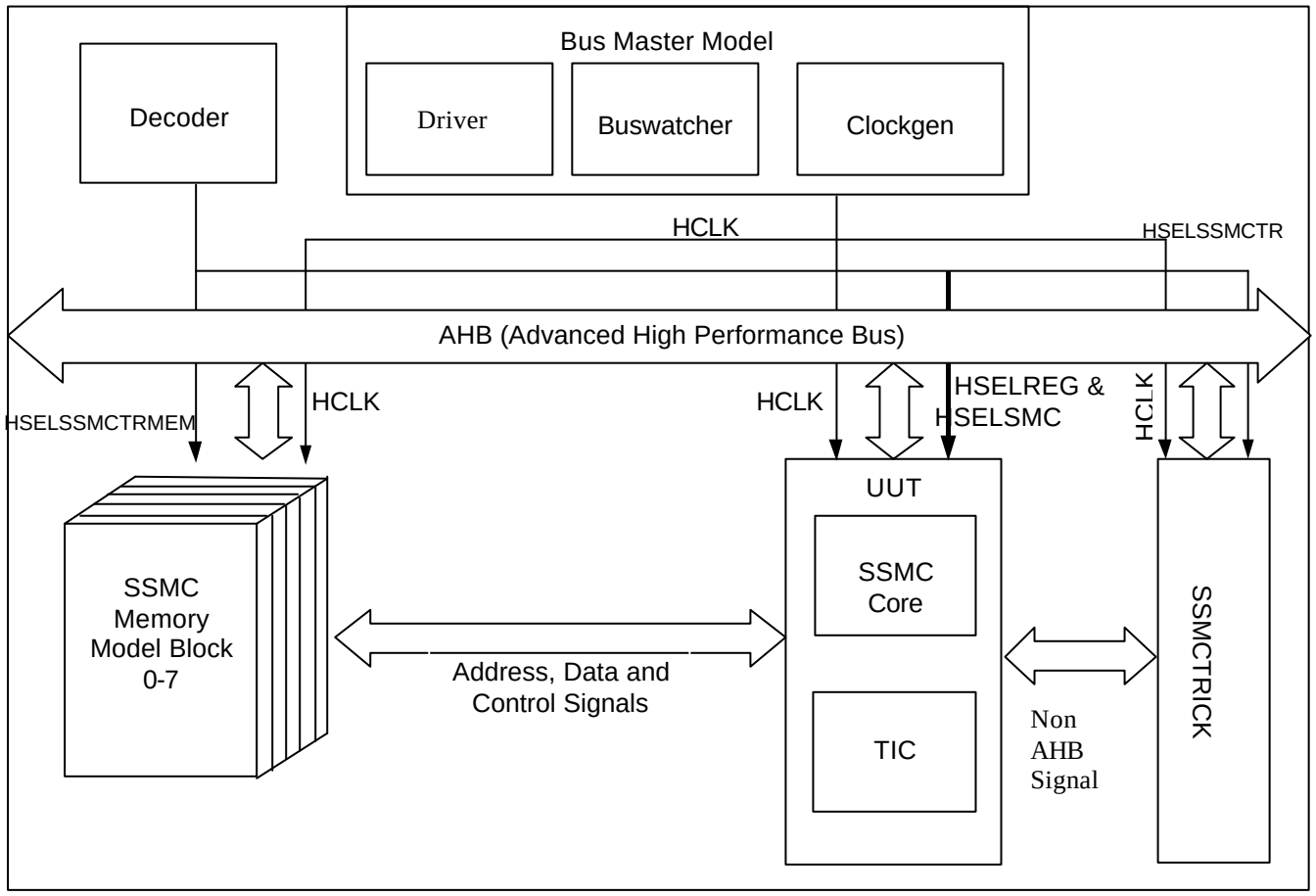


Figure 7.1-1 VHDL Functional Verification Testbench

Figure 7.1-2 illustrates the hierarchy for the VHDL testbench.

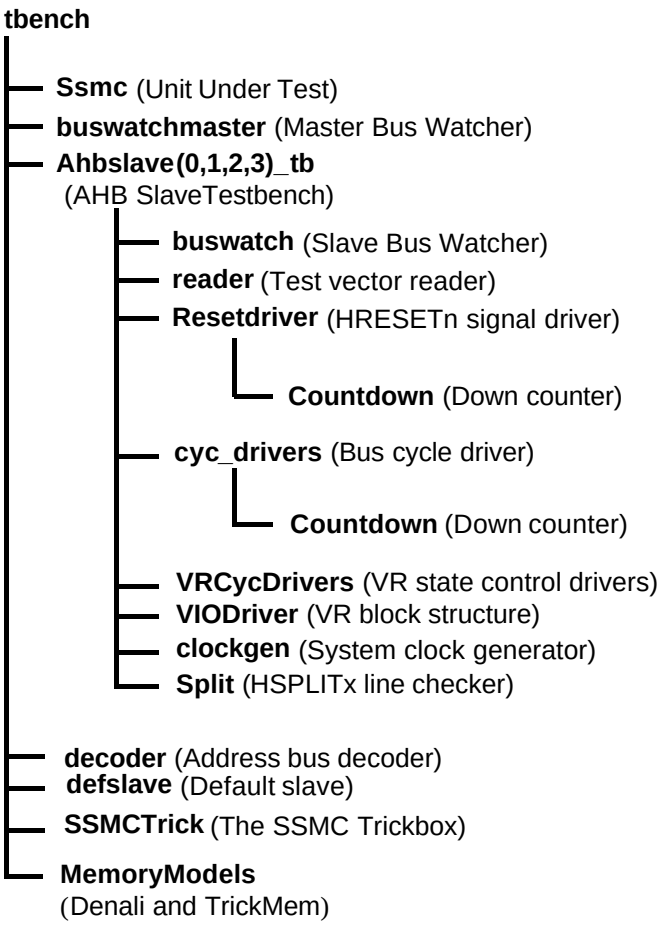


Figure 7.1-2 VHDL Functional Verification Testbench hierarchy

7.1.1 Testbenches

The testbenches, **tbench.vhd**, **tbench_denali.vhd, tbench_denali1.vhd** and **tbench_denali2.vhd** are used for functional testing of the Synchronous Static Memory controller (SSMC). Every testbench block instantiates the SSMC (Unit Under Test), the Trickbox and the memory models. Different memory models are used in different test benches. The memory models used are of two types – trick memory models that are the behavioural model of a static memory device, and the Denali memory models.

All the Scan input pins are tied low to keep them from interfering with the test process.

The HCLK frequency and AHB signal timing budgets are defined in the timing.vhd file.

7.1.2 Unit Under Test

The Unit Under Test is

- SSMC VHDL RTL

7.1.3 Bus Master Protocol Checker

The testbench instantiates the master bus watcher, buswatchmaster.vhd that checks the protocol of master accesses initiated via the test code. This module implements the protocol checks for the AHB master's output signals and will flag warning and error messages if any timing or protocol violations are detected during simulations.

7.1.4 AHB Master/Arbiter Model

The UUT is stimulated primarily by AHB Master/Arbiter behavioural model. The AHB Master/Arbiter block issue commands on the AHB bus under program control.

There are two AHB master ports, but at any instant of time only one can be active, hence one AHB Master/Arbiter model, ahbslave_tb.vhd instantiates the buswatcher, the reader, the reset driver, the cycle drivers, the VR cycle driver, the split checker and the clock generator. The reader block reads the test vectors from the .bif file (from the ***verification/bustest/invec*** directory) one line at a time and drives the information about the bus cycle in the form of packets. If the information driven by the reader indicates that the current cycle is a reset cycle, the reset driver drives the HRESETn signal with the correct timings. The cyc_drivers drive the address, control and data signals for each AHB cycle type, i.e. HSR and HSW etc. The VRCycdrivers selects the appropriate VR registers for a read or write operation. The buswatch module implements the protocol checks for the AHB slave's output signals and flags warning and error messages on detection of mismatches. The Split module verifies that the HMASTER number is asserted on the HSPLITx lines on the expected clock. The clockgen module generates the system clock, HCLK.

The reader reads test vectors from the infile.bif residing in the ***verification/bustest/invec*** directory.

7.1.5 Decoder

This module decodes the addresses and generates the HSELx signals for the slaves. It can generate the select signals for a maximum of 16 slaves.

Any access to the address range

0x00000000 to 0x07FFFFFF

Will access the UUT memory interface via the HSEL[0] select line.

Any access to the address range

0x08000000 to 0x0FFFFFFF

Will access the UUT memory interface via the HSEL[1] select line.

Any access to the address range

0x10000000 to 0x17FFFFFF

Will access the UUT memory interface via the HSEL[2] select line.

Any access to the address range

0x18000000 to 0x1FFFFFFF

Will access the UUT memory interface via the HSEL[3] select line.

Any access to the address range

0x20000000 to 0x27FFFFFF

Will access the UUT memory interface via the HSEL[4] select line.

Any access to the address range

0x28000000 to 0x2FFFFFFF

Will access the UUT memory interface via the HSEL[5] select line.

Any access to the address range

0x30000000 to 0x37FFFFFF

Will access the UUT memory interface via the HSEL[6] select line.

Any access to the address range

0x38000000 to 0x3FFFFFFF

Will access the UUT memory interface via the HSEL[7] select line.

Any access to the address range

0x40000000 to 0x7FFFFFFF

Will access the UUT Register interface via HSEL[8] select line.

Any access to the address range

0x80000000 to 0xBFFFFFFF

Will access the Trickbox register via the HSEL[9] select line.

Any access to the address range

0xC0000000 to 0xEFFFFFFF

Will access the TrickMem via the HSEL[10] select line.

Accesses to any address outside the above ranges will select the Default Slave.

7.1.6 Default Slave

This module drives the response when an access is detected to an unmapped region. The Default slave drives its HREADY output high, when other slaves are not selected. It drives a zero wait State OK response for IDLE and BUSY transfers and a two cycle ERROR response for an NSEQ or SEQ transfer access to an unmapped address.

7.1.7 Protocol Checker

The testbench includes a protocol checker, which reports any protocol or timing violations during accesses to any AHB slave. Timing parameters can be set using the timing.vhd file in the ***verification/vhdl/tbench*** directory. The buswatch module residing in the ***verification/vhdl/buswatcher*** directory checks the Read/Write databus timings and reports any timing violations.

7.1.8 Trickbox

The functionality of the SSMC is verified via the Trickbox. The Trickbox is a programmable AHB slave designed to drive stimulus on the non-AHB input terminals of the SSMC and sample it's non-AHB outputs. It samples the output lines from the SSMC and checks for the protocols. It flags error messages for any unexpected behaviour detected on the SSMC memory signals.

7.1.9 Trickbox Memory Model

The TrickMem is a programmable static memory model, which also is an AHB slave. The TrickMem module is used for testing memory related functionality of the SSMC. The SSMC writes into the TrickMem memory model and the data can be verified via both through the SSMC and the AHB. The TrickMem model checks for the timings of the memory control signals. When timing or protocol violation is detected on the SSMC memory control signals, the TrickMem flags error messages.

7.1.10 Denali Memory Models

The Denali Memory simulation models are used for verification of the SSMC. Memory transfers are done to the Denali models through the SSMC and transfers are verified by reading out the data from the memory through the SSMC. The SOMA and initialising files for the Denali models are located in the ***verification/denali*** directory. The SOMA files are *.soma files and the memory initialising files are *.dat files.

7.1.11 Simulation environment

Generics

The SSMC testbench provides some configurability options using generics. All these generics except SuppressOnReset are configurable through the tbench.vhd file. SuppressOnReset is configurable through the ahbslave_tb.vhd file in the ***verification/vhdl/tbench*** directory.

- XonSig

XonSig is used in the cyc_drivers.vhd file that resides in ***verification/vhdl/bid***.

This generic is used to eliminate the ‘X’s that appear on the address and control signals when these signals are not being actively driven. If XonSig is set to 0 in the top level of the testbench, these signals go to ‘0’ when the corresponding line driver is inactive. If set to 1, the signals go to ‘X’ when the line driver is idle.

- Verbosity

Verbosity is used in the reset_driver.vhd and the cyc_drivers.vhd, which reside in ***verification/vhdl/bid***. It is also used in the buswatch.vhd file that resides in ***verification/vhdl/buswatcher*** and the VIODrivers.vhd, which reside in ***verification/vhdl/vr***.

This generic is used to control the verbosity of the transcript generated during simulation. If Verbosity is set to 1, every command issued will have a corresponding message in the transcript file. If Verbosity is set to 0, a message is issued only if an error occurs on a read.

- HaltOnMismatch

This is used in the cyc_drivers.vhd, which resides in ***verification/vhdl/bid***, the buswatch.vhd which resides in ***verification/vhdl/buswatcher*** and the VIODriver.vhd which resides in ***verification/vhdl/vr***.

This generic is used to stop the simulation if an error occurs. If set to 1, the simulation halts after an error is reported. If set to 0, the error is flagged, but the simulation continues.

- Databuswidth

Databuswidth is used in the cyc_drivers.vhd, which resides in ***verification/vhdl/bid.***

This generic controls the width of the AHB databus. For the SSMC testbench this is set to 32.

- SuppressOnReset

SuppressOnReset is used in the buswatch.vhd, which resides in ***verification/vhdl/buswatcher.***

This generic is used to suppress all protocol checks on the slave's output signals when the HRESETn signal is asserted. If SuppressOnReset is TRUE, the set-up and hold timing violations on these signals are not flagged when HRESETn is asserted. If SuppressOnReset is set to FALSE, the set-up and hold timing violations are flagged irrespective of HRESETn. When SuppressOnReset is FALSE the slave's output signals are not checked for 'X'es when HRESETn is asserted.

Test Vectors

The test vectors for the verification of the SSMC are coded into separate C files, in the ***verification/bustest*** directory. There are separate test files created for each category tests described in the verification plan.

Running Simulations

The **README** file in the ***verification/vhdl/tbench*** directory gives step by step instructions for running simulations using the SSMC test vectors on the VHDL source code.

Run time logs for all the combinations are available in the ***verification/vhdl/Ssmc_rtl*** directory.

7.2 SSMC Verilog Verification Testbench

The ***Figure 7.1-1*** shows the test environment of SSMC. The Testbench contains instantiation of TrickMem block, which serves the purpose of emulating memory models. These will be used for the verification of the SSMC. The Testbench also has a SSMC Trickbox block, which is used to verify the non-memory related signals. It is possible to access the Memory Models (TrickMemory, which is being instantiated eight times in TrickMem), both from the SSMC and the AHB. The slave Testbench is responsible for programming the slave registers in Trickbox, TrickMem and UUT (SSMC). The Trickbox also does all the protocol checking and drives the non-AMBA signals to the SSMC.

Figure 7.2-1 illustrates the Bustalk VHDL testbench for running test vectors.

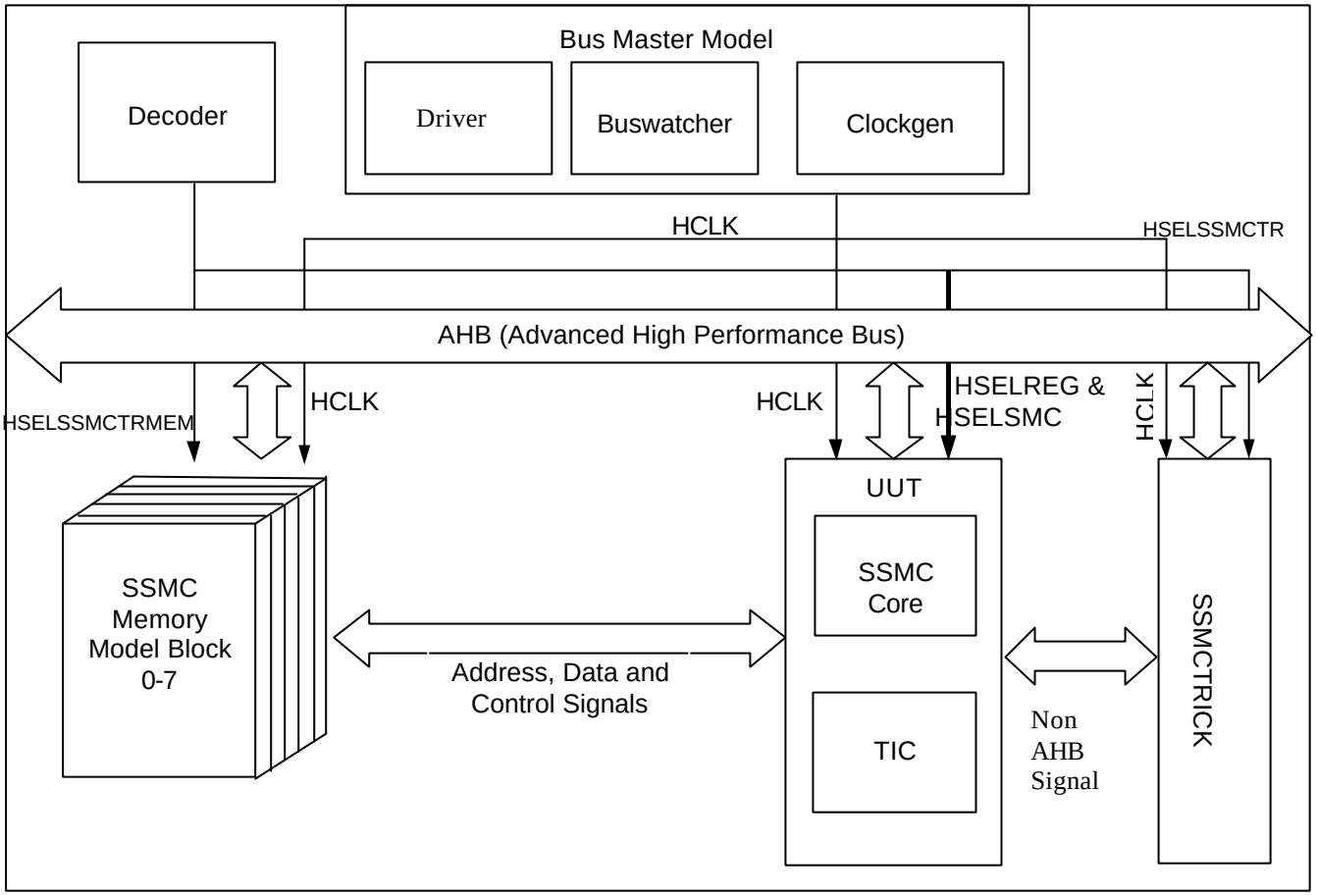


Figure 7.2-1 Verilog Functional Verification Testbench

Figure 7.2-2 illustrates the hierarchy for the Verilog testbench.

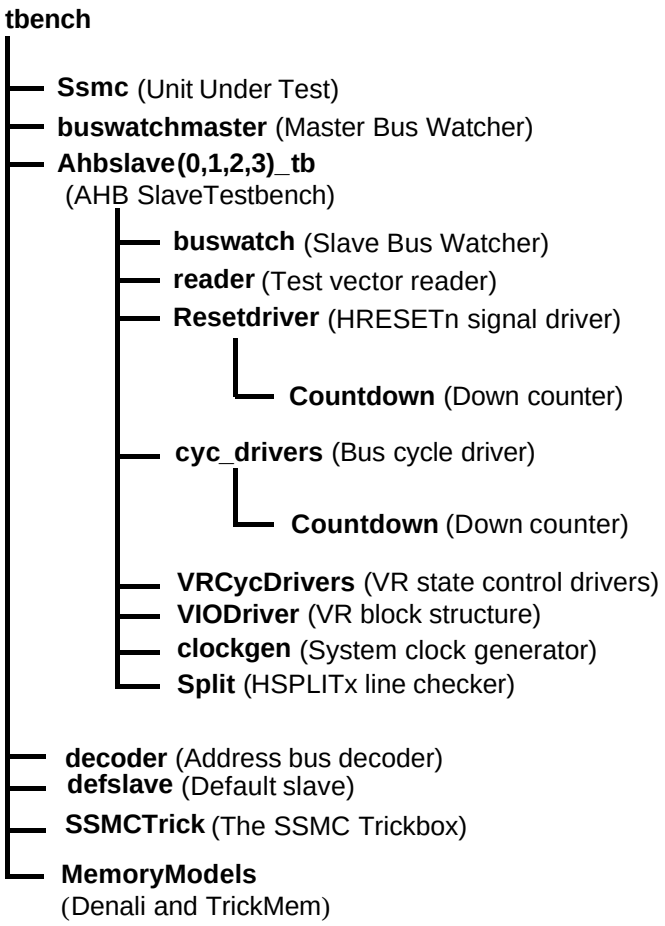


Figure 7.2-2 Verilog Functional Verification Testbench hierarchy

7.2.1 Testbenches

The testbenches, **tbench.v**, **tbench_denali.v, tbench_denali1.v** and **tbench_denali2.v** are used for functional testing of the Synchronous Static Memory controller (SSMC). Every testbench block instantiates the SSMC (Unit Under Test), the Trickbox and the memory models. Different memory models are used in different test benches. The memory models used are of two types – trick memory models that are the behavioural model of a static memory device; and the Denali memory models.

All the Scan input pins are tied low to keep them from interfering with the test process.

The HCLK frequency and AHB signal timing budgets are defined in the timing.v file.

7.2.2 Unit Under Test

The Unit Under Test may be one of the following,

SSMC Verilog RTL

SSMC scan-insert Verilog netlist from Verilog Source

SSMC non-scan Verilog netlist from VHDL Source

SSMC non-scan Verilog netlist from Verilog Source

Creating a link [named uut] in the verification/verilog directory to the corresponding source directory may reference one out of these three versions. [Note that this link will be created automatically by the simulation makefiles].

7.2.3 Bus Master Protocol Checker

The testbench instantiates the master bus watcher, buswatchmaster.v, which checks the protocol of master accesses, initiated via the test code. This module implements the protocol checks for the AHB master's output signals and flags warning and error messages if any timing or protocol violations are detected during simulations.

7.2.4 AHB Master/Arbiter Model

The UUT is stimulated primarily by AHB Master/Arbiter behavioural model. The AHB Master/Arbiter block issue commands on the AHB bus under program control.

There are two AHB master ports, but at any instant of time only one can be active, hence one AHB Master/Arbiter model, ahbslave_tb.vhd instantiates the buswatcher, the reader, the reset driver, the cycle drivers, the VR cycle driver, the split checker and the clock generator. The reader block reads the test vectors from the .bif file (from the ***verification/bustest/invec*** directory) one line at a time and drives the information about the bus cycle in the form of packets. If the information driven by the reader indicates that the current cycle is a reset cycle, the reset driver drives the HRESETn signal with the correct timings. The cyc_drivers drive the address, control and data signals for each AHB cycle type, i.e. HSR and HSW etc. The VRCycdrivers selects the appropriate VR registers for a read or write operation. The buswatch module implements the protocol checks for the AHB slave's output signals and flags warning and error messages on detection of mismatches. The Split module verifies that the HMASTER number is asserted on the HSPLITx lines on the expected clock. The clockgen module generates the system clock, HCLK.

The reader reads test vectors from the infile.bif residing in the ***verification/bustest/invec*** directory.

7.2.5 Decoder

This module decodes the addresses and generates the HSELx signals for the slaves. It can generate the select signals for a maximum of 16 slaves.

Any access to the address range

0x00000000 to 0x07FFFFFF

Will access the UUT memory interface via the HSEL[0] select line.

Any access to the address range

0x08000000 to 0x0FFFFFFF

Will access the UUT memory interface via the HSEL[1] select line.

Any access to the address range

0x10000000 to 0x17FFFFFF

Will access the UUT memory interface via the HSEL[2] select line.

Any access to the address range

0x18000000 to 0x1FFFFFFF

Will access the UUT memory interface via the HSEL[3] select line.

Any access to the address range

0x20000000 to 0x27FFFFFF

Will access the UUT memory interface via the HSEL[4] select line.

Any access to the address range

0x28000000 to 0x2FFFFFFF

Will access the UUT memory interface via the HSEL[5] select line.

Any access to the address range

0x30000000 to 0x37FFFFFF

Will access the UUT memory interface via the HSEL[6] select line.

Any access to the address range

0x38000000 to 0x3FFFFFFF

Will access the UUT memory interface via the HSEL[7] select line.

Any access to the address range

0x40000000 to 0x7FFFFFFF

Will access the UUT Register interface via HSEL[8] select line.

Any access to the address range

0x80000000 to 0xBFFFFFFF

Will access the Trickbox register via the HSEL[9] select line.

Any access to the address range

0xC0000000 to 0xEFFFFFFF

Will access the TrickMem via the HSEL[10] select line.

Accesses to any address outside the above ranges will select the Default Slave.

7.2.6 Default Slave

This module drives the response when an access is detected to an unmapped region. The Default slave drives its HREADY output high, when other slaves are not selected. It drives a zero wait State OK response for IDLE and BUSY transfers and a two cycle ERROR response for an NSEQ or SEQ transfer access to an unmapped address.

7.2.7 Protocol Checker

The testbench includes a protocol checker, which reports any protocol or timing violations during accesses to any AHB slave. Timing parameters can be set using the timing.v file in the ***verification/verilog/tbench*** directory. The buswatch module residing in the ***verification/verilog/buswatcher*** directory checks the Read/Write databus timings and reports any timing violations.

7.2.8 Trickbox

The functionality of the SSMC is verified via the Trickbox. The Trickbox is a programmable AHB slave designed to drive stimulus on the non-AHB input terminals of the SSMC and sample it's non-AHB outputs. It samples the output lines from the SSMC and checks for the protocols. It flags error messages for any unexpected behaviour detected on the SSMC memory signals.

7.2.9 Trickbox Memory Model

The TrickMem is a programmable memory model, which also is an AHB slave. The TrickMem module is used for testing memory related functionality of the SSMC. The SSMC writes into the TrickMem memory model and the data can be verified via both through the SSMC and the AHB. The TrickMem model checks for the timings of the memory control signals. When timing or protocol violation is detected on the SSMC memory control signals, the TrickMem flags error messages.

7.2.10 Denali Memory Models

The Denali Memory simulation models are used for verification of the SSMC. Memory transfers are done to the Denali models through the SSMC and transfers are verified by reading out the data from the memory through the SSMC. The SOMA and initialising files for the Denali models are located in the ***verification/denali*** directory. The SOMA files are *.soma files and the memory initialising files are *.dat files.

7.2.11 Simulation Environment

Simulation configuration (Parameters)

The SSMC testbench provides some configurability options via parameters. All the parameters are configurable through the tbench.v file.

- XonSig

XonSig is used in the cyc_drivers.v file that resides in ***verification/verilog/bid***.

This parameter is used to eliminate the ‘X’s that appear on the address and control signals when these signals are not being actively driven. If XonSig is set to 0 in the top level of the testbench, these signals go to ‘0’ when the corresponding line driver is inactive. If set to 1, the signals go to ‘X’ when the line driver is idle.

- Verbosity

Verbosity is used in the reset_driver.v and the cyc_drivers.v, which reside in ***verification/verilog/bid***. It is also used in the buswatch.v that resides in ***verification/verilog/buswatcher*** and the VIODrivers.v, which reside in ***verification/verilog/vr***.

This parameter is used to control the verbosity of the transcript generated during simulation. If Verbosity is set to 1, every command issued will have a corresponding message in the transcript file. If Verbosity is set to 0, a message is issued only if an error occurs on a read.

- HaltOnMismatch

This is used in the cyc_drivers.v, which resides in ***verification/verilog/bid***, the buswatch.v which resides in ***verification/verilog/buswatcher*** and the VIODrivers.v which resides in ***verification/verilog/vr***.

This parameter is used to stop the simulation if an error occurs. If set to 1, the simulation halts after an error is reported. If set to 0, the error is flagged but the simulation continues.

- SuppressOnReset

SuppressOnReset is used in the buswatch.v, which resides in ***verification/verilog/buswatcher.***

This parameter is used to suppress all protocol checks on the slave's output signals when the HRESETn signal is asserted. If SuppressOnReset is TRUE, the set-up and hold timing violations on these signals are not flagged when HRESETn is asserted. If SuppressOnReset is set to FALSE, the set-up and hold timing violations are flagged irrespective of HRESETn. When SuppressOnReset is FALSE the slave’s output signals are not checked for ‘X’es when HRESETn is asserted.

- TestMode

TestMode is used in the cyc_drivers.v, which resides in ***verification/verilog/bid.***

If the TestMode parameter is set, the testbench is configured to run in Big Endian mode. By default this parameter is cleared.

- Databuswidth

Databuswidth is used in the cyc_drivers.v, which resides in ***verification/verilog/bid.***

This parameter controls the width of the AHB databus. For the SSMC testbench this is set to 32.

Test Vectors

The test vectors for the verification of the SSMC are coded into separate C files, in the ***verification/bustest*** directory. There are separate test files created for each category tests described in the verification plan. Make options have been provided to use them individually.

Running Simulations

The **README** file in the ***verification/verilog/tbench*** directory gives step by step instructions for running simulations using tests on the Verilog source code.

Run time logs for all the combinations are available in the ***verification/verilog/Ssmc_rtl, verification/verilog/Ssmc_noscan_vhdl_net_min*** and ***verification/verilog/Ssmc_scanready_verilog_net_typ*** directories.

7.3 Trickbox Description

The Trickbox is a programmable AHB slave, designed to drive stimulus on the non-AHB input terminals of the Multiport Memory Controller and to sample the non-AHB outputs of the same. The Trickbox also provides a way for Data Transfer Protocol Validation.

The Trickbox provides an AHB interface for accessing the internal registers and controlling the data transfer.

The basic blocks of the Trickbox are illustrated in ***Figure 7.3-1 Block diagram of the Trickbox.***

The block diagram is shown in the ***Figure. 7.3-1.***

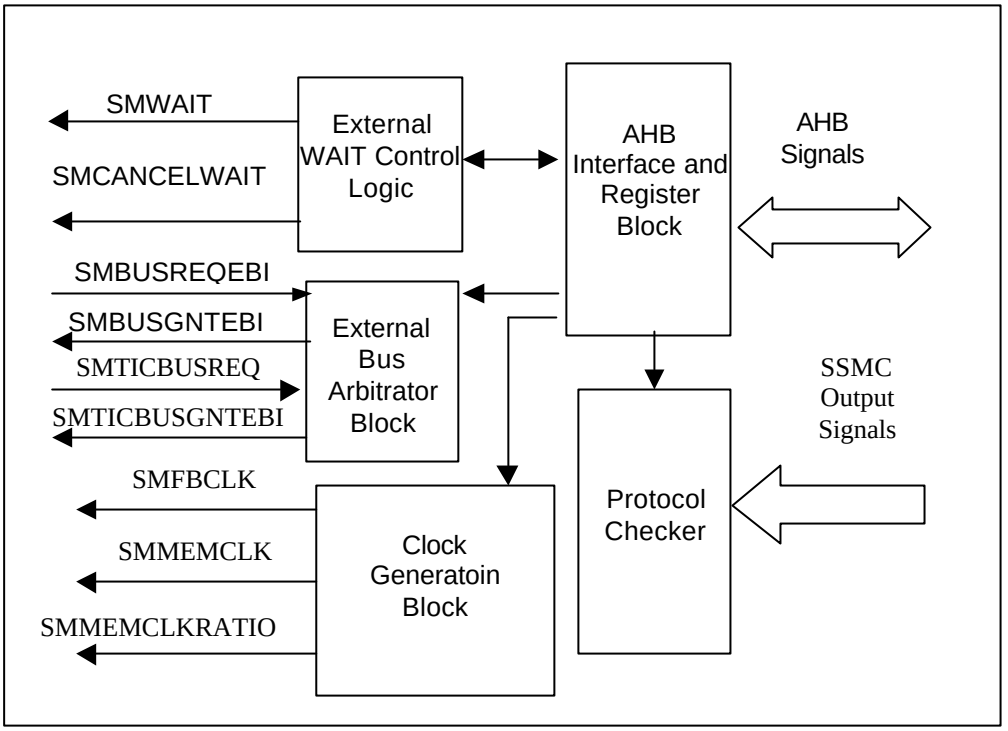


Figure 7.3-1 SSMC Trickbox Block Diagram

7.3.1 Trickbox Signal Description

Multiport Memory Controller Behavioural Trickbox Signals are listed in ***Table 7.3-1*** and ***Table 7.3-2***

Name	Type	Source / Destination	Description
HCLK	IN	Clock Generator	AHB Bus Clock. This clock times all bus transfers. All signal timings are referenced to the rising edge of HCLK.
HRESETn	IN	Reset Controller	Bus reset (active low).
HADDR[5:2]	IN	AHB Master	The 4-bit system address bus.
HTRANS[1:0]	IN	AHB Master	Indicates the type of the current transfer.
HSIZE[2:0]	IN	AHB Master	Indicates the size of the current transfer.
HWRITE	IN	AHB Master	AHB transfer direction signal, indicates a write access when HIGH and read access when LOW.
HWDATA [31:0]	IN	AHB Master	AHB write data bus. During write operations, HWDATA is used to transfer data from the bus master to the slaves.
HREADYINTr	IN	External Slave(s)	The slave will start the transfer only if HREADYINTr is HIGH.
HSELSMCTr	IN	AHB Decoder	SSMC Trick select
HREADYOUTTr	OUT	AHB Slave	HREADYOUTTr along with an OK response indicates the successful completion of a data transfer.
HRESPTr[1:0]	OUT	AHB Slave	The bus transfer response.

Name	Type	Source / Destination	Description
HRDATATr[31:0]	OUT	Test Bench	AHB read data bus. It is used to transfer data from bus slaves to the master during read operations.

Table 7.3-1 AHB signal descriptions

Signal Name	Type	Source / Destination	Description
SMDATAOUT[31:0]	IN	UUT	External Data Out signal from the SSMC. It has been taken in to check for ‘X’es on the bus.
SMADDR[25:0]	IN	UUT	Memory address bus of the SSMC. It has been taken in so as to check for ‘X’es on the bus.
nSMDATAEN[3:0]	IN	UUT	Data bus enable signal. It has been taken in so as to check for ‘X’es on the bus.
nSMWEN	IN	UUT	Memory Write enable signal. It has been taken in so as to check for ‘X’es on the line.
nSMBLS[3:0]	IN	UUT	Memory Bus Lane Enable signals. It has been taken in so as to check for ‘X’es on the bus.
SSMTrCS[7:0] & nSSMTrCS[7:0]	IN	UUT	Chip select signals of each bank. It has been taken in to check for ‘X’es on the bus also.
nSMOEN	IN	UUT	Memory output enable signal. It has been taken in so as to check for ‘X’es on the line.
SMBUSREQEBI	IN	UUT	When SMEXTBUSMUX is 1, then SSMC request for the external bus.
SMTICBUSREQEBI	IN	UUT	TIC requested for External bus.
BIGENDIAN	OUT	UUT	This static configuration bit indicates the type of Endianness of the system. 0 = Litle Endian mode 1 = Big Endian mode
SMEXTBUSMUX	OUT	UUT	This static configuration bit indicates the use of the internal bus MUX or external bus MUX. 0 = Internal bus MUX (DBI) 1 = External Bus MUX (EBI)
SMMWCS7[1:0]	OUT	UUT	Boot memory width.
SMWAIT	OUT	UUT	External Wait input signal into the SSMC.
SMCANCELWAIT	OUT	UUT	External WAIT Cancel Signal. This signal is asserted when the SMWAIT signal is timed out. The time out period is programmable cycles of HCLK.
SMBUSGNTEBI	OUT	UUT	Indicate bus is granted to SSMC.
SMTICBUSGNTEBI	OUT	UUT	Indicate but is granted to TIC.
SMBUSBACKOFFEBI	OUT	UUT	Indicates the UUT to finish the operation as soon as possible when higher priority request is there.
SMFBCLK	OUT	UUT	Feedback clock to UUT.
SMMEMCLK	OUT	UUT	Memory clock to UUT.

SMMEMCLKRATIO[1:0]	OUT	UUT	Indicates HCLK to SMMEMCLKratio.

Table 7.3-2 SSMC Interface signal description

7.3.2 Trickbox register summary

The Base address of the Trickbox is not fixed and depends on the system implementation. However, the offset of any particular register from the Base address is fixed.

The SSMC Trickbox registers are shown in ***Table7.3-1***.

Address	Read Location	Write Location
SSMC Trick Base + 0x0000	SSMCTrMWCS	SSMCTrMWCS
SSMC Trick Base + 0x0004	SSMCTrExtMux	SSMCTrExtMux
SSMC Trick Base + 0x0008	SSMCTrEndian	SSMCTrEndian
SSMC Trick Base + 0x000C	SSMCTrCS2WTR0	SSMCTrCS2WTR0
SSMC Trick Base + 0x0010	SSMCTrCS2WTR1	SSMCTrCS2WTR1
SSMC Trick Base + 0x0014	SSMCTrCS2WTR2	SSMCTrCS2WTR2
SSMC Trick Base + 0x0018	SSMCTrCS2WTR3	SSMCTrCS2WTR3
SSMC Trick Base + 0x001C	SSMCTrCS2WTR4	SSMCTrCS2WTR4
SSMC Trick Base + 0x0020	SSMCTrCS2WTR5	SSMCTrCS2WTR5
SSMC Trick Base + 0x0024	SSMCTrCS2WTR6	SSMCTrCS2WTR6
SSMC Trick Base + 0x0028	SSMCTrCS2WTR7	SSMCTrCS2WTR7
SSMC Trick Base + 0x002C	SSMCTrWTCNCL	SSMCTrWTCNCL
SSMC Trick Base + 0x0030	SSMCTrCR	SSMCTrCR

Table 7.3-1 SSMC Trickbox Register map

SSMCTrMWCS: SSMC Trickbox Memory Width Register

Bit	Name	R/W	Reset value	Function
1 : 0	MWCS	R/W	Unaffected by the HRESETn	MWCS bits determine the width of the boot memory. These bits are clocked but are not affected by HRESETn so values are programmed in these bits after Reset is over and these values remain even after reset is re-asserted. These bits are used to check SMMWCS7 functionality. 0x00 for 8-bit wide memory 0x01 for 16-bit wide memory 0x10 for 32-bit wide memory 0x11 for 8-bit wide memory

Table 7.3-2 SSMC Trickbox Memory width register

SSMCTrExtMux: SSMC Trickbox ExtMux Register

Bit	Name	R/W	Reset value	Function

0	ExtMUX	R/W	0x00	This static configuration bit indicates if the internal bus multiplexer (DBI) is used or External bus multiplexer is used. 0 = Internal bus MUX (DBI) 1 = External bus MUX.

Table 7.3-3 SSMC Trickbox ExtMux register

SSMCTrEndian: SSMC Trickbox Endian Register

Bit	Name	R/W	Reset value	Function
0	Endian	R/W	0x00	This static configuration bit indicates the type of Endianness of the system. The value put in this register is driven on BIGENDIAN input of SSMC. 0 = LITTLE Endian mode 1 = BIG Endian mode

Table 7.3-4 SSMC Trickbox Endian register

SSMCTrCS2WT0-7: SSMC Trickbox Enables SMWAIT Register

Bit	Name	R/W	Reset value	Function
11	WaitEn	R/W	0x0	This bit indicates whether the external wait timing is enabled for the currently targeted Bank. 0 = External Wait disabled. 1 = External Wait Enabled.
10	WaitPol	R/W	0x0	This bit indicates the polarity of the SMWAIT for currently targeted Bank. 0 = Active LOW SMWAIT 1 = Active HIGH SMWAIT
9 : 5	CS2WTRx	R/W	0x00	Chip Select assertion to SMWAIT assertion delay count for Bank 0-7. Chip select assertion is taken as the trigger point and the SMWAIT is asserted after a delay of (CS2WTRx) x Hclk.
4 : 0	WT2DeWT	R/W	0x00	SMWAIT assertion to SMWAIT de-assertion delay count for Bank x. The SMWAIT is de-asserted after a delay of (WT2DeWTx) x Hclk.

Table 7.3-5 SSMC Trickbox Chip Select to SMWAIT assertion delay register

SSMCTrWTCNCL: SSMC Trickbox WTCNCL delay Register

Bit	Name	R/W	Reset value	Function

8	SMCnclEn	R/W	0x0	This bit will mask the SMCANCELWAIT signal. 0 = Then SMCANCELWAIT will be de-asserted though out the operation. 1 = Then SMCANCELWAIT will function according to the programmed delays.
7	SMWAITIgnore	R/W	0x0	This bit will indicate when to assert SMCANCELWAIT signal. 0 = Then WTCNCL starts counting ignoring SMWAIT and CS. 1 = Then WTCNCL will start counting only if SMWAIT is asserted.
6 : 0	WTCNCL	R/W	0x00	The SMCANCELWAIT will be asserted after this delay count. This will be activated after the assertion of external wait signal.

Table 7.3-6 SSMC Trickbox WTCNCL delay Register

7.3.3 SSMC Static Memory Model (TrickMem)

The TrickMem serves the purpose of a memory model, required for the verification of the SsmcCore.

The following features of the TrickMem are programmable.

- Memory width – can be programmed as 8-bit, 16-bit and 32-bit.
- Memory type – can be programmed as SRAM, ROM and BURST ROM.
- Mode type – can be programmed as Synchronous or Asynchronous Memory.
- Read access time.
- Write access time.
- Memory data bus turn around time.
- Polarity and Enabling of the External asynchronous wait input signal.
- Delayed WEN assertion during writes to Memory devices with the programmable delay.
- Delayed OEN assertion during Reads from Memory devices with the programmable delay.
- Configurable size at reset for boot memory bank using external control pins.

The following sections detail the functionality of the TrickMem.

TrickMem Functional Description

The TrickMem contains the following major modules.
The block diagram of the TrickMem is shown in the ***Figure 7.3-1***

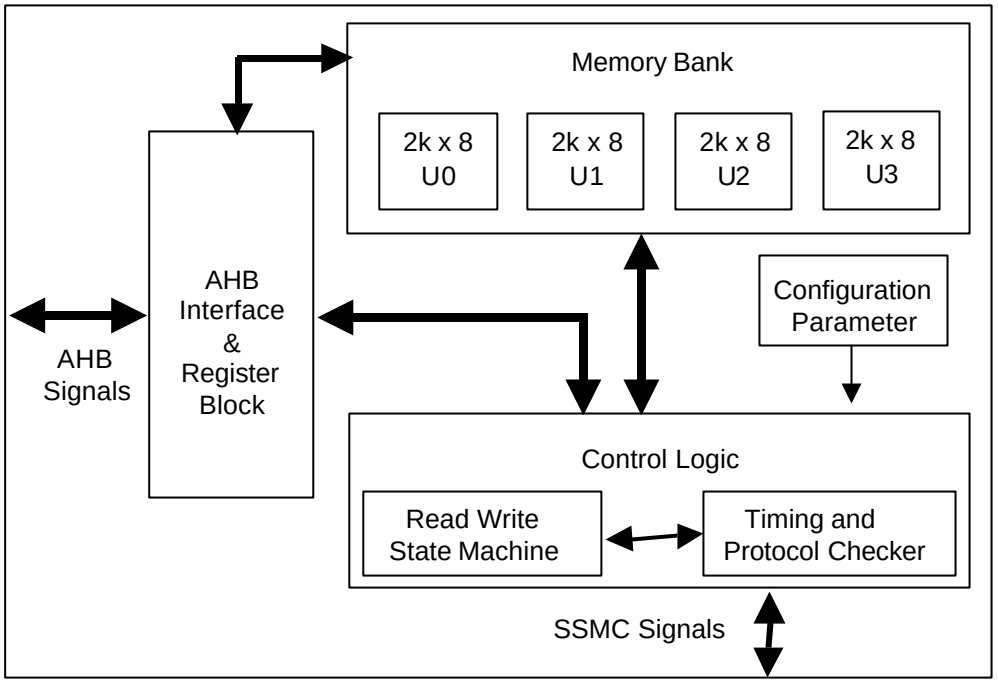


Figure 7.3-1 TrickMem block diagram

AHB Interface and the Register Block

This block interfaces the TrickMem with the AHB. The TrickMem is an AHB slave and it supports all read/write AHB accesses of size 32-bit. The Register Block includes the R/W register local to TrickMem and only write enabled Mirror Registers, which are Mirrored from UUT. These mirror registers are taken to reduce the simulation time by reducing write cycles require to program the UUT and TrickMem in the same cycle.

The AHB interface block performs the following operations.

- Generation of all AHB slave related response.
- Generation of read and write decodes for local registers and write decodes for mirror registers.

TrickMem Register Summary

The SSMC TrickMem registers are shown in ***Table 7.3-9(a)***.

Address	Read Location	Write Location
SSMC TrickMem Base + 0x0000 - 0E000	SSMCTrMEMARRAY	SSMCTrMEMARRAY
SSMC TrickMem Base + 0x10000 - 1E000	SSMCTrMEMBASE	SSMCTrMEMBASE
SSMC TrickMem Base + 0x20000	SSMCTrBurstWT	SSMCTrBurstWT

Table 7.3-9 (a) SSMC Register map

The SSMC TrickMem mirror registers are shown in ***Table 7.3-9 (b)***.

Address	Write Location
UUT Base+ 0x00	SMBIDCYR0
UUT Base + 0x04	SMBWSTRDR0
UUT Base + 0x08	SMBWSTWRR0

UUT Base + 0x0C	SMBWSTOENR0
UUT Base + 0x10	SMBWSTWENR0
UUT Base + 0x14	SMBCR0
UUT Base + 0x1C	SMBWSTBRDR0
UUT Base+ 0x20	SMBIDCYR1
UUT Base + 0x24	SMBWSTRDR1
UUT Base + 0x28	SMBWSTWRR1
UUT Base + 0x2C	SMBWSTOENR1
UUT Base + 0x30	SMBWSTWENR1
UUT Base + 0x34	SMBCR1
UUT Base + 0x3C	SMBWSTBRDR1
UUT Base+ 0x40	SMBIDCYR2
UUT Base + 0x44	SMBWSTRDR2
UUT Base + 0x48	SMBWSTWRR2
UUT Base + 0x4C	SMBWSTOENR2
UUT Base + 0x50	SMBWSTWENR2
UUT Base + 0x54	SMBCR2
UUT Base + 0x5C	SMBWSTBRDR2
UUT Base+ 0x60	SMBIDCYR3
UUT Base + 0x64	SMBWSTRDR3
UUT Base + 0x68	SMBWSTWRR3
UUT Base + 0x6C	SMBWSTOENR3
UUT Base + 0x70	SMBWSTWENR3
UUT Base + 0x74	SMBCR3
UUT Base + 0x7C	SMBWSTBRDR3
UUT Base+ 0x80	SMBIDCYR4
UUT Base + 0x84	SMBWSTRDR4
UUT Base + 0x88	SMBWSTWRR4
UUT Base + 0x8C	SMBWSTOENR4
UUT Base + 0x90	SMBWSTWENR4
UUT Base + 0x94	SMBCR4
UUT Base + 0x9C	SMBWSTBRDR4
UUT Base+ 0xA0	SMBIDCYR5
UUT Base + 0xA4	SMBWSTRDR5
UUT Base + 0xA8	SMBWSTWRR5
UUT Base + 0xAC	SMBWSTOENR5
UUT Base + 0xB0	SMBWSTWENR5
UUT Base + 0xB4	SMBCR5
UUT Base + 0xBC	SMBWSTBRDR5

UUT Base+ 0xC0	SMBIDCYR6
UUT Base + 0xC4	SMBWSTRDR6
UUT Base + 0xC8	SMBWSTWRR6
UUT Base + 0XCC	SMBWSTOENR6
UUT Base + 0XD0	SMBWSTWENR6
UUT Base + 0XD4	SMBCR6
UUT Base + 0xDC	SMBWSTBRDR6
UUT Base+ 0xE0	SMBIDCYR7
UUT Base + 0xE4	SMBWSTRDR7
UUT Base + 0xE8	SMBWSTWRR7
UUT Base + 0XEC	SMBWSTOENR7
UUT Base + 0XF0	SMBWSTWENR7
UUT Base + 0XF4	SMBCR7
UUT Base + 0xFC	SMBWSTBRDR7

Table 7.3-9 (b) SSMC Mirror Register map

The Memory model (TrickMem) is instantiated eight times for eight-memory banks of SSMC. Different base addresses are used for different Memory models.

Memory Block

This block contains a basic memory array of 8-bit width and 2K depth. This memory array is instantiated four times in parallel to create 8 KB of memory of 32-bit width and 2 K depth. Depending on the value programmed in the SMBCR register, it is possible to configure this block as an 8-bit, 16-bit or a 32-bit wide memory. Parameter value MemDeep, in the SSMCTrPackage file determines the depth of the memory block. It is possible to access this memory both from the AHB and from the SSMC.

Control Logic

Control logic contains two parts:

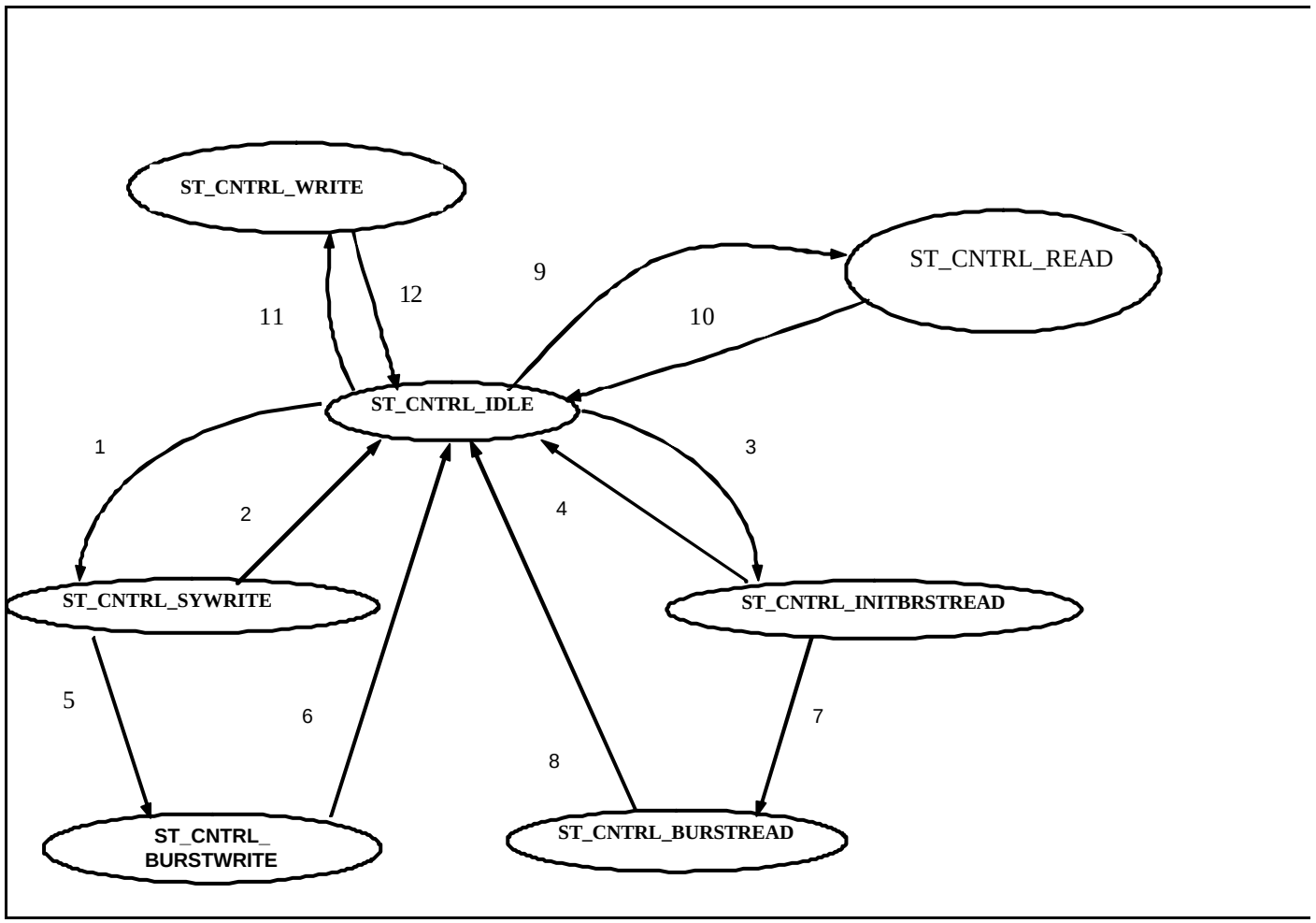
1. State machine

The state machine is shown in ***Figure 7.3-2***. It contains the following states

- **ST_CNTRL_IDLE:** This is the default state after reset. When the nSMWEN and chip select is asserted, the state machine moves to the ST_CNTRL_WRITE state. If the Memory type is SRAM or Flash. Similarly, if nSMOEN and chip select is asserted the state machine moves to the ST_CNTRL_READ state.
- **ST_CNTRL_INITBRSTREAD:** This is the state in which machine goes for the first burst access and if it is continuous burst then it will go into ST_CNTRL_BURSTREAD state other wise goes back into ST_CNTRL_IDLE state. For this, it checks the nSMOEN assertion, higher address bits and memory type.
- **ST_CNTRL_READ:** This is the state the state machine enters during memory read operation for non-burst cases. The state machine can move to the ST_CNTRL_IDLE state. For this, it checks the nSMOEN and nSSMCS assertion and memory type. Else if it is a continuous read the state machine stays in this state it self.
- **ST_CNTRL_BURSTREAD:** This is the state in which the state machine goes after initial burst read during burst memory read case. The state machine can move to the ST_CNTRL_IDLE state after completion of burst read. It checks the chip select de-assertion, if so moves to ST_CNTRL_IDLE state. Otherwise if chip select is asserted and nSMOEN is also asserted then it means subsequent burst reads are expected, so it remains in ST_CNTRL_BURSTREAD.

- **ST_CNTRL_WRITE:** This is the state in which the state machine goes if the memory is configured as asynchronous write enabled. The state machine can move to the ST_CNTRL_IDLE state. For this, it checks the nSMWEN de-assertion and memory type. Else if it is a continuous write the state machine stays in this state it self. If nSMOEN is asserted and nSMWEN de-asserted along with chip select asserted then it goes to ST_CNTRL_IDLE.
- **ST_CNTRL_SYWRITE:** This is the state in which state machine goes if the memory is synchronous write enabled. Machine stays in this state for initial write state only and for next write access it moves into ST_CNTRL_BUSRTWRITE otherwise returned to ST_CNTRL_IDLE.
- **ST_CNTRL_BUSRTWRITE:** This is the state in which the state machine goes after it detect for burst write transfers and stays during memory burst write operation for the subsequent write of burst memory case. The state machine can move to the ST_CNTRL_IDLE state if memory is configured as synchronous write enabled memory. It checks the chip select de-assertion, if so moves to ST_CNTRL_IDLE state. If nSMOEN is asserted and nSMWEN de-asserted along with chip select asserted then it goes to ST_CNTRL_BURSTREAD.

Note: The state machine will be same for Synchronous and Asynchronous Transfers, but for Synchronous transfers we have to consider SSMCLK and ADDRVALID also.



Condition 1: (nCS = 0) && (nSMWEN = 0) && (Synchronous Memory)

Condition 2: (nCS = 1)

Condition 3: (nCS = 0) && (nSMOEN = 0) && (Burst enable memory)

Condition 4: (nCS = 1)

Condition 5: (nCS = 0) && (nSMWEN = 0) && (Synchronous Memory burst enabled)

Condition 6: (nCS = 1)

Condition 7: (nCS = 0) && (nSMOEN = 0) && (Burst enabled Memory)

Condition 8: (nCS = 1)

Condition 9: (nCS = 0) && (nSMWEN = 1) && (nSMOEN = 0) && (Asynchronous memory)

Condition 10: (nCS = 1)

Condition 11: (nCS = 0) && (nSMWEN = 0) && (nSMOEN = 1) && (Asynchronous write enabled Memory)

Condition 12: (nCS = 1)

Figure 7.3-2 TrickMem State Machine

2. Timing and protocol check block, which displays error messages when the following unexpected operations happen

- Access time violation for non-burst read: The access time starts with assertion of chip select and ends with de-assertion of output enable or chip select or address change.
- Access time violation for burst read: The access time starts with change in Address and ends with change in address or de-assertion of chip select.
- Access time violation for write: The access time starts with assertion of chip select and ends with de-assertion of write enable.
- Maximum time limit violation for non-burst read access: This maximum access time is little more than normal access time as mentioned in first protocol.
- Maximum time limit violation for burst read access. This maximum access time is little more than normal access time as mentioned in second protocol.
- Maximum time limit violation for write access: This maximum access time is little more than normal access time as mentioned in third protocol.
- Read and write enable asserted at the same time.
- Address changes when write enable is active.
- Chip select controlled write.
- Chip select changes when read is active.
- Address hold time violation for write accesses with respect to write enable de-assertion.
- Data hold time violation for write accesses with respect to write enable de-assertion.
- Data set-up time violation for write accesses with respect to write enable de-assertion.
- Write to read idle time violation for same and different bank with idle time as one clock period.
- Read to write idle time violation for same and different bank with idle time as (1 + IDCY) * clock period.
- Read to read idle time violation for different bank with idle time as (1 + IDCY) * clock period.
- Write enable to byte select violation.
- Write to ROM or BURST ROM or Write Protected SRAM.
- Chip select assertion to Output Enable assertion time violation. This check is not performed for Wait enabled mode.
- Chip select assertion to Write Enable assertion time violation. This check is not performed for Wait enabled mode.
- Chip select changes when Write is active.
- nSMBLS protocol checks with respect to RBLE.
- nSMWEN/nSMBLS or the nSMOEN assertion checking if all the Chip Selects (SSMCS[7:0]) are de-asserted. Warning message is thrown out if any of the control signals in question are asserted when none of the banks' CS is asserted.
- nSMWEN will be asserted for more than one clock cycle for synchronous write operations.

Configuration Parameters

The following parameter values are set via the configuration file

- Address Set-up / Hold time
- Data Set-up / Hold time
- Depth of Memory

TrickMem Register Description

The base address of the TrickMem is not fixed and may be different based on system implementations. However, the offset of any particular register from the base address is fixed.

SSMCTrMEMARRAY: SSMC TrickMem Memory Array Register

Bit	Name	R/W	Reset value	Function
31 : 0	MEMARRAY	R/W	0x00	Purpose of this register is to provide read write accessibility to the memory. This represents 2k-memory space of 32 bit wide memory.

Table 7.3-10 SSMCTrickMem Memory array register

SSMCTrMEMBASE: SSMC TrickMem Memory addresses base Register

Bit	Name	R/W	Reset value	Function
14 : 0	BASEVALUE	R/W	0x00	The TrickMem contains only 8K of memory. The SSMC can address up to 64MB of memory. Using SSMCTrMEMBASE register value as a base value and the 8K TrickMem internal memory as an offset, it is possible to simulate 64-MB memory requirement for the test.

Table 7.3-11 SSMCTrickMem Memory address base register

SMBIDCYR0-7: SSMC TrickMem Mirror Idle Time Register

Bit	Name	R/W	Reset value	Function
31:4	Reserved	-	0x0	Reserved, Read as Zero.
3 : 0	IDCY	W	0xF	Memory data bus turn around time. The turn around time is IDCY x T_{HCLK}

Table 7.3-12 SSMCTrickMem Mirror Idle Time register

SMBWSTRDR0-7: SMBWSTRDR TrickMem Mirror Read Wait State Control Register

Bit	Name	R/W	Reset value	Function
31:5	Reserved	-	0x0	Reserved, Read as Zero.
4 :0	WSTRD	W	0x1F	In the case of SRAM and ROM, this field indicates read access time. In the case of Burst ROM, this field indicates the initial access time. The wait state time is WSTRD) x T_{HCLK}

Table 7.3-13 SSMCTrickMem Mirror Read Wait state Control register

SMBWSTWRR0-7: SMBWSTWRR TrickMem Mirror Write Wait State Control Register

Bit	Name	R/W	Reset value	Function
31:5	Reserved	-	0x0	Reserved, Read as Zero.

4 : 0	WSTWR	W	0x1F	In the case of SRAM and ROM, this field indicates Write access time. In the case of RAM, this field indicates the number of wait states for write Accesses, and external wait assertion timing for writes. The wait state time is WSTWR x T_{HCLK}

Table 7.3-14 SSMCTrickMem* Mirror *Write Wait state Control register

SMBWSTOENR0-7: SMBWSTOENR TrickMem Mirror output enable delay Control Register

Bit	Name	R/W	Reset value	Function
31:4	Reserved	-	0x0	Reserved, Read as Zero.
3 : 0	WSTOEN	W	0x00	Output enable assertion delay from chip select assertion.

Table 7.3-15 TrickMem Mirror output enable delay Control Register

SMBWSTWENR0-7: SMBWSTWENR TrickMem Mirror Write enable delay Control Register

Bit	Name	R/W	Reset value	Function
31:4	Reserved	-	0x0	Reserved, Read as Zero.
3 : 0	WSTWEN	W	0x01	Write enable assertion delay from chip select assertion.

Table 7.3-16 TrickMem Write enable delay Control Register

SMBWSTBRDR0-7: SMBWSTBRDR TrickMem Burst read delay Control Register

Bit	Name	R/W	Reset value	Function
31:5	Reserved	-	0x0	Reserved, Read as Zero.
4 – 0	WSTBRD	W	0x1F	Burst read wait state after the first read.

Table 7.3-17 TrickMem Mirror Burst read delay Control Register

SMBCR0-7: SMBCR TrickMem Mirror Bank control Register

Bits	Name	R/W	Reset value	Function
31 : 21	Reserved	W	0x0	Reserved, write as zero.
20	AddrValidWriteEn	W	0x0	Controls the behaviour of the signal SMADDRVALID during write operations.0-signal always high. 1-signal active for asynchronous and synchronous writes accesses.
19 :18	BurstLenWrite	W	0x0	Burst transfer length, used to set the number of burst transfers. 00-4 transfer burst (default). 01-8 transfer burst 10- reserved 11-continous burst.

17	SyncEnWrite	W	0x0	Synchronous burst mode write. 0-asynchronouse burst mode 1-synchronous burst mode.
15 : 13	Reserved	-	0x0	-
12	AddrValidReadEn	W	0x0	Controls the behaviour of the signal SMADDRVALID during read operations. 0-signal always high. 1-signal active for asynchronous and synchronous read accesses.
11 :10	BurstLenRead	W	0x0	Burst transfer length, used to set the number of burst transfers. 00-4 transfer burst (default). 01-8 transfer burst 10- reserved 11-continous burst.
9	SyncEnRead	W	0x0	Synchronous burst mode Read. 0-asynchronouse burst mode 1-synchronous burst mode.
8	BMRead	W	0x0	Burst mode read 0-non burst mode read 1-burst mode read.
7 : 6	Reserved	-	0x0	-
5 : 4	MW	W	-	Memory width. 00-8 bit 01-16 bit 10-32 11-reserved
3	WP	W	0x0	Write protect, 0-for SRAM or write enable Flash. 1-for ROM or burst ROM.
2	WaitEn	W	0x0	External Memory controller wait signal enable.
1	WaitPol	W	0x0	Polarity of external wait signal.
0	RBLE	W	0x0	Read byte lane enable.

Table 7.3-18 SMBCR TrickMem Mirror Bank control Register

SSMCTrBurstWT: SSMC Trickbox BurstWT delay Register

Bit	Name	R/W	Reset value	Function
8	BwtMask	R/W	0x00	This bit masks the nSMBURSTWAIT signal to go out ‘0’ nSMBURSTWAIT is not masked. ‘1’ nSMBURSTWAIT is masked.
7 : 4	BeatNo	R/W	0x00	This indicates for which beat of the burst nSMBURSTWAIT should be asserted.

3 : 0	BurstWT	R/W	0x00	This is delay count for which nSMBURSTWAIT will be asserted.

Table 7.3-19 SSMC Trickbox BurstWT delay Register

8 VERIFICATION TESTS

The test vectors that are applied to the verification (BusTalk) testbenches in the SSMC (PL093) are in the ***verification/bustest*** directory.

8.1 Functional Tests

8.1.1 Address Advance Test [AddrAdvanceTest.c]

In this test, the BIWriteEn and the BIReadEn bits in the SMBCRx register are set for the respective banks. Burst Read accesses are performed on the banks such that the Burst Address Counter terminal count is reached and a cross over occurs. A new burst Write/Read access is repeated.

Program the SSMCTrCR, SSMCTrBurstWT, SSMCTrWTCNCL, SSMCTrEndian and SSMCTrExtMuxWT registers to the default values.

Configure Bank 2 and Bank 5 as 8 bit Synchronous SRAMS with the parameters given in the ***Table 8.1-1*** below.

Parameters	Bank 2	Bank 5
WSTIDCY	1	1
WSTRD	2	2
WSTWR	3	3
WSTOEN	1	1
WSTWEN	1	1
WSTBRD	2	2
SMBCRData Fields		
BIWriteEn	0	0
AddrValidWriteEn	1	1
BurstLenWrite	4	4
SyncWriteDev	1	1
BMWrite	1	1
BIReadEn	0	0
AddrValidReadEn	1	1
BurstLenRead	4	4

SyncReadDev	1	1
BMRead	1	1
MW	8	8
WP	0	0
WaitEn	0	0
WaitPol	1	1
RBLE	0	0
SSMCTrCS2WTRData Fields		
TRWAITEN	0	0
TRWAITPOL	1	1
TRCS2WTR	2	2
TRWT2DEWT	2	2

Table 8.1-1 Configuration Data for Different Banks

Initialise the bank 2 and bank 5 with data for 64 locations from the AHB side.

Enable the BIWriteEn and BIReadEn bits in the SMBCR2 and SMBCR5 registers.

Perform burst read accesses with HSIZE = BYTE and HBURST = INCR8 on bank 2.

Perform burst read accesses with HSIZE = BYTE and HBURST = INCR8 on bank 5.

Repeat a new burst on bank 2 with HSIZE = BYTE and HBURST = INCR8.

Disable the BIWriteEn and BIReadEn bits in the SMBCR2 and SMBCR5 registers at the end of the test.

8.1.2 Address toggle test [AddrToggleTests.c]

This test ensures that the addressing of the memory map is done properly. The TrickMem is designed with a memory size of 8K. The remaining higher order address is specified in the base address register SSMCTrMEMB. Therefore, it is possible to access all 64MB addresses of the SSMC by changing the value of SSMCTrMEMB register. This test toggles all memory-related address bits in the Ssmc.

Program the SSMCTrCR, SSMCTrBurstWT, SSMCTrWTCNCL, SSMCTrEndian and SSMCTrExtMuxWT registers to the default values.

Configure Bank 0, Bank 2, Bank 5 and Bank 7 as 32 bit Asynchronous SRAMS with external wait disabled. Program the other parameters of the banks with the data given in the ***Table 8.1-2*** below.

Parameters	Bank 0	Bank 2	Bank 5	Bank 7
WSTIDCY	1	1	1	1
WSTRD	2	2	2	2
WSTWR	3	3	3	3
WSTOEN	1	1	1	1

WSTWEN	1	1	1	1
WSTBRD	2	2	2	2
SMBCRData Fields				
BlWriteEn	0	0	0	0
AddrValidWriteEn	0	0	0	0
BurstLenWrite	4	4	4	4
SyncWriteDev	0	0	0	0
BMWrite	1	1	1	1
BlReadEn	0	0	0	0
AddrValidReadEn	0	0	0	0
BurstLenRead	4	4	4	4
SyncReadDev	0	0	0	0
BMRead	1	1	1	1
MW	32	32	32	32
WP	0	0	0	0
WaitEn	1	0	1	0-
WaitPol	0	0	0	0
RBLE	1	1	1	1
SSMCTrCS2WTRData Fields				
TRWAITEN	1	0	1	0
TRWAITPOL	0	0	0	0
TRCS2WTR	2	2	2	2
TRWT2DEWT	2	2	2	2

Table 8.1-2 Configuration Data for Different Banks

Write/Read accesses are performed on the memory banks as shown in the ***Table 8.1-3 below.***

HSIZE	MSIZE	HBURST	BANK NO	DATA
BYTE	WORD	INCR	0	0x00000000
BYTE	WORD	INCR	2	0xAAAAAAAA
BYTE	WORD	INCR	5	0x55555555
BYTE	WORD	INCR	7	0xFFFFFFFF

Table 8.1-3 Accesses Types Performed

The data written in the Banks 0, Banks 2, Banks 5 and Banks 7, and is read back from AHB side to verify data integrity.

8.1.3 Synchronous Address toggle test [SyAddrToggleTests.c]

This test ensures that the addressing of the memory map is done properly. The TrickMem is designed with a memory size of 8K. The remaining higher order address is specified in the base address register SSMCTrMEMB. Therefore, it is possible to access all 64MB addresses of the SSMC by changing the value of SSMCTrMEMB register. This test toggles all memory-related address bits in the SsmcCore.

Program the SSMCTrCR, SSMCTrBurstWT, SSMCTrWTCNCL, SSMCTrEndian and SSMCTrExtMuxWT registers to the default values.

Configure Bank 1, Bank 3, Bank 4 and Bank 6 as 32 bit Synchronous SRAMS with the parameters as given in the ***Table 8.1-4*** below.

Parameters	Bank 1	Bank 3	Bank 4	Bank 6
WSTIDCY	1	1	1	1
WSTRD	2	2	2	2
WSTWR	3	3	3	3
WSTOEN	1	1	1	1
WSTWEN	1	1	1	1
WSTBRD	2	2	2	2
SMBCRData Fields				
BlWriteEn	0	0	0	0
AddrValidWriteEn	1	1	1	1
BurstLenWrite	4	4	4	4
SyncWriteDev	1	1	1	1
BMWrite	1	1	1	1
BlReadEn	0	0	0	0
AddrValidReadEn	1	1	1	1
BurstLenRead	4	4	4	4
SyncReadDev	1	1	1	1
BMRead	1	1	1	1
MW	32	32	32	32
WP	0	0	0	0
WaitEn	0	0	0	0
WaitPol	0	0	0	0
RBLE	1	1	1	1

SSMCTrCS2WTRData Fields				
TRWAITEN	0	0	0	0
TRWAITPOL	0	0	0	0
TRCS2WTR	2	2	2	2
TRWT2DEWT	2	2	2	2

Table 8.1-4 Configuration Data for Different Banks

Write/Read accesses are performed on the memory banks as shown in the ***Table 8.1-5 below.***

HSIZE	MSIZE	HBURST	BANK NO	DATA
BYTE	WORD	INCR	1	0x00000000
BYTE	WORD	INCR	3	0xAAAAAAAA
BYTE	WORD	INCR	4	0x55555555
BYTE	WORD	INCR	6	0xFFFFFFFF

Table 8.1-5 Accesses Types Performed

The data written in the Banks 0, Banks 2, Banks 5 and Banks 7 is read from AHB side to verify data integrity.

8.1.4 Address Valid Tests [AddrValidTests.c]

This test checks the behaviour of the SMADDRVALID signal whenever an asynchronous write or an asynchronous read access is done to a Synchronous memory.

Program the SSMCTrCR, SSMCTrBurstWT, SSMCTrWTCNCL, SSMCTrEndian and SSMCTrExtMuxWT registers to the default values.

Configure Bank 2 and Bank 5 as 8 bit Synchronous SRAMS with the parameters as given in the ***Table 8.1-6*** below.

Parameters	Bank 2	Bank 5
WSTIDCY	1	1
WSTRD	2	2
WSTWR	3	3
WSTOEN	1	1
WSTWEN	1	1
WSTBRD	2	2
SMBCRData Fields		
BlWriteEn	0	0
AddrValidWriteEn	1	1
BurstLenWrite	4	4

SyncWriteDev	1	0
BMWrite	1	1
BIReadEn	0	0
AddrValidReadEn	1	1
BurstLenRead	4	4
SyncReadDev	0	1
BMRead	1	1
MW	8	8
WP	0	0
WaitEn	0	0
WaitPol	1	1
RBLE	0	0
SSMCTrCS2WTRData Fields		
TRWAITEN	0	0
TRWAITPOL	1	1
TRCS2WTR	2	2
TRWT2DEWT	2	2

Table 8.1-6 Configuration Data for Different Banks

Burst Write and Read accesses are done to the memory bank 2 and memory bank 5 with HSIZE = Byte and HBURST = INCR8. The following access sequence is tested.

- Synchronous Write followed by Asynchronous Read to Synchronous memory bank 2.
- Asynchronous write access followed by Synchronous Read access to synchronous memory bank 5.
- All accesses are done back-to-back.

8.1.5 Asynchronous Bank Crossing test [AsyBankCrossingTests.c]

In this test all the banks are configured as asynchronous memory banks with different parameters. The memory widths of the banks are different as we move from bank to bank and remain the same as when it was configured at the start of the test. The values of SMBWSTRDRx, SMBWSTWRRx, SMBWSTBRDRx, CS2OEN, CS2WEN and IDCY parameters are configured the same for all the banks. The starting address for each memory transfer is configured differently for all the banks.

Burst accesses of various combinations of AHB widths and HBURST lengths are attempted on all the memory banks. The bursts are started at different word boundaries and quadword boundaries with INCR4, INCR8, INCR16, Wrap4, Wrap8 and Wrap16 for HSIZESMC of Byte, HalfWord and Word.

Program the SSMCTrCR, SSMCTrBurstWT, SSMCTrWTCNCL, SSMCTrEndian and SSMCTrExtMuxWT registers to the default values.

The different banks are configured as per the data given in the ***Table 8.1-7*** below.

Parameters	Bank 0	Bank 1	Bank 2	Bank 3	Bank 4	Bank 5	Bank 6	Bank 7
WSTIDCY	1	1	1	1	1	1	1	1
WSTRD	2	2	2	2	2	2	2	2
WSTWR	3	3	3	3	3	3	3	3
WSTOEN	1	1	1	1	1	1	1	1
WSTWEN	1	1	1	1	1	1	1	1
WSTBRD	2	2	2	2	2	2	2	2
SMBCRData Fields								
BlWriteEn	0	0	0	0	0	0	0	0
AddrValidWriteEn	0	0	0	0	0	0	0	0
BurstLenWrite	4	4	4	4	4	4	4	4
SyncWriteDev	0	0	0	0	0	0	0	0
BMWrite	1	1	1	1	1	1	1	1
BlReadEn	0	0	0	0	0	0	0	0
AddrValidReadEn	0	0	0	0	0	0	0	0
BurstLenRead	4	4	4	4	4	4	4	4
SyncReadDev	0	0	0	0	0	0	0	0
BMRead	1	1	1	1	1	1	1	1
MW	32	16	8	32	16	8	32	16
WP	0	0	0	0	0	0	0	0
WaitEn	0	1	0	1	0	1	0	1
WaitPol	0	0	1	1	1	1	0	0
RBLE	1	1	0	1	1	0	1	1
SSMCTrCS2WTRData Fields								
TRWAITEN	0	1	0	1	0	1	0	1
TRWAITPOL	0	0	1	1	1	1	0	0
TRCS2WTR	2	2	2	2	2	2	2	2
TRWT2DEWT	2	2	2	2	2	2	2	2

Table 8.1-7 Configuration Data for Different Banks

The different types of Write/Read accesses, which are done on all the banks with different combinations of HSIZE, HBURSRT and MSIZE, are shown in the ***Table 8.1-8*** below.

HSIZE	HBURST	BANKS
BYTE	SINGLE, INCR, INCR4, INCR8, WRAP4, WRAP8, WRAP16	On all Banks 0 to 7
HALFWORD	SINGLE, INCR, INCR4, INCR8, WRAP4, WRAP8, WRAP16	On all Banks 0 to 7
WORD	SINGLE, INCR, INCR4, INCR8, WRAP4, WRAP8, WRAP16	On all Banks 0 to 7

Table 8.1-8 Accesses Types Performed

8.1.6 Mixed Bank Crossing Test [BankCrossingTests.c]

In this test, Bank0, Bank2, Bank4 and Bank6 are configured as asynchronous banks. Bank1, Bank3, Bank5 and Bank7 are configured as synchronous banks. The memory widths of the banks are different as we move from bank to bank and remain the same as when it was configured at the start of the test. The values of SMBWSTRDRx, SMBWSTWRRx, SMBWSTBRDRx, CS2OEN, CS2WEN and IDCY parameters are configured the same for all the banks. The starting address for each memory transfer is also different from bank to bank.

Burst accesses of various combinations of AHB widths and HBURST lengths are attempted on all the memory banks. The bursts are started at different word boundaries and quadword boundaries with INCR4, INCR8, INCR16, Wrap4, Wrap8 and Wrap16 for HSIZESMC of Byte, HalfWord and Word.

Program the SSMCTrCR, SSMCTrBurstWT, SSMCTrWTCNCL, SSMCTrEndian and SSMCTrExtMuxWT registers to the default values.

The different banks are configured as per the data given in the ***Table 8.1-9*** below.

Parameters	Bank 0	Bank 1	Bank 2	Bank 3	Bank 4	Bank 5	Bank 6	Bank 7
WSTIDCY	1	1	1	1	1	1	1	1
WSTRD	2	2	2	2	2	2	2	2
WSTWR	3	3	3	0	3	3	3	3
WSTOEN	1	1	1	1	1	1	1	1
WSTWEN	1	1	1	0	1	1	1	1
WSTBRD	2	2	2	2	2	2	2	2
SMBCRData Fields								
BIWriteEn	0	0	0	0	0	0	0	0
AddrValidWriteEn	1	0	1	0	1	0	1	0
BurstLenWrite	4	4	4	4	4	4	4	4
SyncWriteDev	1	0	1	0	1	0	1	0
BMWrite	1	1	1	1	1	1	1	1
BIReadEn	0	0	0	0	0	0	0	0
AddrValidReadEn	1	0	1	0	1	0	1	0

BurstLenRead	4	4	4	4	4	4	4	4
SyncReadDev	1	0	1	0	1	0	1	0
BMRead	1	1	1	1	1	1	1	1
MW	32	16	8	32	16	8	32	16
WP	0	0	0	0	0	0	0	0
WaitEn	0	0	0	0	0	1	0	0
WaitPol	0	0	1	1	1	1	0	0
RBLE	1	1	0	1	1	0	1	1
SSMCTrCS2WTRData Fields								
TRWAITEN	0	0	0	0	0	1	0	0
TRWAITPOL	0	0	1	1	1	1	0	0
TRCS2WTR	2	2	2	2	2	2	2	2
TRWT2DEWT	2	2	2	2	2	2	2	2

Table 8.1-9 Configuration Data for Different Banks

Different types of accesses are done on all the banks with different combinations of HSIZE, HBURSRT and MSIZE, as shown in the ***Table 8.1-10*** below. The Write/Read access are done starting from Bank 0 and ending with Bank 7.

HSIZE	HBURST	BANKS
BYTE	SINGLE, INCR, INCR4, INCR8, WRAP4, WRAP8, WRAP16	On all Banks 0 to 7
HALFWORD	SINGLE, INCR, INCR4, INCR8, WRAP4, WRAP8, WRAP16	On all Banks 0 to 7
WORD	SINGLE, INCR, INCR4, INCR8, WRAP4, WRAP8, WRAP16	On all Banks 0 to 7

Table 8.1-10 Accesses Types Performed

The different banks are again reconfigured as per the data given in the ***Table 8.1-11*** below.

Parameters	Bank 0	Bank 1	Bank 2	Bank 3	Bank 4	Bank 5	Bank 6	Bank 7
WSTIDCY	1	1	1	1	1	1	1	1
WSTRD	2	2	2	2	2	2	2	2
WSTWR	3	3	3	0	3	3	3	3
WSTOEN	1	1	1	1	1	1	1	1
WSTWEN	1	1	1	0	1	1	1	1
WSTBRD	2	2	2	2	2	2	2	2

SMBCRData Fields								
BlWriteEn	0	0	0	0	0	0	0	0
AddrValidWriteEn	0	1	0	1	0	1	0	1
BurstLenWrite	4	4	4	4	4	4	4	4
SyncWriteDev	0	1	0	1	0	1	0	1
BMWrite	1	1	1	1	1	1	1	1
BlReadEn	0	0	0	0	0	0	0	0
AddrValidReadEn	0	1	0	1	0	1	0	1
BurstLenRead	4	4	4	4	4	4	4	4
SyncReadDev	0	1	0	1	0	1	0	1
BMRead	1	1	1	1	1	1	1	1
MW	8	32	16	8	32	16	8	32
WP	0	0	0	0	0	0	0	0
WaitEn	0	0	0	0	0	0	0	0
WaitPol	0	0	1	1	1	1	0	0
RBLE	0	1	1	0	1	1	0	1
SSMCTrCS2WTRData Fields								
TRWAITEN	0	0	0	0	0	0	0	0
TRWAITPOL	0	0	1	1	1	1	0	0
TRCS2WTR	2	2	2	2	2	2	2	2
TRWT2DEWT	2	2	2	2	2	2	2	2

Table 8.1-11 Configuration Data for Different Banks

Different types of accesses are done on all the banks with different combinations of HSIZE, HBURSRT and MSIZE, as shown in the ***Table 8.1-12*** below. The Write/Read accesses are done in the sequence given below.

Write accesses are started from Bank 0 and ended with Bank 7.

This is followed by Read accesses starting with Bank 7 and ending with Bank 0.

The Write accesses are again started with Bank 7 and ended with Bank 0.

The data written is then read back starting with Bank 0 and ending with Bank 7.

This ensures that Write/Read accesses crossing across different banks with different HSIZE, MSIZE and HBURST values are covered.

HSIZE	HBURST	BANKS
BYTE	SINGLE, INCR, INCR4, INCR8, WRAP4, WRAP8, WRAP16	On all Banks 0 to 7

HALFWORD	SINGLE, INCR, INCR4, INCR8, WRAP4, WRAP8, WRAP16	On all Banks 0 to 7
WORD	SINGLE, INCR, INCR4, INCR8, WRAP4, WRAP8, WRAP16	On all Banks 0 to 7

Table 8.1-12 Accesses Types Performed

8.1.7 Synchronous Bank Crossing test [SyBankCrossingTests.c]

In this test all the banks are configured as synchronous memory banks with different parameters. The memory widths of the banks are different as we move from bank to bank and remain the same as when it was configured at the start of the test. The values of SMBWSTRDRx, SMBWSTWRRx, SMBWSTBRDRx, CS2OEN, CS2WEN and IDCY parameters are configured the same for all the banks. The starting address for each memory transfer is also different from bank to bank.

Burst accesses of various combinations of AHB widths and HBURST lengths are attempted on all the memory banks. The bursts are started at different word boundaries and quadword boundaries with INCR4, INCR8, INCR16, Wrap4, Wrap8 and Wrap16 for HSIZESMC of Byte, HalfWord and Word.

Program the SSMCTrCR, SSMCTrBurstWT, SSMCTrWTCNCL, SSMCTrEndian and SSMCTrExtMuxWT registers to the default values.

The different banks are configured as per the data given in the ***Table 8.1-13*** below.

Parameters	**Bank 0**	**Bank 1**	**Bank 2**	**Bank 3**	**Bank 4**	**Bank 5**	**Bank 6**	**Bank 7**
WSTIDCY	1	1	1	1	1	1	1	1
WSTRD	2	2	2	2	2	2	2	2
WSTWR	3	3	3	3	3	3	3	3
WSTOEN	1	1	1	1	1	1	1	1
WSTWEN	1	1	1	0	1	1	1	1
WSTBRD	2	2	2	2	2	2	2	2
SMBCRData Fields								
BlWriteEn	0	0	0	0	0	0	0	0
AddrValidWriteEn	1	1	1	1	1	1	1	1
BurstLenWrite	4	4	4	4	4	4	4	4
SyncWriteDev	1	1	1	1	1	1	1	1
BMWrite	1	1	1	1	1	1	1	1
BlReadEn	0	0	0	0	0	0	0	0
AddrValidReadEn	1	1	1	1	1	1	1	1
BurstLenRead	4	4	4	4	4	4	4	4
SyncReadDev	1	1	1	1	1	1	1	1

BMRead	1	1	1	1	1	1	1	1
MW	32	16	8	32	16	8	32	16
WP	0	0	0	0	0	0	0	0
WaitEn	0	0	0	0	0	0	0	0
WaitPol	0	0	1	1	1	1	0	0
RBLE	1	1	0	1	1	0	1	1
SSMCTrCS2WTRData Fields								
TRWAITEN	0	0	0	0	0	1	0	0
TRWAITPOL	0	0	1	1	1	1	0	0
TRCS2WTR	2	2	2	2	2	2	2	2
TRWT2DEWT	2	2	2	2	2	2	2	2

Table 8.1-13 Configuration Data for Different Banks

Different types of accesses are done on all the banks with different combinations of HSIZE, HBURSRT and MSIZE, as shown in the ***Table 8.1-14*** below. The Write/Read access are done starting from Bank 0 and ending with Bank 7.

HSIZE	HBURST	BANKS
BYTE	SINGLE, INCR, INCR4, INCR8, WRAP4, WRAP8, WRAP16	On all Banks 0 to 7
HALFWORD	SINGLE, INCR, INCR4, INCR8, WRAP4, WRAP8, WRAP16	On all Banks 0 to 7
WORD	SINGLE, INCR, INCR4, INCR8, WRAP4, WRAP8, WRAP16	On all Banks 0 to 7

Table 8.1-14 Accesses Types Performed

8.1.8 Asynchronous Write Read Tests [AsyWRRDTests.c]

All the banks in this test are configured as asynchronous banks. This is a generic full-fledged test, which does read-write operation to the different banks with all possible combinations of the following:

- HSIZE values of BYTE, HWRD and WRD
- Memory size of 8 bits, 16 bits and 32 bits
- Different HBURSTSMC values : SINGLE, INCR, WRAP4, INCR4, WRAP8, INCR8, WRAP16, INCR16
- Bank values of 0 to 7

Program the SSMCTrCR, SSMCTrBurstWT, SSMCTrWTCNCL, SSMCTrEndian and SSMCTrExtMuxWT registers to the default values.

The different banks are configured as per the data given in the ***Table 8.1-15*** below.

Parameters	Bank 0	Bank 1	Bank 2	Bank 3	Bank 4	Bank 5	Bank 6	Bank 7

SMBCRData Fields								
BIWriteEn	0	0	0	0	0	0	0	0
AddrValidWriteEn	0	0	0	0	0	0	0	0
BurstLenWrite	4	4	4	4	4	4	4	4
SyncWriteDev	0	0	0	0	0	0	0	0
BMWrite	1	1	1	1	1	1	1	1
BIReadEn	0	0	0	0	0	0	0	0
AddrValidReadEn	0	0	0	0	0	0	0	0
BurstLenRead	4	4	4	4	4	4	4	4
SyncReadDev	0	0	0	0	0	0	0	0
BMRead	1	1	1	1	1	1	1	1
WP	0	0	0	0	0	0	0	0
WaitPol	0	0	1	1	1	1	0	0
RBLE	1	1	0	1	1	0	1	1
SSMCTrCS2WTRData Fields								
TRWAITPOL	0	0	1	1	1	1	0	0
TRCS2WTR	2	2	2	2	2	2	2	2
TRWT2DEWT	2	2	2	2	2	2	2	2

Table 8.1-15 Configuration Data for Different Banks

Write/Read accesses are performed by configuring the UUT and the memory banks with the different values for the parameters stated in the ***Table 8.1-16*** below. The Write/Read tests are repeated with different values for AHB widths and different banks.

AHB WIDTH	MEM WIDTH	WST IDCY	WST READ	WST WRITE	WST OEN	WST WEN	WST BRD	BURSTLEN READ	BURSTLEN WRITE	HBURST	RBLE
8	8	0	0	0	0	0	0	4	4	SIN	0
8	8	2	4	4	2	2	4	8	8	INC	0
8	8	4	8	8	4	4	8	4	4	INC4	0
8	8	6	12	12	6	6	12	8	8	INC8	0
8	8	8	16	16	8	8	16	4	4	INC16	0
8	8	10	20	20	10	10	20	8	8	WRP4	0
8	8	12	24	24	12	12	24	4	4	WRP8	0
8	8	14	28	28	14	14	28	8	8	WRP16	0

8	16	1	2	2	1	1	2	4	4	SIN	1
8	16	3	6	6	3	3	6	8	8	INC	1
8	16	5	10	10	5	5	10	4	4	INC4	1
8	16	7	14	14	7	7	14	8	8	INC8	1
8	16	9	18	18	9	9	18	4	4	INC16	1
8	16	11	22	22	11	11	22	8	8	WRP4	1
8	16	13	26	26	13	13	26	4	4	WRP8	1
8	16	15	30	30	15	15	30	8	8	WRP16	1
8	32	1	1	1	1	1	1	4	4	SIN	1
8	32	3	5	5	3	3	5	8	8	INC	1
8	32	5	9	9	5	5	9	4	4	INC4	1
8	32	7	13	13	7	7	13	8	8	INC8	1
8	32	0	17	17	0	0	17	4	4	INC16	1
8	32	2	21	21	2	2	21	8	8	IWRP4	1
8	32	4	25	25	4	4	25	4	4	WRP8	1
8	32	6	29	29	6	6	29	8	8	WRP16	1

Table 8.1-16 Accesses Types Performed

8.1.9 Synchronous Write Read Tests [SyWRRDTests.c]

All the banks in this test are configured as synchronous banks. This is a generic full-fledged test, which does read-write operation to the different banks with all possible combinations of the following:

- HSIZE values of BYTE, HWRD and WRD
- Memory size of 8 bits, 16 bits and 32 bits
- Different HBURSTSMC values : SINGLE, INCR, WRAP4, INCR4, WRAP8, INCR8, WRAP16, INCR16
- Bank values of 0 to 7

Program the SSMCTrCR, SSMCTrBurstWT, SSMCTrWTCNCL, SSMCTrEndian and SSMCTrExtMuxWT registers to the default values.

The different banks are configured as per the data given in the ***Table 8.1-17*** below.

Parameters	**Bank 0**	**Bank 1**	**Bank 2**	**Bank 3**	**Bank 4**	**Bank 5**	**Bank 6**	**Bank 7**
SMBCRData Fields								
BlWriteEn	0	0	0	0	0	0	0	0
AddrValidWriteEn	1	1	1	1	1	1	1	1

BurstLenWrite	4	4	4	4	4	4	4	4
SyncWriteDev	1	1	1	1	1	1	1	1
BMWrite	1	1	1	1	1	1	1	1
BIReadEn	0	0	0	0	0	0	0	0
AddrValidReadEn	1	1	1	1	1	1	1	1
BurstLenRead	4	4	4	4	4	4	4	4
SyncReadDev	1	1	1	1	1	1	1	1
BMRead	1	1	1	1	1	1	1	1
WP	0	0	0	0	0	0	0	0
WaitPol	0	0	1	1	1	1	0	0
SSMCTrCS2WTRData Fields								
TRWAITPOL	0	0	1	1	1	1	0	0
TRCS2WTR	2	2	2	2	2	2	2	2
TRWT2DEWT	2	2	2	2	2	2	2	2

Table 8.1-17 Configuration Data for Different Banks

Write/Read accesses are performed by configuring the UUT and the memory banks with the different values for the parameters stated in the ***Table 8.1-18*** below. The Write/Read tests are repeated with different values for AHB widths and different banks.

AHB WIDTH	MEM WIDTH	WST IDCY	WST READ	WST WRITE	WST OEN	WST WEN	WST BRD	BURSTLEN READ	BURSTLEN WRITE	HBURST	RBLE
8	8	0	0	0	0	0	0	4	4	SIN	0
8	8	2	4	4	2	2	4	8	8	INC	0
8	8	4	8	8	4	4	8	4	4	INC4	0
8	8	6	12	12	6	6	12	8	8	INC8	0
8	8	8	16	16	8	8	16	4	4	INC16	0
8	8	10	20	20	10	10	20	8	8	WRP4	0
8	8	12	24	24	12	12	24	4	4	WRP8	0
8	8	14	28	28	14	14	28	8	8	WRP16	0
8	16	1	2	2	1	1	2	4	4	SIN	1
8	16	3	6	6	3	3	6	8	8	INC	1
8	16	5	10	10	5	5	10	4	4	INC4	1
8	16	7	14	14	7	7	14	8	8	INC8	1

8	16	9	18	18	9	9	18	4	4	INC16	1
8	16	11	22	22	11	11	22	8	8	WRP4	1
8	16	13	26	26	13	13	26	4	4	WRP8	1
8	16	15	30	30	15	15	30	8	8	WRP16	1
8	32	1	1	1	1	1	1	4	4	SIN	1
8	32	3	5	5	3	3	5	8	8	INC	1
8	32	5	9	9	5	5	9	4	4	INC4	1
8	32	7	13	13	7	7	13	8	8	INC8	1
8	32	0	17	17	0	0	17	4	4	INC16	1
8	32	2	21	21	2	2	21	8	8	IWRP4	1
8	32	4	25	25	4	4	25	4	4	WRP8	1
8	32	6	29	29	6	6	29	8	8	WRP16	1

Table 8.1-18 Accesses Types Performed

8.1.10 Burst ROM Test [BROMTests.c]

This test checks the operation of the SsmcCore with different types of BURST ROM's connected to different banks. The banks are configured as asynchronous memory banks.

The test sequence is as follows. Program bank 1, 3,5, 7 and the TrickMem connected to these banks as BURST ROMs with different SMBWSTRDRx, SMBWSTWRRx, SMBWSRBRDRx and word size values. Initialise the BURST ROM contents from the AHB side in Configuration register and the TrickMem. Perform burst and non-burst reads from memory banks through the SSMC using different HSIZESMCSMC and different HBURSTSMC values.

Program the SSMCTrCR, SSMCTrBurstWT, SSMCTrWTCNCL, SSMCTrEndian and SSMCTrExtMuxWT registers to the default values.

The different banks are configured as per the data given in the ***Table 8.1-19*** below.

Parameters	**Bank 0**	**Bank 1**	**Bank 2**	**Bank 3**	**Bank 5**	**Bank 7**
WSTIDCY	3	3	3	3	4	2
WSTRD	0	0	0	7	12	5
WSTWR	0	0	0	5	6	2
WSTOEN	0	0	0	3	4	0
WSTWEN	0	0	0	0	0	0
WSTBRD	1	2	1	4	6	8
SMBCRData Fields						
BIWriteEn	0	0	0	0	0	0
AddrValidWriteEn	0	0	0	0	0	0

BurstLenWrite	4	4	4	4	4	4
SyncWriteDev	0	0	0	0	0	0
BMWrite	0	0	0	0	0	0
BlReadEn	0	0	0	0	0	0
AddrValidReadEn	0	0	0	0	0	0
BurstLenRead	4	4	4	4	4	4
SyncReadDev	0	0	0	0	0	0
BMRead	1	1	1	1	1	1
MW	16	32	8	16	8	32
WP	1	1	1	1	1	1
WaitEn	1	1	1	0	1	0
WaitPol	0	0	0	1	1	0
RBLE	1	1	0	1	0	1
SSMCTrCS2WTRData Fields						
TRWAITEN	1	1	1	0	1	0
TRWAITPOL	0	0	0	1	1	0
TRCS2WTR	2	2	2	2	2	2
TRWT2DEWT	2	2	2	2	2	2

Table 8.1-19 Configuration Data for Different Banks

Initialise the required number of memory locations in the different banks with data from the AHB side.

Read accesses to the different banks with the following parameters are performed as shown in the ***Table 8.1-20*** below.

HSIZE	Bank	MSIZE	HBURST
Word	Bank 1	Word	INCR
HalfWord	Bank 3	HalfWord	INCR4
Byte	Bank 5	Byte	INCR8
Word	Bank 7	Word	INCR16
HalfWord	Bank 1	Word	WRAP8
Byte	Bank 3	HalfWord	WRAP8
Word	Bank 5	Byte	WRAP16
HalfWord	Bank 7	Word	SINGLE
Word	Bank 1	Word	Non Burst

HalfWord	Bank 0	HalfWord	WRAP4
Byte	Bank 2	Byte	Wrap4
Byte	Bank 2	Byte	WRAP8

Table 8.1-20 Accesses Types Performed

Writes are attempted on the banks 1, 3 and 5. The data is then read back from the locations on which the writes were attempted and crosschecked for any corruption.

8.1.11 Synchronous BURST ROM TEST [SyBROMTests.c]

This test checks the operation of the SsmcCore with different types of BURST ROMs connected to different banks. The banks 1, 3, 5, 7 are configured as Synchronous memory banks.

The test sequence is as follows. Program bank 1, 3, 5, 7 and the TrickMem connected to these banks as BURST ROMs with different SMBWSTRDRx, SMBWSTWRRx, SMBWSRBRDRx and word size values. Initialise the BURST ROM contents from the AHB side in Configuration register and the TrickMem. Perform burst and non-burst reads from memory banks through the SSMC using different HSIZESMCSMC and different HBURSTSMC values.

Program the SSMCTrCR, SSMCTrBurstWT, SSMCTrWTCNCL, SSMCTrEndian and SSMCTrExtMuxWT registers to the default values.

The different banks are configured as per the data given in the ***Table 8.1-21*** below.

Parameters	**Bank 0**	**Bank 1**	**Bank 2**	**Bank 3**	**Bank 5**	**Bank 7**
WSTIDCY	3	3	3	3	4	2
WSTRD	0	0	0	7	12	5
WSTWR	0	0	0	5	6	2
WSTOEN	0	0	0	3	4	0
WSTWEN	0	0	0	0	0	0
WSTBRD	1	2	1	4	6	8
SMBCRData Fields						
BlWriteEn	0	0	0	0	0	0
AddrValidWriteEn	0	0	0	0	0	0
BurstLenWrite	4	4	4	4	4	4
SyncWriteDev	0	0	0	0	0	0
BMWrite	0	0	0	0	0	0
BlReadEn	0	0	0	0	0	0
AddrValidReadEn	0	0	0	0	0	0
BurstLenRead	4	4	4	4	4	4
SyncReadDev	0	0	0	0	0	0

BMRead	1	1	1	1	1	1
MW	16	32	8	16	8	32
WP	1	1	1	1	1	1
WaitEn	1	1	1	0	1	0
WaitPol	0	0	0	1	1	0
RBLE	1	1	0	1	0	1
SSMCTrCS2WTRData Fields						
TRWAITEN	1	1	1	0	1	0
TRWAITPOL	0	0	0	1	1	0
TRCS2WTR	2	2	2	2	2	2
TRWT2DEWT	2	2	2	2	2	2

Table 8.1-21 Configuration Data for Different Banks

Initialise the required number of memory locations in the different banks with data from the AHB side.

Read accesses to the different banks with the following parameters are performed as shown in Table ***8.1-22*** below.

HSIZE	Bank	MSIZE	HBURST
Word	Bank 1	Word	INCR
HalfWord	Bank 3	HalfWord	INCR4
Byte	Bank 5	Byte	INCR8
Word	Bank 7	Word	INCR16
HalfWord	Bank 1	Word	WRAP8
Byte	Bank 3	HalfWord	WRAP8
Word	Bank 5	Byte	WRAP16
HalfWord	Bank 7	Word	SINGLE
Word	Bank 1	Word	Non Burst
HalfWord	Bank 0	HalfWord	WRAP4
Byte	Bank 2	Byte	Wrap4
Byte	Bank 2	Byte	WRAP8

Table 8.1-22 Access Type Performed

Writes are attempted on the banks 1, 3 and 5. The data is then read back from the locations on which the writes were attempted and crosschecked for any corruption.

8.1.12 Busy and Idle Accesses Test [BusAccessTests.c]

In these tests, adding Busy Transfer in between reads, writes and burst reads from asynchronous memory tests SsmcCore. During continuous sequential Burst reads transfers, a BUSY can be inserted by the AHB master and then continue sequential Burst transfers. The SsmcCore is expected to perform sequential Burst reads after the BUSY transfer (assuming the quad-boundary is not reached). On the other hand, as SSMC breaks burst transfer if an Idle transfer come in between a burst transfer, so SSMC is tested for this case by inserting Idle transfers in between burst transfers. Idle transfers are also inserted in between reads and writes.

Program the SSMCTrCR, SSMCTrBurstWT, SSMCTrWTCNCL, SSMCTrEndian and SSMCTrExtMuxWT registers to the default values.

The Bank 3 is configured as per the data given in the ***Table 8.1-23*** below.

Parameters	Bank 3
WSTIDCY	1
WSTRD	2
WSTWR	3
WSTOEN	1
WSTWEN	1
WSTBRD	2
SMBCRData Fields	
BlWriteEn	0
AddrValidWriteEn	0
BurstLenWrite	4
SyncWriteDev	0
BMWrite	1
BlReadEn	0
AddrValidReadEn	0
BurstLenRead	4
SyncReadDev	0
BMRead	1
MW	32
WP	1
WaitEn	1
WaitPol	1
RBLE	1
SSMCTrCS2WTRData Fields	

TRWAITEN	1
TRWAITPOL	1
TRCS2WTR	2
TRWT2DEWT	2

Table 8.1-23 Configuration Data for the Bank

The following test are carried out where the AHB master

inserts BUSY in between Reads

inserts BUSY in between Writes

inserts BUSY in between Burst Reads

inserts BUSY in between Burst Writes

inserts IDLE in between Reads

inserts IDLE in between Writes

inserts IDLE in between Burst Reads

inserts IDLE in-between Burst Writes

8.1.13 Synchronous Busy and Idle Accesses Test [SyBusAccessTests.c]

In these tests, adding Busy Transfer in between reads, writes and burst reads from synchronous memory tests SsmcCore. During continuous sequential Burst reads transfers, a BUSY can be inserted by the AHB master and then continue sequential Burst transfers. The SsmcCore is expected to perform sequential Burst reads after the BUSY transfer (assuming the quad-boundary is not reached). On the other hand, as SSMC breaks burst transfer if an Idle transfer come in between a burst transfer, so SSMC is tested for this case by inserting Idle transfers in between burst transfers. Idle transfers are also inserted in between reads and writes.

The different banks are configured as per the data given in the ***Table 8.1-24*** below.

Parameters	Bank 3
WSTIDCY	1
WSTRD	2
WSTWR	3
WSTOEN	1
WSTWEN	1
WSTBRD	2
SMBCRData Fields	
BIWriteEn	0
AddrValidWriteEn	1
BurstLenWrite	4
SyncWriteDev	1

BMWrite	1
BlReadEn	0
AddrValidReadEn	1
BurstLenRead	4
SyncReadDev	1
BMRead	1
MW	32
WP	1
WaitEn	0
WaitPol	1
RBLE	1
SSMCTrCS2WTRData Fields	
TRWAITEN	0
TRWAITPOL	1
TRCS2WTR	2
TRWT2DEWT	2

Table 8.1-24 Configuration Data for the Bank

The following tests are carried out.

Inserts BUSY in between Reads

Inserts BUSY in between Writes

Inserts BUSY in between Burst Reads

Inserts BUSY in-between Burst Writes

Inserts IDLE in between Reads

Inserts IDLE in between Writes

Inserts IDLE in between Burst Reads

8.1.14 Bus Grant Tests [BusGrantTests.c]

In this test, different Write/Read transfers are carried out by using the External Bus Multiplexer instead of the Internal Bus Multiplexer. The banks, Bank0, Bank2, Bank4, Bank6 are defined as Synchronous banks, while the remaining banks, Bank1, Bank3, Bank5 and Bank7 are defined as Asynchronous banks. The memory widths of the banks are configured differently from bank to bank.

Different Write/Read transfers are carried out with different AHB widths, different HBURST values.

Program the SSMCTrCR, SSMCTrBurstWT, SSMCTrWTCNCL, SSMCTrEndian and SSMCTrExtMuxWT registers to the default values.

The different banks are configured as per the data given in the ***Table 8.1-25*** table below.

Parameters	Bank 0	Bank 1	Bank 2	Bank 3	Bank 4	Bank 5	Bank 6	Bank 7
WSTIDCY	1	1	1	1	1	1	1	1
WSTRD	2	2	2	2	2	2	2	0
WSTWR	3	0	3	3	0	3	3	3
WSTOEN	1	1	1	1	1	1	1	0
WSTWEN	1	0	1	1	0	1	1	1
WSTBRD	2	2	2	2	2	2	2	2
SMBCRData Fields								
BlWriteEn	0	0	0	0	0	0	0	0
AddrValidWriteEn	1	0	1	0	1	0	1	0
BurstLenWrite	4	4	4	4	4	4	4	4
SyncWriteDev	1	0	1	0	1	0	1	0
BMWrite	1	1	1	1	1	1	1	1
BlReadEn	0	0	0	0	0	0	0	0
AddrValidReadEn	1	0	1	0	1	0	1	0
BurstLenRead	4	4	4	4	4	4	4	4
SyncReadDev	1	0	1	0	1	0	1	0
BMRead	1	1	1	1	1	1	1	1
MW	32	16	8	32	16	8	32	8
WP	0	0	0	0	0	0	0	0
WaitEn	1	0	0	0	0	0	0	0
WaitPol	0	0	1	1	1	1	0	0
RBLE	1	1	0	1	1	0	1	0
SSMCTrCS2WTRData Fields								
TRWAITEN	1	0	0	0	0	0	0	0
TRWAITPOL	0	0	1	1	1	1	0	0
TRCS2WTR	2	2	2	2	2	2	2	2
TRWT2DEWT	2	2	2	2	2	2	2	2

Table 8.1-25 Configuration Data for the Different Banks

The following table shows the tests that are carried out using the different values for the parameters as listed in the ***Table 8.1-26*** below.

AHB SIZE	MEMORY SIZE	HBURST	BANKNO
WORD	WORD	SINGLE	0
HALFWORD	HALFWORD	INCR	1
BYTE	BYTE	INCR4	2
WORD	WORD	INCR8	3
HALFWORD	HALFWORD	INCR16	4
BYTE	BYTE	WRAP4	5
HALFWORD	WORD	WRAP8	6
WORD	BYTE	WRAP16	7
WORD	HALFWORD	INC	1

Table 8.1-26 Access Type Performed.

8.1.15 Wrap Read Tests [WrapReadTests.c]

The test verifies the functionality of the WrapRead bit in bank control register for synchronous memories.

- Bank 0 and 1 are configured as synchronous banks of memory size 32 bit and 16 bit wide respectively.
- Set the WrapRead bit in bank control register.
- Initialize both the memories.
- Verify the data by wrap reads of wrap8 and wrap16 respectively.
- Repeat the same write – read accesses with different data and address combinations.

8.1.16 Chip Select Test [CSTests.c]

Corresponding to any address, one and only one CS should be asserted. This test is performed to check this functionality. This is tested by applying all eight values on HADDR[31:26], keeping remaining 26 address bits [25:0] unchanged. Eight memory locations are written into (such that [25:0] is same, but the chip-select changes), each with different data. The data is read back and compared. If it matches then unique CS assertion for each address is proved.

Program the SSMCTrCR, SSMCTrBurstWT, SSMCTrWTCNCL, SSMCTrEndian and SSMCTrExtMuxWT registers to the default values.

The different banks are configured as per the data given in the ***Table 8.1-27*** below.

Parameters	Bank 0	Bank 1	Bank 2	Bank 3	Bank 4	Bank 5	Bank 6	Bank 7
WSTIDCY	1	1	1	1	1	1	1	1
WSTRD	2	2	2	2	2	2	2	2
WSTWR	3	3	3	3	3	3	3	3
WSTOEN	1	1	1	1	1	1	1	1

WSTWEN	1	1	1	1	1	1	1	1
WSTBRD	2	2	2	2	2	2	2	2
SMBCRData Fields								
BlWriteEn	0	0	0	0	0	0	0	0
AddrValidWriteEn	0	0	0	0	0	0	0	0
BurstLenWrite	4	4	4	4	4	4	4	4
SyncWriteDev	0	0	0	0	0	0	0	0
BMWrite	1	1	1	1	1	1	1	1
BlReadEn	0	0	0	0	0	0	0	0
AddrValidReadEn	0	0	0	0	0	0	0	0
BurstLenRead	4	4	4	4	4	4	4	4
SyncReadDev	0	0	0	0	0	0	0	0
BMRead	1	1	1	1	1	1	1	1
MW	32	32	32	32	32	32	32	32
WP	0	0	0	0	0	0	0	0
WaitEn	1	0	1	0	1	0	1	0
WaitPol	0	0	1	1	1	1	0	0
RBLE	1	1	1	1	1	1	1	1
SSMCTrCS2WTRData Fields								
TRWAITEN	1	0	1	0	1	0	1	0
TRWAITPOL	0	0	1	1	1	1	0	0
TRCS2WTR	2	2	2	2	2	2	2	2
TRWT2DEWT	2	2	2	2	2	2	2	2

Table 8.1-27 Configuration Data for Different Banks

Single Write/Read accesses are performed on the individual banks with different offsets and Data.

8.1.17 Synchronous Chip Select Test [SyCSTests.c]

Corresponding to any address, one and only one CS should be asserted. This test is performed to check this functionality.

This is tested by applying all eight values on HADDR[31:26], keeping remaining 26 address bits [25:0] unchanged. Eight memory locations are written into (such that [25:0] is same, but the chip-select changes), each with different data. The data is read back and compared. If it matches then unique CS assertion for each address is proved.

Program the SSMCTrCR, SSMCTrBurstWT, SSMCTrWTCNCL, SSMCTrEndian and SSMCTrExtMuxWT registers to the default values.

The different banks are configured as per the data given in the ***Table 8.1-28*** below.

Parameters	Bank 0	Bank 1	Bank 2	Bank 3	Bank 4	Bank 5	Bank 6	Bank 7
WSTIDCY	1	1	1	1	1	1	1	1
WSTRD	2	2	2	2	2	2	2	2
WSTWR	3	3	3	3	3	3	3	3
WSTOEN	1	1	1	1	1	1	1	1
WSTWEN	1	1	1	1	1	1	1	1
WSTBRD	2	2	2	2	2	2	2	2
SMBCRData Fields								
BlWriteEn	0	0	0	0	0	0	0	0
AddrValidWriteEn	1	1	1	1	1	1	1	1
BurstLenWrite	4	4	4	4	4	4	4	4
SyncWriteDev	1	1	1	1	1	1	1	1
BMWrite	1	1	1	1	1	1	1	1
BlReadEn	0	0	0	0	0	0	0	0
AddrValidReadEn	1	1	1	1	1	1	1	1
BurstLenRead	4	4	4	4	4	4	4	4
SyncReadDev	1	1	1	1	1	1	1	1
BMRead	1	1	1	1	1	1	1	1
MW	32	32	32	32	32	32	32	32
WP	0	0	0	0	0	0	0	0
WaitEn	0	0	0	0	0	0	0	0
WaitPol	0	0	1	1	1	1	0	0
RBLE	1	1	1	1	1	1	1	1
SSMCTrCS2WTRData Fields								
TRWAITEN	0	0	0	0	0	0	0	0
TRWAITPOL	0	0	1	1	1	1	0	0
TRCS2WTR	2	2	2	2	2	2	2	2
TRWT2DEWT	2	2	2	2	2	2	2	2

Table 8.1-28 Configuration Data for Different Banks

Single Write/Read accesses are performed on the individual banks with different offsets and Data.

8.1.18 Chip select to nSMOEN and nSMWEN assertion Tests [DelayValueTests.c]

In these tests, different values are programmed in CS2OEN, CS2WEN registers, and then burst reads, non-burst reads and writes are performed. All combinations mentioned below of read, write and burst read are tested.

- read followed by burst reads to same and different banks
- read followed by writes to same and different banks
- burst read followed by reads to same and different banks
- burst read followed by writes to same and different banks
- write followed by reads to same and different banks
- write followed by burst reads to same and different banks

Program the SSMCTrCR, SSMCTrBurstWT, SSMCTrWTCNCL, SSMCTrEndian and SSMCTrExtMuxWT registers to the default values.

The different banks are configured as per the data given in the ***Table 8.1-29*** below.

Parameters	Bank 0	Bank 1	Bank 2	Bank 3
WSTIDCY	1	1	1	1
WSTRD	2	2	2	2
WSTWR	3	3	3	3
WSTOEN	1	1	1	1
WSTWEN	1	1	1	1
WSTBRD	2	2	2	2
SMBCRData Fields				
BIWriteEn	0	0	0	0
AddrValidWriteEn	0	0	0	0
BurstLenWrite	4	4	4	4
SyncWriteDev	0	0	0	0
BMWrite	1	1	1	1
BIReadEn	0	0	0	0
AddrValidReadEn	0	0	0	0
BurstLenRead	4	4	4	4
SyncReadDev	0	0	0	0
BMRead	1	1	1	1
MW	32	16	8	32
WP	0	0	0	0
WaitEn	0	1	0	1

WaitPol	0	0	1	1
RBLE	1	1	1	1
SSMCTrCS2WTRData Fields				
TRWAITEN	0	1	0	1
TRWAITPOL	0	0	1	1
TRCS2WTR	2	2	2	2
TRWT2DEWT	2	2	2	2
SMCTrMEMBData	0x00007FFF	0x00007FFF	0x00007FFF	0x00007FFF

Table 8.1-29 Configuration Data for Different Banks

The following types of operations are carried out for a given set of CS2OEN and CS2WEN count values on individual banks as shown in the ***Table 8.1-30*** below.

CS2OEN count value	CS2WEN count value	Tests
5	9	Burst Writes followed by Burst Reads to same Bank
5	9	Burst Reads followed by Burst Writes to same Bank
5	9	Burst Reads followed by Burst Writes to Different Banks
5	9	Burst Writes followed by Burst Reads to Different Banks
5	9	Burst Writes followed by Burst Writes to same Bank
5	9	Burst Reads followed by Burst Reads to same Bank
5	9	Burst Writes followed by Burst Writes to Different Banks
5	9	Burst Reads followed by Burst Reads to Different Banks
5	9	Non-Burst Write followed by Non-Burst Read to same Bank
5	9	Non-Burst Read followed by Non-Burst Write to same Bank
5	9	Non-Burst Read followed by Non-Burst Write to Different Bank
5	9	Non-Burst Write followed by Non-Burst Read to Different Bank
5	9	Non-Burst Write followed by Non-Burst Write to same Bank
5	9	Non-Burst Read followed by Non-Burst Read to same Bank
5	9	Non-Burst Write followed by Non-Burst Write to Different Bank
5	9	Non-Burst Read followed by Non-Burst Read to Different Bank

Table 8.1-30 Access Types Performed

Note: The above tests hold good for synchronous memories, since the single synchronous read operations have the same control signal timing as an asynchronous read operations.

8.1.19 Asynchronous Multiple memory bank test [MemoryBankTests.c]

This test checks the operation of the SsmcCore across the bank. The test sequence is as follows. Program the bank zero and the TrickMem connected to bank zero as SRAM of data size 32-bit. Program bank one and the TrickMem connected to bank one as SRAM of data size of 16-bit. Program bank 3 and the TrickMem connected to bank three as SRAM of data size of 8-bit. Program the different banks with different access times and different memory idle times. Read and a write accesses to the memory from AHB with different addresses are performed such that it makes the SsmcCore access different banks.

Program the SSMCTrCR, SSMCTrBurstWT, SSMCTrWTCNCL, SSMCTrEndian and SSMCTrExtMuxWT registers to the default values.

The different banks are configured as per the data given in the ***Table 8.1-31*** below.

Parameters	Bank 0	Bank 1	Bank 2
WSTIDCY	1	1	1
WSTRD	2	2	2
WSTWR	3	3	3
WSTOEN	1	1	1
WSTWEN	1	1	1
WSTBRD	2	2	2
SMBCRData Fields			
BlWriteEn	0	0	0
AddrValidWriteEn	0	0	0
BurstLenWrite	4	4	4
SyncWriteDev	0	0	0
BMWrite	1	1	1
BlReadEn	0	0	0
AddrValidReadEn	0	0	0
BurstLenRead	4	4	4
SyncReadDev	0	0	0
BMRead	1	1	1
MW	32	16	8
WP	0	0	0
WaitEn	1	1	1
WaitPol	0	0	1
RBLE	1	1	0
SSMCTrCS2WTRData Fields			

TRWAITEN	1	1	1
TRWAITPOL	0	0	1
TRCS2WTR	2	2	2
TRWT2DEWT	2	2	2

Table 8.1-31 Configuration Data for Different Banks

The different tests, which are carried out, are listed in ***Table 8.1-32*** below

Operation	Bank No	Offset	HSIZE	MSIZE	HBURST
WRITE	1	0x07DC	WORD	HALFWORD	INCR
WRITE	0	0x7D0	WORD	WORD	INCR
WRITE	2	0x004	WORD	BYTE	INCR
WRITE	1	0X008	WORD	HALFWORD	INCR
WRITE	2	0X00C	WORD	BYTE	INCR
WRITE	0	0X010	WORD	WORD	INCR
WRITE	1	0X014	WORD	HALFWORD	INCR
READ	1	0x07DC	WORD	HALFWORD	INCR
READ	0	0x7D0	WORD	WORD	INCR
READ	2	0x004	WORD	HALFWORD	INCR
READ	1	0X008	WORD	BYTE	INCR
READ	2	0X00C	WORD	HALFWORD	INCR
READ	0	0X010	WORD	WORD	INCR
READ	1	0X014	WORD	BYTE	INCR

Table 8.1-32 Access Types Performed

8.1.20 Synchronous Multiple memory bank test [SyMemoryBankTests.c]

Program the bank zero and the TrickMem connected to bank two as synchronous SRAM of data size 32-bit. Program bank 4 and the TrickMem connected to bank four as Synchronous SRAM of data size of 16-bit. Program bank 6 and the TrickMem connected to bank six as Synchronous SRAM of data size of 8-bit. Program the different banks with different access times and different memory idle times. Read and a write accesses to the memory from AHB with different addresses are performed such that it makes the SsmcCore access different banks.

Program the SSMCTrCR, SSMCTrBurstWT, SSMCTrWTCNCL, SSMCTrEndian and SSMCTrExtMuxWT registers to the default values.

The different banks are configured as per the data given in the ***Table 8.1-33*** below.

Parameters	Bank 0	Bank 1	Bank 2

WSTIDCY	1	1	1
WSTRD	2	2	2
WSTWR	3	3	3
WSTOEN	1	1	1
WSTWEN	1	1	1
WSTBRD	2	2	2
SMBCRData Fields			
BlWriteEn	0	0	0
AddrValidWriteEn	1	1	1
BurstLenWrite	4	4	4
SyncWriteDev	1	1	1
BMWrite	1	1	1
BlReadEn	0	0	0
AddrValidReadEn	1	1	1
BurstLenRead	4	4	4
SyncReadDev	1	1	1
BMRead	1	1	1
MW	32	16	8
WP	0	0	0
WaitEn	1	1	1
WaitPol	0	0	1
RBLE	1	1	0
SSMCTrCS2WTRData Fields			
TRWAITEN	1	1	1
TRWAITPOL	0	0	1
TRCS2WTR	2	2	2
TRWT2DEWT	2	2	2

Table 8.1-33 Configuration Data for the Different Banks

The different tests, which are carried out, are listed in ***Table 8.1-36*** below

Operation	Bank No	Offset	HSIZE	MSIZE	HBURST
WRITE	1	0x07DC	WORD	HALFWORD	INCR

WRITE	0	0x7D0	WORD	WORD	INCR
WRITE	2	0x004	WORD	BYTE	INCR
WRITE	1	0X008	WORD	HALFWORD	INCR
WRITE	2	0X00C	WORD	BYTE	INCR
WRITE	0	0X010	WORD	WORD	INCR
WRITE	1	0X014	WORD	HALFWORD	INCR
READ	1	0x07DC	WORD	HALFWORD	INCR
READ	0	0x7D0	WORD	WORD	INCR
READ	2	0x004	WORD	HALFWORD	INCR
READ	1	0X008	WORD	BYTE	INCR
READ	2	0X00C	WORD	HALFWORD	INCR
READ	0	0X010	WORD	WORD	INCR
READ	1	0X014	WORD	BYTE	INCR

Table 8.1-34 Access Types Performed

8.1.21 Performance tests [PerformanceTests.c]

SsmcCore performs reads, writes and burst reads to memory with different sizes and different banks. So this test is used to check whether the SsmcCore gives data to the requester immediately. After completion of a valid read or write the SsmcCore can still hold HREADYOUT low. This test checks its performance by verifying that the read or write is completed within the expected number of clock cycles. The following timings are checked in this test:

- SMBWSTRDR
- SMBWSTWRR
- SMBWSTBRD
- Idle/Turn-Around Cycles

Additional Corner Case Tests:

1. When a fresh write transfer (for which the HREADYOUT + OKAY response is given with zero waits) is in progress, next write transfer is initiated with the following condition. Between these two transfer requests, IDLE cycles equal to the WSTWRR (write access time value) are introduced. This condition is used to align the subsequent write request (kept pending on bus) exactly to the last cycle of the current write transaction.
2. In the above scenario instead of the subsequent write, a read with the same parameters is initiated.
3. A new/initial write transfer is initiated when the SsmcCore is performing turnarounds (i.e. it is in the ST_TURNAROUND state). Then the number of IDLE cycles required before the next write or a read transfer is requested is equal to the number of turn-around cycles remaining in addition to WSTRDR, WSTWRR values. This condition is again used to align the subsequent write/read request (kept pending on bus) exactly to the last cycle of the current write transaction. With these specific conditions Write-Write and Write-Read transfer tests are performed.

4. Tests are done with write transaction to a bank having {a} WSTOEN = 1 and WSTRDR = 1, and {b} WSTOEN and WSTWEN having it value other than ‘1’. This will test the SMWEN assertion delay logic and the access time counter logic for the equal values of the two parameters.

8.1.22 Write Protection and Bus Error Flag tests [ProtectionTests.c]

This test checks the setting and clearing logic of the HSIZESMCERR and WriteProtErr flags.

The HSIZESMCERR flag is set when the master tries to perform an operation to the memory, with a transfer size greater than 32-bits. Writing ‘1’ to this bit location clears this flag.

WriteProtErr flag is set when the master tries to perform a write operation to a bank, which is configured as ROM, or Burst ROM. Writing ‘1’ to this bit location clears this flag. Tests are also performed for checking WriteProtErr flag for write protection mode for SRAM’s. In this mode, a write operation is performed to a bank, which is configured as SRAM with Write Protection bit set.

Program the SSMCTrCR, SSMCTrBurstWT, SSMCTrWTCNCL, SSMCTrEndian and SSMCTrExtMuxWT registers to the default values.

The different banks are configured as per the data given in the ***Table 8.1-35*** below.

Parameters	Bank 0	Bank 4	Bank 5	Bank 6	Bank 7
WSTIDCY	1	1	1	1	1
WSTRD	2	2	2	2	2
WSTWR	3	3	3	3	3
WSTOEN	1	1	1	1	1
WSTWEN	1	1	1	1	1
WSTBRD	2	2	2	2	2
SMBCRData Fields					
BlWriteEn	0	0	0	0	0
AddrValidWriteEn	0	0	0	0	0
BurstLenWrite	4	4	4	4	4
SyncWriteDev	0	0	0	0	0
BMWrite	0	0	0	0	0
BlReadEn	0	0	0	0	0
AddrValidReadEn	0	0	0	0	0
BurstLenRead	4	4	4	4	4
SyncReadDev	0	0	0	0	0
BMRead	1	1	1	1	1
MW	32	32	32	8	16
WP	1	1	1	1	1

WaitEn	0	1	1	0	0
WaitPol	0	1	1	0	0
RBLE	1	1	1	0	0
SSMCTrCS2WTRData Fields					
TRWAITEN	0	1	1	0	0
TRWAITPOL	0	1	1	0	0
TRCS2WTR	2	2	2	2	2
TRWT2DEWT	2	2	2	2	2

Table 8.1-35 Configuration Data for Different Banks

The different memories are initialised from the AHB side.

The different tests that are carried out are as given below;

Attempting a Non-Word access on to the Registers and expecting an error response.

Writes are attempted on the Write Protected memories and Error Response is expected.

A Double-Word access is attempted on the registers and error response is expected.

8.1.23 RBLE tests [RBLETests.c]

In these tests, SsmcCore is tested for RBLE functionality. In these tests we configure alternate banks with RBLE value ‘1’. Then various burst transfers are tried which crossover from one bank to next bank. These transfers are listed in the ***Table 8.1-36***

Current Transfer with RBLE status	Next transfer with RBLE status
RBLE enabled read transfer	RBLE disabled write transfer to different bank
RBLE enabled write transfer	RBLE disabled read transfer to different bank
RBLE enabled read transfer	RBLE disabled read transfer to different bank
RBLE enabled write transfer	RBLE disabled write transfer to different bank
RBLE disabled read transfer	RBLE enabled write transfer to different bank
RBLE disabled read transfer	RBLE enabled read transfer to different bank
RBLE disabled write transfer	RBLE enabled read transfer to different bank
RBLE disabled write transfer	RBLE enabled write transfer to different bank
RBLE enabled write transfer	RBLE enabled read transfer to same bank
RBLE enabled read transfer	RBLE enabled write transfer to same bank
RBLE disabled write transfer	RBLE disabled read transfer to same bank
RBLE disabled read transfer	RBLE disabled write transfer to same bank

Table 8.1-36 Different RBLE Transfers

8.1.24 ROM test [ROMTests.c]

This test checks the operation of the SsmcCore with different types of ROMs connected to different banks.

The test sequence is as follows. Program bank zero, two, four, six and the TrickMem connected to these banks as ROMs with different SMBWSTRDRx values and different word sizes. Initialise the ROM contents from the AHB side in Configuration register and TrickMem. Read the memory banks through the SsmcCore using different HSIZESMCSMC and different HBURSTSMC values.

In addition, this test emulates the scenario when IDLE transfers are introduced immediately after a read transaction with the following combinations HSELSMC = 1, HWRITESMC = 0 and Read has just completed (i.e. HREADYOUTSMC has been given for the read transaction).

Program the SSMCTrCR, SSMCTrBurstWT, SSMCTrWTCNCL, SSMCTrEndian and SSMCTrExtMuxWT registers to the default values.

The different banks are configured as per the data given in the ***Table 8.1-37*** below.

Parameters	Bank 0	Bank 2	Bank 4	Bank 6
WSTIDCY	2	3	4	2
WSTRD	5	7	11	10
WSTWR	4	0	7	8
WSTOEN	2	3	4	5
WSTWEN	0	0	0	0
WSTBRD	2	2	2	2
SMBCRData Fields				
BlWriteEn	0	0	0	0
AddrValidWriteEn	0	0	0	0
BurstLenWrite	4	4	4	4
SyncWriteDev	0	0	0	0
BMWrite	0	0	0	0
BlReadEn	0	0	0	0
AddrValidReadEn	0	0	0	0
BurstLenRead	4	4	4	4
SyncReadDev	0	0	0	0
BMRead	1	1	1	1
MW	16	32	8	32
WP	1	1	1	1
WaitEn	1	0	1	0
WaitPol	0	1	1	0

RBLE	1	1	0	1
SSMCTrCS2WTRData Fields				
TRWAITEN	1	0	1	0
TRWAITPOL	0	1	1	0
TRCS2WTR	2	2	2	2
TRWT2DEWT	2	2	2	2

Table 8.1-37 Configuration Data for Different Banks

Initialise the ROMs from the AHB side with the data to be read. Read accesses are done on the different banks with the different parameters as given in the ***Table 8.1-38*** below.

HSIZE	HBURST	MSIZE	BANK	ACCESS
WORD	INCR	16	0	Read
HALFWORD	INCR4	16	0	READ
BYTE	INCR8	16	0	READ
WORD	INCR16	32	2	READ
HALFWORD	WRAP4	32	2	READ
BYTE	WRAP8	32	2	READ
WORD	WRAP16	8	4	READ
HALFWORD	INCR	8	4	READ
BYTE	INCR4	8	4	READ
WORD	INCR8	32	6	READ
HALFWORD	INCR16	32	6	READ
BYTE	WRAP4	32	6	READ
BYTE	WRAP8	32	2	READ
BYTE	SINGLE	32	2	READ
WORD	WRAP16	8	4	READ

Table 8.1-38 Access Types Performed

8.1.25 Synchronous ROM Tests [SyROMTests.c]

This test checks the operation of the SsmcCore with different types of ROMs connected to different banks.

The test sequence is as follows. Program bank zero, two, four, six and the TrickMem connected to these banks as ROMs with different SMBWSTRDRx values and different word sizes. Initialise the ROM contents from the AHB side in Configuration register and TrickMem. Read the memory banks through the SsmcCore using different HSIZESMCSMC and different HBURSTSMC values.

In addition, this test emulates the scenario when IDLE transfers are introduced immediately after a read transaction with the following combinations HSELSMC = 1, HWRITESMC = 0 and Read has just completed (i.e. HREADYOUTSMC has been given for the read transaction).

Program the SSMCTrCR, SSMCTrBurstWT, SSMCTrWTCNCL, SSMCTrEndian and SSMCTrExtMuxWT registers to the default values.

The different banks are configured as per the data given in the ***Table 8.1-39*** below.

Parameters	Bank 0	Bank 2	Bank 4	Bank 6
WSTIDCY	2	3	4	2
WSTRD	5	7	11	10
WSTWR	4	0	7	8
WSTOEN	2	3	4	5
WSTWEN	0	0	0	0
WSTBRD	2	2	2	2
SMBCRData Fields				
BlWriteEn	0	0	0	0
AddrValidWriteEn	1	1	1	1
BurstLenWrite	4	4	4	4
SyncWriteDev	1	1	1	1
BMWrite	1	1	1	1
BlReadEn	0	0	0	0
AddrValidReadEn	1	1	1	1
BurstLenRead	4	4	4	4
SyncReadDev	1	1	1	1
BMRead	1	1	1	1
MW	16	32	8	32
WP	1	1	1	1
WaitEn	1	0	1	0
WaitPol	0	1	1	0
RBLE	1	1	0	1
SSMCTrCS2WTRData Fields				
TRWAITEN	1	0	1	0
TRWAITPOL	0	1	1	0

TRCS2WTR	2	2	2	2
TRWT2DEWT	2	2	2	2

Table 8.1-39 Configuration Data for Different Banks

Initialise the ROMs from the AHB side with the data to be read. Read accesses are done on the different banks with the different parameters as given in the ***Table 8.1-40*** below.

HSIZE	HBURST	MSIZE	BANK	ACCESS
WORD	INCR	16	0	Read
HALFWORD	INCR4	16	0	READ
BYTE	INCR8	16	0	READ
WORD	INCR16	32	2	READ
HALFWORD	WRAP4	32	2	READ
BYTE	WRAP8	32	2	READ
WORD	WRAP16	8	4	READ
HALFWORD	INCR	8	4	READ
BYTE	INCR4	8	4	READ
WORD	INCR8	32	6	READ
HALFWORD	INCR16	32	6	READ
BYTE	WRAP4	32	6	READ
BYTE	WRAP8	32	2	READ
BYTE	SINGLE	32	2	READ
WORD	WRAP16	8	4	READ

Table 8.1-40 Access Types Performed

8.1.26 Random tests [RandomTests.c, SyRandomTests.c]

In these tests, various combinations of different sizes, different bursts and transfer types are attempted on different Banks and on both types of devices. These accesses are generated randomly using a random function.

Program the SSMCTrCR, SSMCTrBurstWT, SSMCTrWTCNCL, SSMCTrEndian and SSMCTrExtMuxWT registers to the default values.

The different banks are configured as per the data given in the ***Table 8.1-41*** below.

Parameters	Bank 0	Bank 1	Bank 2	Bank 3	Bank 4	Bank 5	Bank 6	Bank 7
WSTIDCY	2	2	3	1	4	3	4	2
WSTRD	6	5	5	7	7	7	6	5
WSTWR	5	4	4	4	7	5	5	5

WSTOEN	2	1	1	1	2	3	1	1
WSTWEN	2	0	1	0	1	2	1	1
WSTBRD	2	2	2	2	2	2	2	2
BlWriteEn	0	0	0	0	0	0	0	0
AddrValidWriteEn	0/1	0/1	0/1	0/1	0/1	0/1	0/1	0/1
BurstLenWrite	4	4	4	4	4	4	4	4
SMBCRData Fields								
SyncWriteDev	0/1	0/1	0/1	0/1	0/1	0/1	0/1	0/1
BMWrite	1	1	1	1	1	1	1	1
BlReadEn	0	0	0	0	0	0	0	0
AddrValidReadEn	0/1	0/1	0/1	0/1	0/1	0/1	0/1	0/1
BurstLenRead	4	4	4	4	4	4	4	4
SyncReadDev	0/1	0/1	0/1	0/1	0/1	0/1	0/1	0/1
BMRead	1	1	1	1	1	1	1	1
MW	32	16	8	32	16	8	32	16
WP	0	0	0	0	0	0	0	0
WaitEn	0	0	1	1	1	1	0	0
WaitPol	0	0	1	1	1	1	0	0
RBLE	1	1	0	1	1	0	1	1
SSMCTrCS2WTRData Fields								
TRWAITEN	0	0	1	1	1	1	0	0
TRWAITPOL	0	0	1	1	1	1	0	0
TRCS2WTR	2	2	2	2	2	2	2	2
TRWT2DEWT	2	2	2	2	2	2	2	2

Table 8.1-41 Configuration Data for Different Banks

8.1.27 Register reset value tests [ResetTests.c]

This test ensures that the reset has been initiated by the testbench, and checks this by reading the registers for the default values.

The Reset signal is asserted. The following registers are read immediately to check for the reset values as defined in the ***Table 8.1-42*** below. The actual address of the register is the SSMCRegBase + the address offset.

Register Name	Address Offset	Reset Value

SMBIDCYR0	0x00	0xF
SMBIDCYR1	0x20	0xF
SMBIDCYR2	0x40	0xF
SMBIDCYR3	0x60	0xF
SMBIDCYR4	0x80	0xF
SMBIDCYR5	0xA0	0xF
SMBIDCYR6	0xC0	0xF
SMBIDCYR7	0xE0	0xF
SMBWSTRDR0	0x04	0x1F
SMBWSTRDR1	0x24	0x1F
SMBWSTRDR2	0x44	0x1F
SMBWSTRDR3	0x64	0x1F
SMBWSTRDR4	0x84	0x1F
SMBWSTRDR5	0xA4	0x1F
SMBWSTRDR6	0xC4	0x1F
SMBWSTRDR7	0xE4	0x1F
SMBWSTWRR0	0x08	0x1F
SMBWSTWRR1	0x28	0x1F
SMBWSTWRR2	0x48	0x1F
SMBWSTWRR3	0x68	0x1F
SMBWSTWRR4	0x88	0x1F
SMBWSTWRR5	0xA8	0x1F
SMBWSTWRR6	0xC8	0x1F
SMBWSTWRR7	0xE8	0x1F
SMBWSTBRDR0	0x1C	0x1F
SMBWSTBRDR1	0x3C	0x1F
SMBWSTBRDR2	0x5C	0x1F
SMBWSTBRDR3	0x7C	0x1F
SMBWSTBRDR4	0x9C	0x1F
SMBWSTBRDR5	0xBC	0x1F
SMBWSTBRDR6	0xDC	0x1F

SMBWSTBRDR7	0xFC	0x1F
SMBWSTOENR0	0x0C	0x0
SMBWSTOENR1	0x2C	0x0
SMBWSTOENR2	0x4C	0x0
SMBWSTOENR3	0x6C	0x0
SMBWSTOENR4	0x8C	0x0
SMBWSTOENR5	0xAC	0x0
SMBWSTOENR6	0xCC	0x0
SMBWSTOENR7	0xEC	0x0
SMBWSTWENR0	0x10	0x1
SMBWSTWENR1	0x30	0x1
SMBWSTWENR2	0x50	0x1
SMBWSTWENR3	0x70	0x1
SMBWSTWENR4	0x90	0x1
SMBWSTWENR5	0xB0	0x1
SMBWSTWENR6	0xD0	0x1
SMBWSTWENR7	0xF0	0x1
SMBCR0	0x14	0x80
SMBCR1	0x34	0x00
SMBCR2	0x54	0x40
SMBCR3	0x74	0x00
SMBCR4	0x94	0x80
SMBCR5	0xB4	0x80
SMBCR6	0xD4	0x40
SMBCR7	0xF4	0x00
SMBSR0	0x18	0x0
SMBSR1	0x38	0x0
SMBSR2	0x58	0x0
SMBSR3	0x78	0x0
SMBSR4	0x98	0x0
SMBSR5	0xB8	0x0

SMBSR6	0xD8	0x0
SMBSR7	0xF8	0x0
SSMCSR	0x200	0x0
SSMCCR	0x204	0x1
SSMCITCR	0x208	0x0
SSMCITOP	0x20C	0x0
SSMCITIP	0x210	System Dependent
SSMCPeriphID0	0xFE0	0x93
SSMCPeriphID1	0xFE4	0x10
SSMCPeriphID2	0xFE8	0x04
SSMCPeriphID3	0xFEC	0x00
SSMCPCellID0	0xFF0	0x0D
SSMCPCellID1	0xFF4	0xF0
SSMCPCellID2	0xFF8	0x05
SSMCPCellID3	0xFFC	0xB1

Table 8.1-42 Reset Test Data Format

8.1.28 Register read / write test [RegisterTests.c]

This is to check the normal mode of operation of the testbench and the uut. It ensures that the read/write operations and the addressing of the registers are done properly.

All writeable registers are written with patterns such as 0x00000000, 0x55555555, 0xAAAAAAAA and 0xFFFFFFFF. The data is read back from the registers and compared with the expected data. The expected data will depend on the register size and read/write ability of the register. The setting and clearing mechanism of the SSMCBSRx registers will be checked during other memory transfer and SsmcCore performance tests.

HSELREG is used for accessing the internal registers of the SsmcCore. Asserting HSELSMC and ascertaining that the internal register contents are not modified and vice-versa additionally verify the internal register read-writes.

This test targets the register bits individually. The objective of these tests is:
Every bit that is a read only bit should return only the read-only value (zero / one) irrespective of the values of other bits / registers.
Every bit that is a read write bit should accept the new value written into it, irrespective of the values of other bits / registers.
The following ***Table 8.1-43*** lists the read-only registers of the SSMC. It also indicates the read-only value of these registers. During the register read-write tests, the register will be written with different write patterns as listed in the previous test case. However while reading the same register, the expected value should be same as it is in table.

Register Name	Address Offset	Read Value
SSMCSR	0x200	0x0
SSMCPeriphID0	0xFE0	0x93
SSMCPeriphID1	0xFE4	0x10

SSMCPeriphID2	0xFE8	0x04
SSMCPeriphID3	0xFEC	0x00
SSMCPCellID0	0xFF0	0x0D
SSMCPCellID1	0xFF4	0xF0
SSMCPCellID2	0xFF8	0x05
SSMCPCellID3	0xFFC	0xB1

Table 8.1-43 Address Offset and Expected Data Of Read Only Registers

The read-write registers and the valid bits of the SSMC are listed in ***Table 8.1-44*** below. The valid bits are defined as the actual read-write bits in the register. The other bits may be read-only, write-only or reserved bits. A mask pattern for every register is also listed out in the table. In the tests described below, write the pattern as per the tests. While reading a register, the write pattern should be AND-ed with the mask pattern of the register and that forms the expected value to be read from the register.

Register Name	Address Offset	Valid Bits	Mask Pattern
SMBIDCYR0	0x00	3:0	0x0000000F
SMBWSTRDR0	0x04	4:0	0x0000001F
SMBWSTWRR0	0x08	4:0	0x0000001F
SMBWSTOENR0	0x0C	3:0	0x0000000F
SMBWSTWENR0	0x10	3:0	0x0000000F
SMBCR0	0x14	7:0	0x000000FF
SMBSR0	0x18	2:0	0x00000007
SMBWSTBRDR0	0x1C	4:0	0x0000001F
SMBIDCYR1	0x20	3:0	0x0000000F
SMBWSTRDR1	0x24	4:0	0x0000001F
SMBWSTWRR1	0x28	4:0	0x0000001F
SMBWSTOENR1	0x2C	3:0	0x0000000F
SMBWSTWENR1	0x30	3:0	0x0000000F
SMBCR1	0x34	7:0	0x000000FF
SMBSR1	0x38	2:0	0x00000007
SMBWSTBRDR1	0x3C	4:0	0x0000001F
SMBIDCYR2	0x40	3:0	0x0000000F
SMBWSTRDR2	0x44	4:0	0x0000001F
SMBWSTWRR2	0x48	4:0	0x0000001F
SMBWSTOENR2	0x4C	3:0	0x0000000F
SMBWSTWENR2	0x50	3:0	0x0000000F
SMBCR2	0x54	7:0	0x000000FF
SMBSR2	0x58	2:0	0x00000007
SMBWSTBRDR2	0x5C	4:0	0x0000001F
SMBIDCYR3	0x60	3:0	0x0000000F
SMBWSTRDR3	0x64	4:0	0x0000001F
SMBWSTWRR3	0x68	4:0	0x0000001F

SMBWSTOENR3	0x6C	3:0	0x00000007
SMBWSTWENR3	0x70	3:0	0x0000000F
SMBCR3	0x74	7:0	0x000000FF
SMBSR3	0x78	2:0	0x00000007
SMBWSTBRDR3	0x7C	4:0	0x0000001F
SMBIDCYR4	0x80	3:0	0x0000000F
SMBWSTRDR4	0x84	4:0	0x0000001F
SMBWSTWRR4	0x88	4:0	0x0000001F
SMBWSTOENR4	0x8C	3:0	0x0000000F
SMBWSTWENR4	0x90	3:0	0x0000000F
SMBCR4	0x94	7:0	0x000000FF
SMBSR4	0x98	2:0	0x00000007
SMBWSTBRDR4	0x9C	4:0	0x0000001F
SMBIDCYR5	0xA0	3:0	0x0000000F
SMBWSTRDR5	0xA4	4:0	0x0000001F
SMBWSTWRR5	0xA8	4:0	0x0000001F
SMBWSTOENR5	0xAC	3:0	0x0000000F
SMBWSTWENR5	0xB0	3:0	0x0000000F
SMBCR5	0xB4	7:0	0x000000FF
SMBSR5	0xB8	2:0	0x00000007
SMBWSTBRDR5	0xBC	4:0	0x0000001F
SMBIDCYR6	0xC0	3:0	0x0000000F
SMBWSTRDR6	0xC4	4:0	0x0000001F
SMBWSTWRR6	0xC8	4:0	0x0000001F
SMBWSTOENR6	0xCC	3:0	0x0000000F
SMBWSTWENR6	0xD0	3:0	0x0000000F
SMBCR6	0xD4	7:0	0x000000FF
SMBSR6	0xD8	2:0	0x00000007
SMBWSTBRDR6	0xDC	4:0	0x0000001F
SMBIDCYR7	0xE0	3:0	0x0000000F
SMBWSTRDR7	0xE4	4:0	0x0000001F
SMBWSTWRR7	0xE8	4:0	0x0000001F
SMBWSTOENR7	0xEC	3:0	0x0000000F
SMBWSTWENR7	0xF0	3:0	0x0000000F
SMBCR7	0xF4	7:0	0x000000FF
SMBSR7	0xF8	2:0	0x00000007
SMBWSTBRDR7	0xFC	4:0	0x0000001F
SSMCCR	0x204	0	0x00000001
SSMCICTR	0x208	0	0x00000001

SSMCITOP	0x20C	1:0	0x00000003
SSMCITIP	0X210	3:0	0x0000000F

Table 8.1-44 Address Offset and Mask Pattern For Read/Write Registers

8.1.29 SMWAIT Flag tests [SMWAITFlagTests.c]

In these tests, SMWAIT is asserted and de-asserted before and after 32 wait states and its effect on SMWAIT flag is monitored. SMWAIT is also asserted after SMBWSTRDRx/SMBWSTWRRx followed by transfers to that bank and other banks to check SMWAIT flag is not set and it gives error responses. Boundary cases of assertion of SMWAIT just before the SMBWSTRDRx/SMBWSTWRRx counter expiry and also on the same clock of counter expiry are also tested. A boundary case of de-assertion of SMWAIT on the 32-wait state is also tested.

In this set of test cases, active high polarity for the SMWAIT is used for the different bank i.e. WaitPol = 1. Since the external wait signal polarity is programmable, this ensures that the opposite polarity condition is also tested when compared to the tests in the SMWAITTests test code.

For the External Wait Timeout case, the status of the SMWAIT input is stored by the SsmcCore in the SMBEWS read-only register, which can be polled by the software. Asserting the SMCANCELWAIT input tests this functionality, when the SMWAIT input is already asserted. The setting of the Wait Time-Out Error flag is first checked and then the SSMCSR register is continuously polled to check if the SMWAIT status is reflected according to expectation. In addition, memory transfers are initiated from the AHB after the Wait Timeout Error flag is set, to check if error response is generated.

Additional Test: This is an explicit test in which the SMWAIT is asserted exactly when the External Wait assertion counter reaches the expiry value. In this scenario it is expected that the SsmcCore give more priority to the SMWAIT assertion over the counter expiry.

Program the SSMCTrCR, SSMCTrBurstWT, SSMCTrWTCNCL, SSMCTrEndian and SSMCTrExtMuxWT registers to the default values.

The different banks are configured as per the data given in the ***Table 8.1-45*** below.

Parameters	Bank 0	Bank 1	Bank 2	Bank 3	Bank 4	Bank 5	Bank 6	Bank 7
WSTIDCY	**2**	**2**	**2**	**2**	**2**	**3**	**4**	**2**
WSTRD	**15**	**15**	**0**	**0**	**10**	**10**	**11**	**12**
WSTWR	**15**	**15**	**4**	**4**	**4**	**0**	**2**	**1**
WSTOEN	**2**	**2**	**0**	**0**	**2**	**3**	**4**	**5**
WSTWEN	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**
WSTBRD	**1**	**2**	**2**	**2**	**2**	**2**	**2**	**2**
BlWriteEn	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**
AddrValidWriteEn	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**
BurstLenWrite	**4**	**4**	**4**	**4**	**4**	**4**	**4**	**4**
SMBCRData Fields								
SyncWriteDev	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**
BMWrite	**1**	**1**	**1**	**1**	**1**	**1**	**1**	**1**

BlReadEn	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**
AddrValidReadEn	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**
BurstLenRead	**4**	**4**	**4**	**4**	**4**	**4**	**4**	**4**
SyncReadDev	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**
BMRead	**1**	**1**	**1**	**1**	**1**	**1**	**1**	**1**
MW	**32**	**32**	**32**	**32**	**16**	**16**	**8**	**8**
WP	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**
WaitEn	**1**	**1**	**1**	**1**	**1**	**1**	**1**	**1**
WaitPol	**1**	**1**	**1**	**1**	**1**	**1**	**1**	**1**
RBLE	**1**	**1**	**1**	**1**	**1**	**1**	**0**	**0**
SSMCTrCS2WTRData Fields								
TRWAITEN	**1**	**1**	**1**	**1**	**1**	**1**	**1**	**1**
TRWAITPOL	**1**	**1**	**1**	**1**	**1**	**1**	**1**	**1**
TRCS2WTR	**2**	**2**	**2**	**2**	**2**	**2**	**2**	**2**
TRWT2DEWT	**2**	**2**	**2**	**2**	**2**	**2**	**2**	**2**

Table 8.1-45 Configuration Data for the Different Banks

The following tests are carried out to check the Error Response when

SMWAIT is asserted before counter expiry

SMWAIT is asserted after counter expiry

SMWAIT is asserted at counter expiry

Cancel Wait signal is asserted during Write Access for all banks

SMWAIT is De-Asserted after SMCANCELWAIT assertion

SMWAIT asserted when count = 1

Case where an externally waited write to a 16 bit memory is cancelled and a Read access is done to a 32 bit memory

8.1.30 Endianness Tests [SsmcEndiannessTests.c]

The Endianness tests perform the following test sequences:

- Memory configured as 32 bits. Transactions are initiated from the AHB with HSIZESMC of Byte, Half-Word and Word. These transactions may be burst or single transfers. The written data patterns are read back from memory both via the AHB interface of the TrickMem and via the SsmcCore.
- Memory configured as 16 bits. Transactions are initiated from the AHB with HSIZESMC of Byte, Half-Word and Word. These transactions may be burst or single transfers. The written data patterns are read back from memory both via the AHB interface of the TrickMem and via the SsmcCore.

- Memory configured as 8 bits. Transactions are initiated from the AHB with HSIZESMC of Byte, Half-Word and Word. These transactions may be burst or single transfers. The written data patterns are read back from memory both via the AHB interface of the TrickMem and via the SsmcCore.

The above 3 test sequences are repeated for Little and Big Endian modes of operation. Once Endianness is set, it is same for the whole system (AHB and Memory models).

Program the SSMCTrCR, SSMCTrBurstWT, SSMCTrWTCNCL, SSMCTrEndian and SSMCTrExtMuxWT registers to the default values.

The different banks are configured as per the data given in the ***Table 8.1-46*** below.

Parameters	Bank 0	Bank 1	Bank 2	Bank 3	Bank 4	Bank 5	Bank 6	Bank 7
WSTIDCY	3	3	3	3	3	3	3	3
WSTRD	7	7	7	7	7	7	7	7
WSTWR	0	0	0	0	0	0	0	0
WSTOEN	0	0	0	0	0	0	0	0
WSTWEN	0	0	0	0	0	0	0	0
WSTBRD	2	1	2	3	2	2	2	2
SMBCRData Fields								
BIWriteEn	0	0	0	0	0	0	0	0
AddrValidWriteEn	0	0	0	0	0	0	0	0
BurstLenWrite	4/8	4/8	4/8	4/8	4/8	4/8	4/8	4/8
SyncWriteDev	0	0	0	0	0	0	0	0
BMWrite	1	1	1	1	1	1	1	1
BIReadEn	0	0	0	0	0	0	0	0
AddrValidReadEn	0	0	0	0	0	0	0	0
BurstLenRead	4/8	4/8	4/8	4/8	4/8	4/8	4/8	4/8
SyncReadDev	0	0	0	0	0	0	0	0
BMRead	1	1	1	1	1	1	1	1
MW	8/16/32	8/16/32	8/16/32	8/16/32	8/16/32	8/16/32	8/16/32	8/16/32
WP	0	0	0	0	0	0	0	0
WaitEn	1/0	1/0	1/0	1/0	1/0	1/0	1/0	1/0
WaitPol	0	0	0	0	0	0	0	0
RBLE	1/0	1/0	1/0	1/0	1/0	1/0	1/0	1/0
SSMCTrCS2WTRData Fields								
TRWAITEN	1/0	1/0	1/0	1/0	1/0	1/0	1/0	1/0

TRWAITPOL	0	0	0	0	0	0	0	0
TRCS2WTR	2	2	2	2	2	2	2	2
TRWT2DEWT	2	2	2	2	2	2	2	2

Table 8.1-46 Configuration Data for Different Banks

Burst Write/Reads are carried out on the selected banks to check the operation in the small Endian and Big Endian. The process is repeated for all the banks.

The ***Table 8.1-47*** given below illustrates the different types of accesses carried out on the Bank 0 depending upon the parameters programmed.

Bank No	HSIZE	MSIZE	BURSTLEN RD	BURSTLEN WR	ENDIANNESS	HBURST	RBL
0	8	8	4	4	Small	SIN, INCR, INC4, INC8, INC16, WRAP4,WRAP8, WRAP16	0
0	8	8	8	8	Big	SIN, INCR, INC4, INC8, INC16, WRAP4,WRAP8, WRAP16	0
0	16	8	4	4	Small	SIN, INCR, INC4, INC8, INC16, WRAP4,WRAP8, WRAP16	0
0	16	8	8	8	Big	SIN, INCR, INC4, INC8, INC16, WRAP4,WRAP8, WRAP16	0
0	32	8	4	4	Small	SIN, INCR, INC4, INC8, INC16, WRAP4,WRAP8, WRAP16	0
0	32	8	8	8	Big	SIN, INCR, INC4, INC8, INC16, WRAP4,WRAP8, WRAP16	0
0	8	16	4	4	Small	SIN, INCR, INC4, INC8, INC16, WRAP4,WRAP8, WRAP16	1
0	8	16	8	8	Big	SIN, INCR, INC4, INC8, INC16, WRAP4,WRAP8, WRAP16	1
0	16	16	4	4	Small	SIN, INCR, INC4, INC8, INC16, WRAP4,WRAP8, WRAP16	1
0	16	16	8	8	Big	SIN, INCR, INC4, INC8, INC16, WRAP4,WRAP8, WRAP16	1
0	32	16	4	4	Small	SIN, INCR, INC4, INC8, INC16, WRAP4,WRAP8, WRAP16	1
0	32	16	8	8	Big	SIN, INCR, INC4, INC8, INC16, WRAP4,WRAP8, WRAP16	1
0	8	32	4	4	Small	SIN, INCR, INC4, INC8, INC16, WRAP4,WRAP8, WRAP16	1
0	8	32	8	8	Big	SIN, INCR, INC4, INC8, INC16, WRAP4,WRAP8, WRAP16	1
0	16	32	4	4	Small	SIN, INCR, INC4, INC8, INC16,	1

						WRAP4,WRAP8, WRAP16	
0	16	32	8	8	Big	SIN, INCR, INC4, INC8, INC16, WRAP4,WRAP8, WRAP16	1
0	32	32	4	4	Small	SIN, INCR, INC4, INC8, INC16, WRAP4,WRAP8, WRAP16	1
0	32	32	8	8	Big	SIN, INCR, INC4, INC8, INC16, WRAP4,WRAP8, WRAP16	1

Table 8.1-47 Access Types Performed

The procedure is repeated for all the banks.

8.1.31 Synchronous Endianness test [SyEndianTests.c]

The Endianness tests perform the following test sequences:

- Memory configured as 32 bits. Transactions are initiated from the AHB with HSIZESMC of Byte, Half-Word and Word. These transactions may be burst or single transfers. The written data patterns are read back from memory both via the AHB interface of the TrickMem and via the SsmcCore.
- Memory configured as 16 bits. Transactions are initiated from the AHB with HSIZESMC of Byte, Half-Word and Word. These transactions may be burst or single transfers. The written data patterns are read back from memory both via the AHB interface of the TrickMem and via the SsmcCore.
- Memory configured as 8 bits. Transactions are initiated from the AHB with HSIZESMC of Byte, Half-Word and Word. These transactions may be burst or single transfers. The written data patterns are read back from memory both via the AHB interface of the TrickMem and via the SsmcCore.

The above 3 test sequences are repeated for Little and Big Endian modes of operation. Once Endianness is set, it is same for the whole system (AHB and Memory models).

Hence by toggling the endianness state in the control register, the endianness of the system will change and the proper routing of the data into correct byte lanes is validated y reading and writing the same memory location. Tests with a combination of transfers with HTRANS, HBURSTSMC and HSIZESMC are used to test the different combinations.

Program the SSMCTrCR, SSMCTrBurstWT, SSMCTrWTCNCL, SSMCTrEndian and SSMCTrExtMuxWT registers to the default values.

The different banks are configured as per the data given in the ***Table 8.1-48*** below.

Parameters	Bank 0	Bank 1	Bank 2	Bank 3	Bank 4	Bank 5	Bank 6	Bank 7
WSTIDCY	3	3	3	3	3	3	3	3
WSTRD	7	7	7	7	7	7	7	7
WSTWR	0	0	0	0	0	0	0	0
WSTOEN	0	0	0	0	0	0	0	0
WSTWEN	0	0	0	0	0	0	0	0
WSTBRD	2	1	2	3	2	2	2	2
SMBCRData Fields								

BlWriteEn	0	0	0	0	0	0	0	0
AddrValidWriteEn	1	1	1	1	1	1	1	1
BurstLenWrite	4/8	4/8	4/8	4/8	4/8	4/8	4/8	4/8
SyncWriteDev	1	1	1	1	1	1	1	1
BMWrite	1	1	1	1	1	1	1	1
BlReadEn	0	0	0	0	0	0	0	0
AddrValidReadEn	1	1	1	1	1	1	1	1
BurstLenRead	4/8	4/8	4/8	4/8	4/8	4/8	4/8	4/8
SyncReadDev	1	1	1	1	1	1	1	1
BMRead	1	1	1	1	1	1	1	1
MW	8/16/32	8/16/32	8/16/32	8/16/32	8/16/32	8/16/32	8/16/32	8/16/32
WP	0	0	0	0	0	0	0	0
WaitEn	1/0	1/0	1/0	1/0	1/0	1/0	1/0	1/0
WaitPol	0	0	0	0	0	0	0	0
RBLE	1/0	1/0	1/0	1/0	1/0	1/0	1/0	1/0
SSMCTrCS2WTRData Fields								
TRWAITEN	1/0	1/0	1/0	1/0	1/0	1/0	1/0	1/0
TRWAITPOL	0	0	0	0	0	0	0	0
TRCS2WTR	2	2	2	2	2	2	2	2
TRWT2DEWT	2	2	2	2	2	2	2	2

Table 8.1-48 Configuration Data for Different Banks

Burst Write/Reads are carried out on the selected banks to check the operation in the small Endian and Big Endian. The process is repeated for all the banks.

The ***Table 8.1-49*** given below illustrates the different types of accesses carried out on the Bank 0 depending upon the parameters programmed.

Bank No	HSIZE	MSIZE	BURSTLEN RD	BURSTLEN WR	ENDIANNESS	HBURST	RBL
0	8	8	4	4	Small	SIN, INCR, INC4, INC8, INC16, WRAP4,WRAP8, WRAP16	0
0	8	8	8	8	Big	SIN, INCR, INC4, INC8, INC16, WRAP4,WRAP8, WRAP16	0
0	16	8	4	4	Small	SIN, INCR, INC4, INC8, INC16, WRAP4,WRAP8, WRAP16	0
0	16	8	8	8	Big	SIN, INCR, INC4, INC8, INC16,	0

						WRAP4,WRAP8, WRAP16	
0	32	8	4	4	Small	SIN, INCR, INC4, INC8, INC16, WRAP4,WRAP8, WRAP16	0
0	32	8	8	8	Big	SIN, INCR, INC4, INC8, INC16, WRAP4,WRAP8, WRAP16	0
0	8	16	4	4	Small	SIN, INCR, INC4, INC8, INC16, WRAP4,WRAP8, WRAP16	1
0	8	16	8	8	Big	SIN, INCR, INC4, INC8, INC16, WRAP4,WRAP8, WRAP16	1
0	16	16	4	4	Small	SIN, INCR, INC4, INC8, INC16, WRAP4,WRAP8, WRAP16	1
0	16	16	8	8	Big	SIN, INCR, INC4, INC8, INC16, WRAP4,WRAP8, WRAP16	1
0	32	16	4	4	Small	SIN, INCR, INC4, INC8, INC16, WRAP4,WRAP8, WRAP16	1
0	32	16	8	8	Big	SIN, INCR, INC4, INC8, INC16, WRAP4,WRAP8, WRAP16	1
0	8	32	4	4	Small	SIN, INCR, INC4, INC8, INC16, WRAP4,WRAP8, WRAP16	1
0	8	32	8	8	Big	SIN, INCR, INC4, INC8, INC16, WRAP4,WRAP8, WRAP16	1
0	16	32	4	4	Small	SIN, INCR, INC4, INC8, INC16, WRAP4,WRAP8, WRAP16	1
0	16	32	8	8	Big	SIN, INCR, INC4, INC8, INC16, WRAP4,WRAP8, WRAP16	1
0	32	32	4	4	Small	SIN, INCR, INC4, INC8, INC16, WRAP4,WRAP8, WRAP16	1
0	32	32	8	8	Big	SIN, INCR, INC4, INC8, INC16, WRAP4,WRAP8, WRAP16	1

Table 8.1-49 Access Types Performed

The procedure is repeated for all the banks.

8.1.32 Asynchronous SMWAIT tests [SsmcSMWAITTests.c]

The SMWAIT signal, when asserted, is supposed to ‘hold’ the memory accesses attempted by SsmcCore. For the SsmcCore, start of an access is signalled by assertion of Chip-Select or the change in SMADDR lines. The access finishes when a counter inside the UUT (the duration of count is controlled by the values programmed into the SMBWSTRDRx/SMBWSTWRRx registers) expires.

To check this functionality, SMWAIT signal is asserted and de-asserted in various possible ways during an access. There are two registers in the TrickMem (SMCTrCEWT andSSMCTrCS2WT) which help in controlling the time of assertion and de-assertion of SMWAIT, relative to Chip-Select and Counter Expiry, respectively.

Similarly, in case of the synchronous devices the SMBURSTWAIT signal is asserted for a fixed time, in the burst mode

The tests are repeated for both read and write accesses. These set of tests are performed with active low polarity of SMWAIT for the different banks i.e. with WaitPol = 0. The external wait signal polarity is programmable and the tests for checking the opposite polarity are carried out in the SMWAITFlagTests test code.

Program the SSMCTrCR, SSMCTrBurstWT, SSMCTrWTCNCL, SSMCTrEndian and SSMCTrExtMuxWT registers to the default values.

The different banks are configured as per the data given in the ***Table 8.1-50*** below.

Parameters	Bank 0	Bank 1	Bank 2	Bank 3	Bank 4	Bank 5	Bank 6	Bank 7
WSTIDCY	9	9	9	9	9	9	9	9
WSTRD	15	15	15	15	15	15	15	15
WSTWR	15	15	15	15	15	15	15	15
WSTOEN	0	0	0	0	0	0	0	0
WSTWEN	0	0	0	0	0	0	0	0
WSTBRD	2	1	2	3	2	2	2	2
SMBCRData Fields								
BIWriteEn	0	0	0	0	0	0	0	0
AddrValidWriteEn	0	0	0	0	0	0	0	0
BurstLenWrite	4	4	4	4	4	4	4	4
SyncWriteDev	0	0	0	0	0	0	0	0
BMWrite	1	1	1	1	1	1	1	1
BIReadEn	0	0	0	0	0	0	0	0
AddrValidReadEn	0	0	0	0	0	0	0	0
BurstLenRead	4	4	4	4	4	4	4	4
SyncReadDev	0	0	0	0	0	0	0	0
BMRead	1	1	1	1	1	1	1	1
MW	32	32	32	32	16	16	8	8
WP	0	0	0	0	0	0	0	0
WaitEn	1	1	1	1	1	1	1	1
WaitPol	1	0	0	0	0	0	0	0
RBLE	1	1	1	1	1	1	0	0
SSMCTrCS2WTRData Fields								
TRWAITEN	1	1	1	1	1	1	1	1

TRWAITPOL	1	0	0	0	0	0	0	0
TRCS2WTR	2	2	2	2	2	2	2	2
TRWT2DEWT	2	2	2	2	2	2	2	2

Table 8.1-50 Configuration Data for Different Banks

Program the SMBCR register with the data calculated with the help of the data given in the table.

Program the SSMCTrCS2WTx register with the data calculated with the help of the data given in the table.

Program the SSMCTrWTCNCL register with the data given in the table and Enable the External Cancel Wait.

Perform memory Write and Read accesses to the selected bank configured with the parameter values given in the table.

Program the SSMCTrWTCNCL register to Disable the External Cancel Wait.

Poll the status register and then clear them.

The steps are then repeated for all values that can be calculated for the SMBCR register form the data available in the table below. The Configuration register of the bank 0 is programmed with the data values for the different parameters as given in the table below. Write and Read accesses are done to the memory bank. The process is repeated for all the values given in the ***Table 8.1-51*** below.

Bank N0	MSIZE	RBLE	WAIT Enable	Wait Polarity	Wait Ignore	CS2WT Count	WT2DEWT Count	WTCNCL Count
0	32	1	EN	0	1	3	3	63
0	32	1	EN	0	1	3	5	63
0	32	1	EN	1	1	3	10	63
0	32	1	EN	1	1	3	21	63
0	32	1	EN	0	1	5	3	63
0	32	1	EN	0	1	5	5	63
0	32	1	EN	1	1	5	10	63
0	32	1	EN	1	1	5	21	63
0	32	1	EN	0	1	10	3	63
0	32	1	EN	0	1	10	5	63
0	32	1	EN	1	1	10	10	63
0	32	1	EN	1	1	10	21	63
0	16	1	EN	0	1	3	3	63
0	16	1	EN	0	1	3	5	63
0	16	1	EN	1	1	3	10	63
0	16	1	EN	1	1	3	21	63
0	16	1	EN	0	1	5	3	63

0	16	1	EN	0	1	5	5	63
0	16	1	EN	1	1	5	10	63
0	16	1	EN	1	1	5	21	63
0	16	1	EN	0	1	10	3	63
0	16	1	EN	0	1	10	5	63
0	16	1	EN	1	1	10	10	63
0	16	1	EN	1	1	10	21	63
0	8	0	EN	0	1	3	3	63
0	8	0	EN	0	1	3	5	63
0	8	0	EN	1	1	3	10	63
0	8	0	EN	1	1	3	21	63
0	8	0	EN	0	1	5	3	63
0	8	0	EN	0	1	5	5	63
0	8	0	EN	1	1	5	10	63
0	8	0	EN	1	1	5	21	63
0	8	0	EN	0	1	10	3	63
0	8	0	EN	0	1	10	5	63
0	8	0	EN	1	1	10	10	63
0	8	0	EN	1	1	10	21	63

Table 8.1-51 Access Types Performed

The process is then repeated for all the remaining banks.

8.1.33 Synchronous Burst Wait Tests [SyBurstWAITTests.c]

In case of the synchronous devices the nSMBURSTWAIT signal, when asserted, is supposed to ‘hold’ the memory accesses attempted by SsmcCore. For the SsmcCore, start of an access is signalled by assertion of Chip-Select or the change in SMADDR lines. The access finishes when a counter inside the UUT (the duration of count is controlled by the values programmed into the SMBWSTRDRx/SMBWSTWRRx registers) expires. In case of the synchronous devices the SMBURSTWAIT signal is asserted for a fixed time, in the burst mode

To check this functionality, nSMBURSTWAIT signal is asserted and de-asserted in various possible ways during an access. There are two registers in the TrickMem (SMCTrCEWT andSSMCTrCS2WT) which help in controlling the time of assertion and de-assertion of nSMBURSTWAIT, relative to Chip-Select and Counter Expiry, respectively.

The tests are repeated for both read and write accesses. These set of tests are performed with active low polarity of SMWAIT for the different banks i.e. with WaitPol = 0. The external wait signal polarity is programmable and the tests for checking the opposite polarity are carried out in the SMWAITFlagTests test code.

Program the SSMCTrCR, SSMCTrBurstWT, SSMCTrWTCNCL, SSMCTrEndian and SSMCTrExtMuxWT registers to the default values.

The different banks are configured as per the data given in the ***Table 8.1-52*** below.

Parameters	Bank 0	Bank 1	Bank 2	Bank 3	Bank 4	Bank 5	Bank 6	Bank 7
WSTIDCY	2	9	2	2	2	2	2	2
WSTRD	0	1	2	3	4	5	6	7
WSTWR	0	1	2	3	4	5	6	7
WSTOEN	0	0	0	0	0	0	0	0
WSTWEN	0	0	0	0	0	0	0	0
WSTBRD	0	1	2	3	2	2	2	2
SMBCRData Fields								
BIWriteEn	0	0	0	0	0	0	0	0
AddrValidWriteEn	1	1	1	1	1	1	1	1
BurstLenWrite	4	4	4	4	4	4	4	4
SyncWriteDev	1	1	1	1	1	1	1	1
BMWrite	1	1	1	1	1	1	1	1
BIReadEn	0	0	0	0	0	0	0	0
AddrValidReadEn	1	1	1	1	1	1	1	1
BurstLenRead	4	4	4	4	4	4	4	4
SyncReadDev	1	1	1	1	1	1	1	1
BMRead	1	1	1	1	1	1	1	1
MW	8-16-32	8-16-32	8-16-32	8-16-32	8-16-32	8-16-32	8-16-32	8-16-32
WP	0	0	0	0	0	0	0	0
WaitEn	1	1	1	1	1	1	1	1
WaitPol	0	0	1	1	1	1	0	0
RBLE	1	1	0	1	1	0	1	1
SSMCTrCS2WTRData Fields								
TRWAITEN	0	1	0	1	0	1	0	1
TRWAITPOL	0	0	1	1	1	1	0	0
TRCS2WTR	2	2	2	2	2	2	2	2
TRWT2DEWT	2	2	2	2	2	2	2	2

Table 8.1-52 Configuration Data for Different Banks

The Configuration register of the bank 0 is programmed with the data values for the different parameters as given in the table below. Write and Read accesses are done to the memory bank. The process is repeated for all the values given in the ***Table 8.1-53*** below.

Bank N0	HSIZE	MSIZE	BLENRD	BLENWR	HBURST	BwtMask	BeatNo	BWTCount	RBLE
0	8	8	4	4	INCR4	DI	Beat0	15	0
0	8	8	4	4	INCR4	DI	Beat1	15	0
0	8	8	4	4	INCR4	DI	Beat2	14	0
0	8	8	4	4	INCR4	DI	Beat3	13	0
0	8	8	8	8	INCR4	DI	Beat0	12	0
0	8	8	8	8	INCR4	DI	Beat1	12	0
0	8	8	8	8	INCR4	DI	Beat2	11	0
0	8	8	8	8	INCR4	DI	Beat3	10	0
0	8	8	8	8	INCR8	DI	Beat4	9	0
0	8	8	8	8	INCR8	DI	Beat5	8	0
0	8	8	8	8	INCR8	DI	Beat6	7	0
0	8	8	8	8	INCR8	DI	Beat7	6	0
0	8	8	8	8	INCR16	DI	Beat3	5	0
0	8	8	8	8	INCR16	DI	Beat5	4	0
0	8	8	8	8	INCR16	DI	Beat7	3	0
0	8	8	Cont	Cont	INCR4	DI	Beat0	2	0
0	8	8	Cont	Cont	INCR4	DI	Beat1	2	0
0	8	8	Cont	Cont	INCR4	DI	Beat2	1	0
0	8	8	Cont	Cont	INCR8	DI	Beat3	1	0
0	8	8	Cont	Cont	INCR8	DI	Beat4	2	0
0	8	8	Cont	Cont	INCR16	DI	Beat5	3	0
0	8	8	Cont	Cont	INCR16	DI	Beat6	4	0
0	16	16	4	4	INCR4	DI	Beat0	15	0
0	16	16	4	4	INCR4	DI	Beat1	15	0
0	16	16	4	4	INCR4	DI	Beat2	14	0
0	16	16	4	4	INCR4	DI	Beat3	13	0
0	16	16	8	8	INCR4	DI	Beat0	12	0
0	16	16	8	8	INCR4	DI	Beat1	12	0
0	16	16	8	8	INCR4	DI	Beat2	11	0
0	16	16	8	8	INCR4	DI	Beat3	10	0

0	16	16	8	8	INCR8	DI	Beat4	9	0
0	16	16	8	8	INCR8	DI	Beat5	8	0
0	16	16	8	8	INCR8	DI	Beat6	7	0
0	16	16	8	8	INCR8	DI	Beat7	6	0
0	16	16	8	8	INCR16	DI	Beat3	5	0
0	16	16	8	8	INCR16	DI	Beat5	4	0
0	16	16	8	8	INCR16	DI	Beat7	3	0
0	16	16	Cont	Cont	INCR4	DI	Beat0	2	0
0	16	16	Cont	Cont	INCR4	DI	Beat1	2	0
0	16	16	Cont	Cont	INCR4	DI	Beat2	1	0
0	16	16	Cont	Cont	INCR8	DI	Beat3	1	0
0	16	16	Cont	Cont	INCR8	DI	Beat4	2	0
0	16	16	Cont	Cont	INCR16	DI	Beat5	3	0
0	16	16	Cont	Cont	INCR16	DI	Beat6	4	0
0	32	32	4	4	INCR4	DI	Beat0	15	0
0	32	32	4	4	INCR4	DI	Beat1	15	0
0	32	32	4	4	INCR4	DI	Beat2	14	0
0	32	32	4	4	INCR4	DI	Beat3	13	0
0	32	32	8	8	INCR4	DI	Beat0	12	0
0	32	32	8	8	INCR4	DI	Beat1	12	0
0	32	32	8	8	INCR4	DI	Beat2	11	0
0	32	32	8	8	INCR4	DI	Beat3	10	0
0	32	32	8	8	INCR8	DI	Beat4	9	0
0	32	32	8	8	INCR8	DI	Beat5	8	0
0	32	32	8	8	INCR8	DI	Beat6	7	0
0	32	32	8	8	INCR8	DI	Beat7	6	0
0	32	32	8	8	INCR16	DI	Beat3	5	0
0	32	32	8	8	INCR16	DI	Beat5	4	0
0	32	32	8	8	INCR16	DI	Beat7	3	0
0	32	32	Cont	Cont	INCR4	DI	Beat0	2	0
0	32	32	Cont	Cont	INCR4	DI	Beat1	2	0

0	32	32	Cont	Cont	INCR4	DI	Beat2	1	0
0	32	32	Cont	Cont	INCR8	DI	Beat3	1	0
0	32	32	Cont	Cont	INCR8	DI	Beat4	2	0
0	32	32	Cont	Cont	INCR16	DI	Beat5	3	0
0	32	32	Cont	Cont	INCR16	DI	Beat6	4	0

Table 8.1-53 Access Types Performed

The process is then repeated for all the remaining banks.

8.1.34 Transfer Sequences Tests [SsmcTranSeqTests.c]

In this test, banks 1, 3, 5, 7 are configured as asynchronous banks while the banks 0, 2, 4, 6 are configured as synchronous banks with the parameter values in the ***Table 8.1-54*** below.

Parameters	Bank 0	Bank 1	Bank 2	Bank 3	Bank 4	Bank 5	Bank 6	Bank 7
WSTIDCY	1	1	1	1	1	1	1	1
WSTRD	2	2	0	2	2	0	2	2
WSTWR	3	3	0	3	3	0	3	3
WSTOEN	1	1	0	1	1	0	1	1
WSTWEN	1	1	0	1	1	0	1	1
WSTBRD	2	2	2	2	2	2	2	2
BlWriteEn	0	0	0	0	0	0	0	0
AddrValidWriteEn	0	0	1	0	1	0	1	0
BurstLenWrite	4	4	4	4	4	4	4	4
SMBCRData Fields								
SyncWriteDev	0	0	1	0	1	0	1	0
BMWrite	1	1	1	1	1	1	1	1
BlReadEn	0	0	0	0	0	0	0	0
AddrValidReadEn	1	0	1	0	1	0	1	0
BurstLenRead	4	4	4	4	4	4	4	4
SyncReadDev	1	0	1	0	1	0	1	0
BMRead	1	1	1	1	1	1	1	1
MW	32	16	32	32	16	32	32	32
WP	0	0	0	0	0	0	0	0
WaitEn	0	0	0	0	0	1	0	0

WaitPol	0	0	1	1	1	1	0	0
RBLE	1	1	1	1	1	1	1	1
SSMCTrCS2WTRData Fields								
TRWAITEN	0	0	0	0	0	1	0	0
TRWAITPOL	0	0	1	1	1	1	0	0
TRCS2WTR	2	2	2	2	2	2	2	2
TRWT2DEWT	2	2	2	2	2	2	2	2

Table 8.1-54 Configuration Data for Different Banks

Different transfer sequences are carried out on the different banks as listed below.

- Asynchronous Writes followed by Synchronous Reads
- Synchronous Burst Writes followed by Asynchronous writes with SMWAIT enabled and WSTWR = 0
- Wait enabled Asynchronous writes followed by Synchronous reads with WSTRD = 0
- Wait enabled Asynchronous writes followed by Synchronous writes with WSTWR = 0
- Asynchronous reads followed by Synchronous reads with WSTRD = 0
- Asynchronous reads followed by Synchronous writes with WSTWR = 0
- Asynchronous writes followed by Synchronous reads with WSTRD = 0
- Synchronous Writes followed by WAIT enabled Asynchronous writes
- Synchronous Writes followed by asynchronous reads
- Synchronous reads followed by Asynchronous write within the same bank.

8.1.35 Specified length burst test [TransferTests.c]

This test performs read / write operations with all combinations of

- HSIZESMC with values of byte, half word, word,
- Memory size with values of 8-bit, 16-bit, 32-bit,
- Different bursts combinations,
- Bank values of zero to seven.

The SsmcCore does not support SPLIT or RETRY responses. For each transfer, the slave buswatcher of the top-level testbench continuously monitors the slave response. Any deviation from the expected response will cause the error message to be flagged. Different tests use different values of read and write access values. Different test combination cases are tabulated in ***Table 8.1-55*** below.

Transfer Sequences
NONSEQ
NONSEQ – NONSEQ to same bank
NONSEQ – NONSEQ to different banks
NONSEQ – NONSEQ - NONSEQ to same bank

NONSEQ – NONSEQ - NONSEQ to different banks
NONSEQ – SEQ – SEQ
NONSEQ - IDLE - NONSEQ to same bank
NONSEQ - IDLE - NONSEQ to different banks
NONSEQ – BUSY – SEQ
NONSEQ – BUSY - NONSEQ to same bank
NONSEQ – BUSY – NONSEQ to different banks
NONSEQ – BUSY – IDLE
NONSEQ - SEQ – BUSY – SEQ
NONSEQ – BUSY – SEQ - BUSY – SEQ
NONSEQ – BUSY – SEQ – BUSY
NONSEQ – BUSY - BUSY – SEQ
NONSEQ - SEQ - SEQ – SEQ – SEQ - SEQ – SEQ – SEQ
NSR-NSW-NSR to same and different banks
NSW-NSR-NSW to same and different banks
NSR-SR-NSW to same and different banks
NSW-SW-NSR to same and different banks

Table 8.1-55 Different Transfer Sequences

In the above table, the following acronyms have been used:

NSR: NONSEQ Read

NSW: NONSEQ Write

SR: SEQ Read

SW: SEQ Write

The tests listed in the ***Table 8.1-55*** will be performed with the following different delays.

- Zero enable and wait delays
- One enable delay
- Two enable delays
- One wait state
- Two wait states
- One enable delay and two wait states
- Two enable delays and one wait state
- Non-zero turnaround cycles between transfers

All the transfers are verified by reading back the data from the AHB side of the Memory model. This ensures that the external address, control and data lines are properly driven out by the SSMC.

Program the SSMCTrCR, SSMCTrBurstWT, SSMCTrWTCNCL, SSMCTrEndian and SSMCTrExtMuxWT registers to the default values.

The different banks are configured as per the data given in the ***Table 8.1-56*** below.

Parameters	Bank 0	Bank 1	Bank 2	Bank 3	Bank 4	Bank 5	Bank 6	Bank 7
WSTIDCY	2	2	2	2	2	3	4	2
WSTRD	5	5	5	5	2	7	6	5
WSTWR	4	4	4	4	4	0	2	1
WSTOEN	2	2	2	2	2	3	4	4
WSTWEN	0	0	0	0	0	0	0	0
WSTBRD	2	2	2	2	2	2	2	2
BlWriteEn	0	0	0	0	0	0	0	0
AddrValidWriteEn	0	0	0	0	0	0	0	0
BurstLenWrite	4	4	4	4	4	4	4	4
SMBCRData Fields								
SyncWriteDev	0	0	0	0	0	0	0	0
BMWrite	1	1	1	1	1	1	1	1
BlReadEn	0	0	0	0	0	0	0	0
AddrValidReadEn	0	0	0	0	0	0	0	0
BurstLenRead	4	4	4	4	4	4	4	4
SyncReadDev	0	0	0	0	0	0	0	0
BMRead	1	1	1	1	1	1	1	1
MW	32	32	32	32	32	32	32	32
WP	0	0	0	0	0	0	0	0
WaitEn	0	0	0	0	0	0	0	0
WaitPol	0	0	1	1	1	1	0	0
RBLE	1	1	1	1	1	1	1	1
SSMCTrCS2WTRData Fields								
TRWAITEN	0	0	0	0	0	0	0	0
TRWAITPOL	0	0	1	1	1	1	0	0
TRCS2WTR	2	2	2	2	2	2	2	2
TRWT2DEWT	2	2	2	2	2	2	2	2

Table 8.1-56 Configuration Data for Different Banks

Different types of Write/Read accesses are performed as shown in the ***Table 8.1-57*** below.

Banks	HSIZE	MSIZE	HBURST	OPERATION

Bank 1	WORD	WORD	SINGLE	NS-NS to same bank
Banks 2 & 3	WORD	WORD	SINGLE	NS-NS to different banks
Bank 4	WORD	WORD	SINGLE	NS-NS-NS to same bank
Banks 5, 6 & 7	WORD	WORD	SINGLE	NS-NS-NS to different banks
Banks 2	WORD	WORD	INCR	N-S-S-S to same bank
Bank4	WORD	WORD	SINGLE	NS-I-NS to same bank
Banks 3, 5 & 6	WORD	WORD	SINGLE	NS-I-NS to different banks
Bank 7	WORD	WORD	INCR	NS-B-S to same bank
Bank 4	WORD	WORD	INCR	NS-B-NS to same bank
Bank 5 & 7	WORD	WORD	INCR	NS-B-NS to different banks
Bank 4	WORD	WORD	INCR	NS-B-I to same bank
Bank 4	WORD	WORD	INCR	NS-S-B-S to same bank
Bank 4	WORD	WORD	INCR	NS-B-S-B-S to same bank
Bank 4	WORD	WORD	INCR4	NS-S-B-S to same bank with WSTBRD = 0
Bank 4	WORD	WORD	INCR4	NS-S-B-B-B-B-B-S-S to same bank
Bank 4	WORD	WORD	INCR4	NS-S-S-B-B-B-B-B-B-S to same bank
Bank 4	WORD	WORD	INCR4	NS –B-B-B-B-B-B-S-S-S to same bank
Bank 2 & 3	BYTE	WORD & HALFWORD	INCR	NS-I-NS 32 BIT BANK FOLLOWED BY 16 BIT BANK ACCESS
Bank 2 & 6	WORD & BYTE	BYTE & HALFWORD	INCR	NS-I-NS 8 BIT BANK TO 32 BIT BANK
Bank 3 & 6	HALFWORD & BYTE	Byte	INCR	NS-I-NS 16 BIT BANK TO 8 BIT BANK
Bank 3 & 6	Byte & WORD	BYTE & HALFWORD	INCR	NS-I-NS 8 BIT BANK TO 16 BIT BANK
Bank 2 & 6	WORD	WORD & BYTE	SINGLE	NS-I-NS 32 BIT BANK TO 8 BIT BANK

Table 8.1-57 Access Types Performed

8.1.36 Denali Memory Model based Tests [DenaliTests.c, DenaliTests1.c, DenaliTests2.c]

The Denali memory models verify the SsmcCore performance, which emulates real world memory device. Transfers are done to the memory with different values of HBURSTSMC, HSIZESMC and HTRANS. Write protection tests are done by writing to ROM models. Corresponding to every transfer, the status registers (SMBSRx) are read for the updated status of the device. Denali models are also tested with the clock running always and clock active only during memory accesses to save power.

The memory models used for the simulation are given in ***Table 8.1-58*** below. The SPC files mentioned are in the *verification/denali* directory.

Sr. No.	SOMA File name	Datasheet used
1.	sram1.spc	KM681002A-15, 128K x 8 High speed SRAM (Samsung).
3.	sram2.spc	K6R1016C1C-12, 64K x 16 bit High speed CMOS SRAM (Samsung).
4.	sram3.spc	K6R1016C1C-20, 64K x 16 bit High speed CMOS SRAM (Samsung).
5.	sram4.spc	KM681002A-20, 128K x 8 High speed SRAM (Samsung).
6.	flash1.spc	28F128J3A Intel Strata Flash Memory (Intel).
7.	flash2.spc	28F800C3 Advanced+ Boot Block Flash Memory (Intel).
8.	mrom1.spc	K3P5V(U) 2000D-SC 512K x 32 bit CMOS Mask ROM (Samsung).
9.	mrom2.spc	K3P6C2000B-SC15 1M x 32 bit CMOS Mask ROM (Samsung).
10.	28f6408w30b70.spc	28F6408W30, 16 bit Wireless Flash Memory (Intel)

Table 8.1-58 Details of datasheets used for Denali Simulation.

8.2 Verification Test Vector Generation

A ***make sourcefilename*** (without the .c extension) in the ***verification/bustest*** directory will create **.bif** formatted and **.sim**-formatted vectors in the ***verification*/*bustest/invec*** directory. The makefile in ***verification*/*bustest*** directory can be referred to for the make options.

9 INTEGRATION TESTBENCH

The integration testbench is based on an AHB TICTalk testbench. The VHDL and Verilog TICTalk testbench files reside in the ***integration/vhdl*** and ***integration/verilog*** directories respectively. This testbench is used for testing the SSMC's TIC module as well as for running the integration tests of the SSMC (PL093). For running the integration tests, the SSMC's TIC is used.

9.1 VHDL Integration testbench

The AMBA TICTalk VHDL testbench used for integration testing and for testing of the SSMC's TIC is located in the ***integration/vhdl/tbench*** directory. The Integration test vectors are located in the ***integration/bustest*** directory.

Figure 9.1-1 illustrates the VHDL testbench to apply TICTalk test vectors.

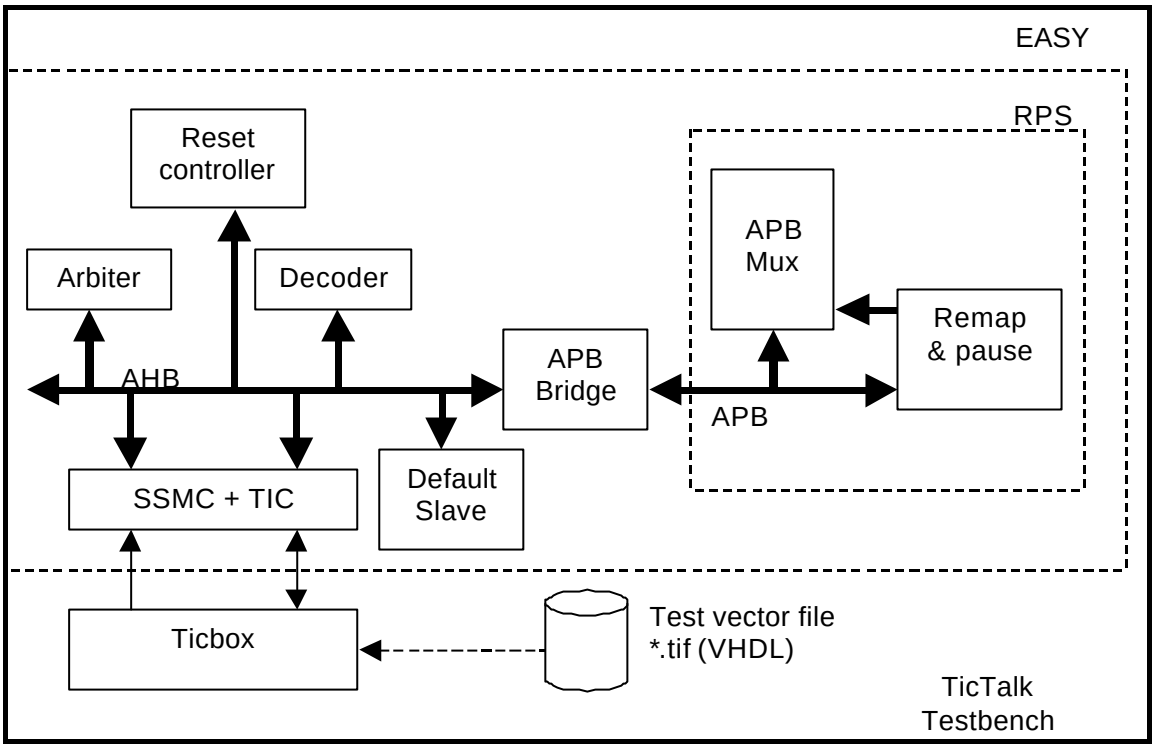


Figure 9.1-1 VHDL Testbench for simulating TICTalk test vectors

The VHDL TicTalk testbench is a stripped down version of the standard EASY system testbench.

It has decoder which is used to provide a select signal HSELx for each slave on the bus, arbiter which ensures that only one master has access to the bus, default slave which provides response signals when non-existence memory address is accessed, reset controller which generates reset.

For further information refer to the Example AMBA System user guide which is part of ARM Micropack documentation.

Figure 9.1-2 illustrates the hierarchy for the TICTalk VHDL testbench.

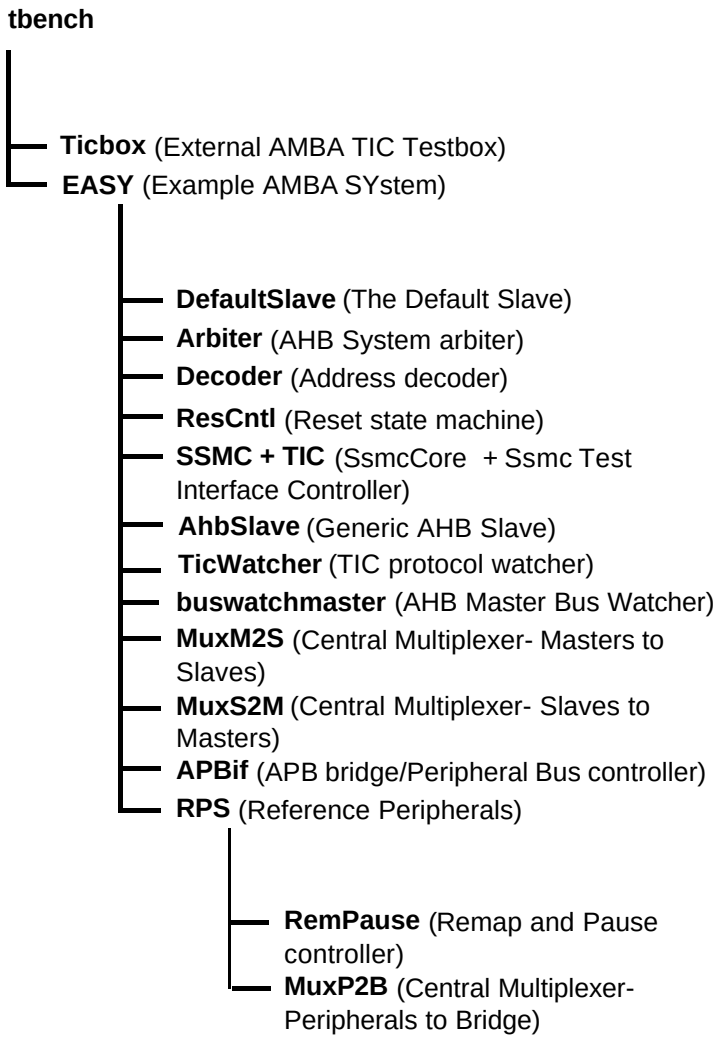


Figure 9.1-2 VHDL Testbench hierarchy for simulating TICTalk test vectors

The testing of the TIC module is done in the TicTalk world. The Ticbox module in the TicTalk world drives the TIC vectors onto the TIC module. The Ticbox reads the infile.tif in the ***integration/bustest/invec*** directory and generates TIC vectors. A generic AHB slave is used for the testing of the SsmcTIC module functionality. The TIC generates AHB transfers to the generic AHB slave. Different types of transfers are initiated and the behaviour of the TIC is tested. TICWatcher module checks for the data integrity and protocol checks on the TIC generated AHB signals.

9.1.1 Bus Master Protocol Checker

The EASY instantiates the master bus watcher, buswatchmaster.vhd that checks the protocol of TIC master accesses initiated via the test code. This module implements the protocol checks for the AHB master's output signals and will flag warning and error messages if any timing or protocol violations are detected during simulations.

9.1.2 TIC Protocol Checker

The TIC Watcher module instantiated in the EASY is a TIC data integrity/protocol checker. This module mirrors the logic of the TIC and generates the AHB signals internally. The internally generated AHB signals are compared with that from the TIC module. It flags error messages whenever a protocol violation is detected.

9.1.3 Generic AHB Slave

The Generic AHB Slave is instantiated in the EASY. Using a generic AHB Slave, which will respond to the AHB transfers generated by the TIC, validates the TIC. By tracking the responses from the generic slave on each transfer, protocol and timing checks are done. The Generic AHB Slave is a Split-capable slave device that aids the validation of the TIC. In addition to the bus interface logic, it contains programmable registers and data arrays that can be used to control the way in which each AHB transfer is responded to.

9.1.4 Test Vectors

The test vectors for the verification of the TIC are coded into separate C files, in the ***integration/bustest*** directory, depending on the logical region of the TIC, which these vectors are testing. Make options have been provided to use the entire test vectors together or to use them individually.

9.1.5 Running Simulations

The **README** file in the ***integration/vhdl/tbench*** directory gives step-by-step instructions for running simulations using tests on the VHDL source code.

Run time logs for all the combinations are available in the ***integration/vhdl/Ssmc_rtl*** directory.

9.2 Verilog Integration testbench

The AMBA TICTalk Verilog testbench used for testing of the TIC is located in the ***integration/verilog/tbench*** directory. The Integration test vectors are located in the ***integration/bustest*** directory.

Figure 9.2-1 illustrates the Verilog testbench to apply TICTalk test vectors.

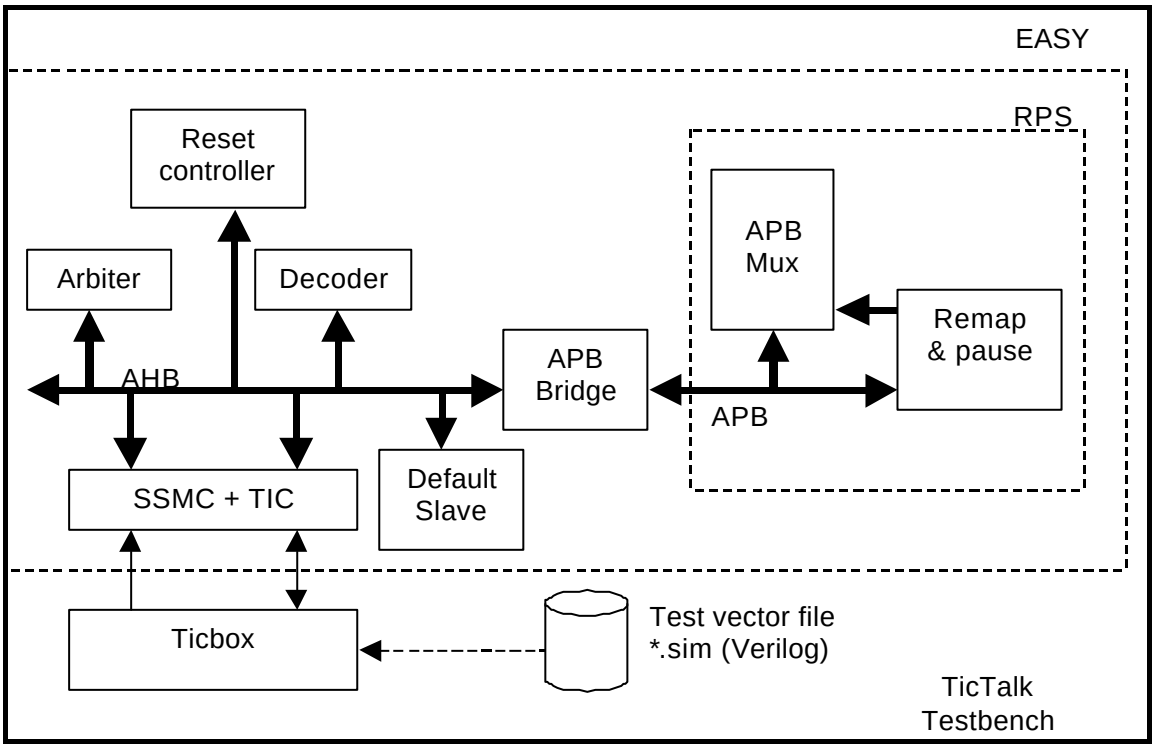


Figure 9.2-1 Verilog Testbench for simulating TICTalk test vectors

The Verilog TicTalk testbench is a stripped down version of the standard EASY system testbench.

It has decoder which is used to provide a select signal HSELx for each slave on the bus, arbiter which ensures that only one master has access to the bus, default slave which provides response signals when non-existence memory address is accessed and reset controller which generates reset for the system.

For further information refer to the Example AMBA System user guide which is part of ARM Micropack documentation.

Figure 9.2-2 illustrates the hierarchy for the TICTalk Verilog testbench.

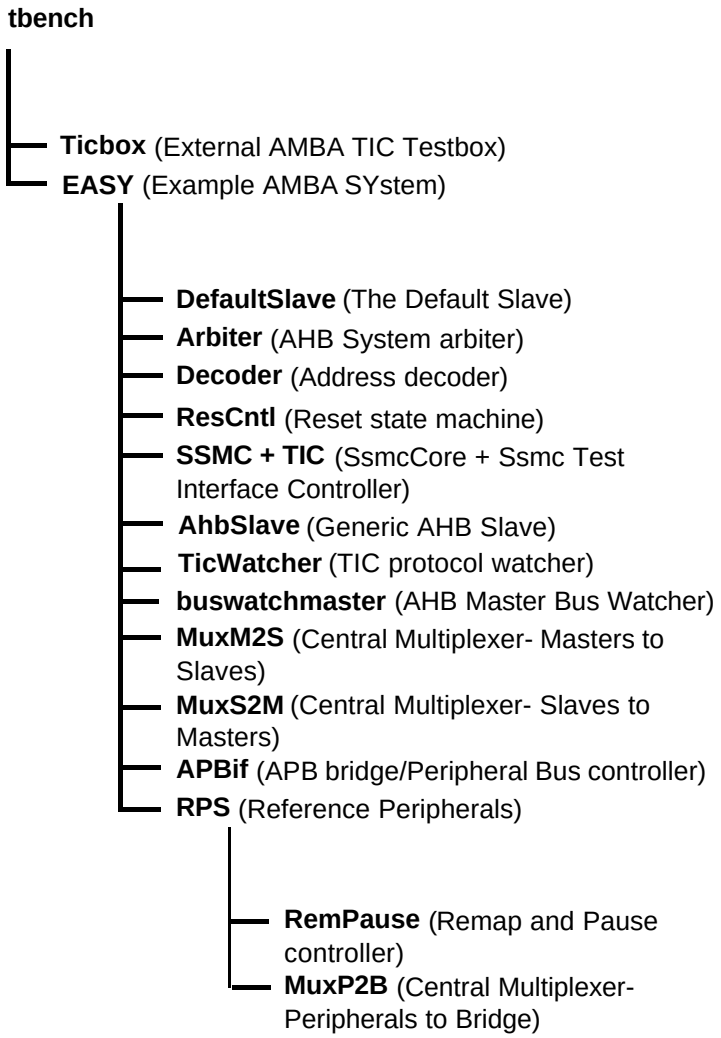


Figure 9.2-2 Verilog Testbench hierarchy for simulating TICTalk test vectors

The testing of the TIC module is done in the TicTalk world. The Ticbox module in the TicTalk world drives the TIC vectors onto the TIC module. The Ticbox reads the tif.sim in the ***integration/bustest/invec*** directory and generates TIC vectors. A generic AHB slave is used for the testing of the SsmcTIC module of the SSMC. The TIC generates AHB transfers to the generic AHB slave. Different types of transfers are initiated and the behaviour of the TIC is tested. TICWatcher module checks for the data integrity and protocol checks on the TIC generated AHB signals.

9.2.1 Bus Master Protocol Checker

The TicTalk testbench instantiates the master bus watcher, buswatchmaster.v that checks the protocol of TIC master accesses initiated via the test code. This module implements the protocol checks for the AHB master's output signals and will flag warning and error messages if any timing or protocol violations are detected during simulations.

9.2.2 TIC Protocol Checker

The TIC Watcher module instantiated in the EASY is a TIC data integrity/protocol checker. This module mirrors the logic of the TIC and generates the AHB signals internally. The internally generated AHB signals are compared with that from the TIC module. It flags error messages whenever a protocol violation is detected.

9.2.3 Generic AHB Slave

The Generic AHB Slave is instantiated in the EASY. Using a generic AHB Slave, which will respond to the AHB transfers generated by the TIC, validates the TIC. By tracking the responses from the generic slave on each transfer, protocol and timing checks are done. The Generic AHB Slave is a Split-capable slave device that aids the validation of the TIC. In addition to the bus interface logic, it contains programmable registers and data arrays that can be used to control the way in which each AHB transfer is responded to.

9.2.4 Test Vectors

The test vectors for the verification of the TIC are coded into separate C files, in the ***integration/bustest*** directory, depending on the logical region of the TIC, which these vectors are testing. Make options have been provided to use the entire test vectors together or to use them individually.

9.2.5 Running Simulations

The **README** file in the ***integration/verilog/tbench*** directory gives step by step instructions for running simulations using tests on the Verilog source code.

Run time logs for all the combinations are available in the ***integration/verilog/Ssmc_rtl*** directory.

9.3 Trickbox Description

9.3.1 Generic AHB Slave

Using a generic AHB Slave, which will respond to the AHB transfers generated by the TIC, validates the TIC. By tracking the responses from the generic slave on each transfer, protocol and timing checks are done. The Generic AHB Slave is a Split-capable slave device that aids the validation of the TIC. In addition to the bus interface logic, it contains programmable registers and data arrays that can be used to control the way in which each AHB transfer is responded to.

The block diagram of the Generic AHB Slave is shown in ***Figure 9.3-1***.

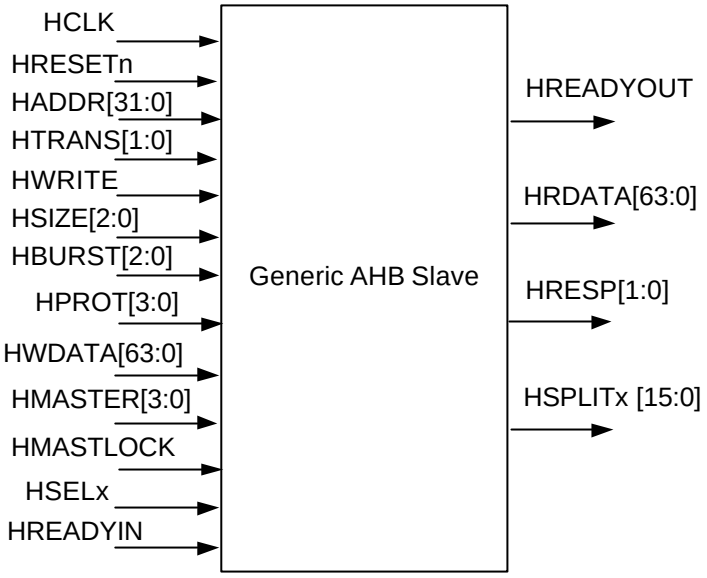


Figure 9.3-1 Generic AHB Slave Block diagram

Generic Slave Specifications

Functional Mode Registers:

Address	Type	Width	Reset Value	Name	Description
Base Address + 0x00	R/W	32	0x0000	WSC1	Wait State Register 1 defines the wait states to be introduced while accessing memory array 1
Base Address + 0x04	R/W	32	0x0000	WSC2	Wait State Register 2 defines the wait states to be introduced while accessing memory array 2
Base Address + 0x08	R/W	32	0x0000	CR	Control Register defines DELAYEN bits for all output signals and PROTECTION bits for HRESP and HRDATA. Also defines the endianness of the system.
Base Address + 0x0C	R/W	32	0x0000	TMR	Timeout register defines the number of clock cycles after which the transfer times out in a SPLIT/RETRY transfer.
Base Address + 0x10	R/W	32	0x0000	XR	Transfer delay register determines when the slave has to respond to an access to a RETRY/SPLIT space with an OK response

Table 9.3-1 Functional Mode Registers

Memory Arrays:

Address	Type	Width	Reset Value	Name	Description
Base Address + (0x100 – 0x4FF)	R/W	64	0x00	ARRAY1	Memory Array 1 to store data to check read and write operations
Base Address + (0x500 – 0x8FF)	R/W	64	0x00	ARRAY2	Memory Array 2 to store data to check read and write operations

Table 9.3-2 Memory Arrays

Generic Slave Description

The AHB slave contains two register arrays and five configuration registers as per the following description. In the following all addresses are word addresses:

Memory Array

This is an array of 128 DWORDs (64 bits) which is accessible for reads or writes over the AHB. Aligned byte / half-word, word and dword accesses to this array of 1KB are supported with wait states controlled on a per-beat basis as per the Array Wait State Control Register. The array is divided into four parts such that accessing each part will give a different type of HRESP response.

Wait State Control Register

Any R/W access to a particular array location will introduce that many wait states as is programmed in the wait State register. The 32 bit wait State register is split into 8 fields with each field denoting the number of wait states to be introduced during an access to each beat of a burst transfer to that particular array. It is intended to do read/write operation to the register with zero wait states.

For a 4 word burst

Bits 7:0 = wait states in beat 0

Bits 15:8 = wait states in beat 1

Bits 23:16 = wait states in beat 2

Bits 31:24 = wait states in beat 3

For an 8 word burst

Bits 3:0 = wait states in beat 0

Bits 7:4 = wait states in beat 1

Bits 11:8 = wait states in beat 2

Bits 15:12 = wait states in beat 3

Bits 19:16 = wait states in beat 4

Bits 23:20 = wait states in beat 5

Bits 27:24 = wait states in beat 6

Bits 31:27 = wait states in beat 7

Control Register

The Control register contains 32 bits of which 6 are used and the others are reserved. Each of the response signals generated depends upon the two programmable bits in the generic slave control register. These two bits helps to find out whether the TIC is capable enough to detect the error in the responses given out by the generic slave.

Bit [0]: BIGENDIAN – This bit will determine whether the Slave will behave as little endian or big endian.

Bit [1]: ENDIANEN – When this bit is set, Slave will behave as a big endian system, driving the data only in the required byte lanes.

Bit[2]: ForceErrResp – When this bit is set, will force the slave to drive the ERROR response. If not set then slave is intended to behave normally.

Bit[3]: ForceDataErr – When this bit is set, will force the slave to drive the complement of expected data on HRDATA bus.

Bit [4]: RESPEN – This bit, if not set, will force the slave to drive the OK response. Otherwise, if set, slave is intended to behave normally.

Bit [5]: DELAYEN – If this bit is enabled, all the output signals are delayed for the default value programmed to check the timing protocols of the signal. This will help in checking the timing protocols of the signals.

Timeout Register

The 32-bit register determines when the slave will timeout for a retry/split transfer. After timeout, the slave won't proceed with the ongoing split/retry transfer.

Transfer Delay Register

The first 16 bits will determine on which re-issue of the SPLIT transfer, the generic slave has to respond with an OK response. The higher order 16 bits will determine the number of re-issues required in a retry transfer.

Transfer Modes

The generic slave supports almost all modes in AHB Slave architecture. The modes supported are listed below.

HTRANS[1:0]	All combinations are supported
HSIZE[2:0]	All transfers upto DWORD are supported
HBURST[2:0]	All combinations are supported
HRESP[1:0]	All combinations are supported
HPROT[3:0]	Not supported

9.3.2 TIC Watcher Module

The TIC Watcher module is a TIC data integrity/protocol checker. This module mirrors the logic of the TIC and generates the AHB signals internally. The internally generated AHB signals are compared with that from the TIC module. It flags error messages whenever a protocol violation is detected.

TIC Watcher Signal Description

The following table summarises the TIC Watcher signal list.

Signal Name	Type	Source / Destination	Description

Signal Name	Type	Source / Destination	Description
TCLK	IN	Clock Generator	Test mode clock input. This clock times all TIC bus transfers. All signal timings are referenced to the rising edge of TCLK.
HRESETn	IN	Reset Controller	Bus reset (active low).
TESTREQA	IN	TicBox (External tester)	Test bus request A
TESTREQB	IN	TicBox (External tester)	Test bus request B
TESTACK	IN	TicBox (External tester)	Test Acknowledge
TESTBUS[31:0]	IN	TicBox (External tester)	Test data bus
HADDROUT[31:0]	IN	TIC	The 32-bit system address bus.
HTRANSOUT[1:0]	IN	TIC	Indicates the type of the current transfer, which can be Non-Sequential, Sequential or Idle. The TIC does not use the Busy transfer type.
HWRITEOUT	IN	TIC	When HIGH this signal indicates a write transfer and when LOW a read transfer.
HSIZEOUT[2:0]	IN	TIC	Indicates the size of the transfer, which is typically byte (8-bit), halfword (16-bit) or word (32-bit). The TIC does not support larger transfer sizes.
HBURSTOUT[2:0]	IN	TIC	Indicates if the transfer forms part of a burst. The TIC always performs incrementing bursts of unspecified length.
HPROTOUT[3:0]	IN	TIC	The Protection control signals indicate if the transfer is an opcode fetch or data access, as well as if the transfer is a supervisor mode access or user mode access. These signals can also indicate whether the current access is cacheable or bufferable.
HWDATAOUT[31:0]	IN	TIC	The write data bus is used to transfer data from the master to bus slaves during write operations. A minimum data bus width of 32 bits is recommended, however this may easily be extended to allow for higher bandwidth operation.

Table 9.3-3: The TIC Watcher Signal Description

TIC Watcher Functional Description

The TIC Watcher block diagram is shown in ***Figure 9.3-4***.

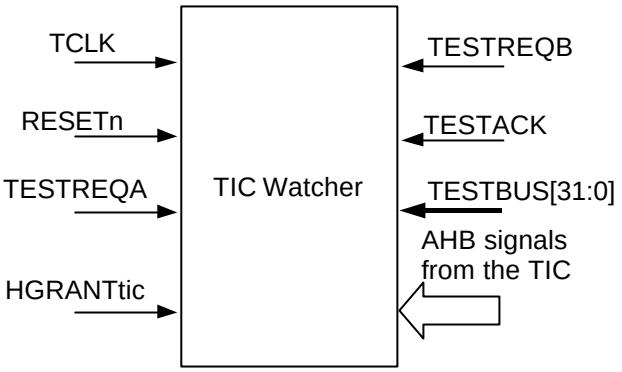


Figure 9.3-4 The TIC Watcher Block Diagram

The TIC Watcher receives the same vectors as the TIC does, from the Ticbox and generates AHB signals internally. It then compares the internally generated signals with the corresponding signals from the TIC and flags error messages whenever a protocol violation is detected on the bus.

Test Vectors

The test vectors for the integration testing of the SSMC are coded into separate C files for testing accesses to all the SSMC registers and for Integration testing. Make options are provided to run the tests together or individually.

10 INTEGRATION TESTS

The test vectors that are applied to the TICTalk testbenches are in the ***integration/bustest*** directory. The test vectors for integration testing are written in BusTalk and then converted into **.tif** format to be applied to the TICTalk integration testbench. The TIC validation test cases and the integration tests are used to test the SSMC's TIC and for integration testing respectively.

10.1 TIC Validation Test Cases

The TIC validation tests are used to test the functionality of the TIC in the SSMC. These tests are performed in the integration test environment rather than the standard Bustalk verification environment because of the need to stimulate the TIC interface through the ticbox, which is not available in the BusTalk verification environment.

10.1.1 Transfer tests

In this test, vectors are applied in different sequences with different HSIZE values. Tests are done in Address Increment enabled/disabled mode of the TIC. Address boundary check is done by applying writes and reads to the boundary address in the address increment enabled mode.

10.1.2 Response tests

In this test, the generic slave is configured to give out different responses (ERROR/SPLIT/RETRY) and the TIC performance is tested.

10.2 Integration Tests

10.2.1 AMBA signal integration test

The AHB signals connected to the AHB Slave ports of the Synchronous Static Memory Controller signals are tested with AHB transactions to the SSMC.

10.2.2 Non-AMBA intra-chip integration test

Integration Testing of Intra-chip Inputs

When Integration tests are run with the SSMC in a stand-alone test set-up:

Write a ‘1’ to the ITEN bit in the Control register. This selects the test path from the SSMCITIP[0] register bits to the internal signal.

Write different values to the SSMCITIP[0] register address and read the same register address to ensure that the value written is read out.

Repeat the above steps for other valid bits of SSMCITIP register.

When Integration tests are run with the SSMC as part of an integrated system:

Write a ‘0’ to the ITEN bit in the Control register. This selects the normal path from the external Intra-chip Input pin to the internal core logic.

Write different values into output register of source intra-chip device so as to toggle the Intra-chip signals. Read from the SSMCITIP[6:0] register to verify the value.

Integration Testing of intra-chip outputs

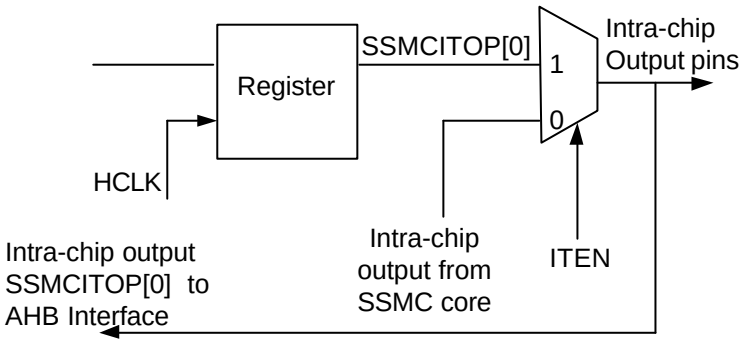


Figure 10.2-1 Output integration test harness Intra-chip

When Integration tests are run with the SSMC in a stand-alone test set-up:

Write a ‘1’ to the ITEN bit in the Control register. This selects the test path from the SSMCITOP[0] register bit to the intra-chip output signal.

Write a ‘1’ and ‘0’ to the SSMCITOP[0] register address and read the same register address to ensure that the value written is read out.

Repeat the above steps with SSMCITOP[1] register bit.

When Integration tests are run with the SSMC as part of an integrated system:

Write a ‘1’ to the ITEN bit in the Control register. This selects the test path from the SSMCITOP[0] register bit to the Intra-chip Outputs.

Write ‘1’ and ‘0’ into the SSMCITOP[0] register and verify it by reading from input register of target intra-chip device.

Repeat the above step with SSMCITOP[1] register bit.

10.2.3 Primary I/O signal integration test

The Primary inputs and outputs (external memory signals) can be tested external to the chip with some memory accesses through the SSMC.

10.3 Test Vector Generation in the Integration Test environment

A ***make sourcefilename*** (without the .c extension) in the ***integration/bustest*** directory will create **.tif** formatted and **.sim**-formatted vectors in the ***integration*/*bustest/invec*** directory. The makefile in ***integration*/*bustest*** directory can be referred for the make options.

The ***make all*** option can be used to create a test vector file including all the tests.

< End of document >